

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC
THÀNH PHỐ HỒ CHÍ MINH
Khoa Công Nghệ Thông Tin



MÔN: TRÍ TUỆ NHÂN TẠO

**ĐỀ TÀI: ỨNG DỤNG KỸ THUẬT HỌC MÁY
TRONG VIỆC DỰ ĐOÁN
BỆNH ĐỘT QUÝ TRONG NGÀNH Y TẾ**

Giảng viên hướng dẫn: ThS. Trần Thị Thanh Thảo

Nhóm: 06

1. Nguyễn Đoàn Vân Anh 23DH114201

Thành phố Hồ Chí Minh, ngày 03 tháng 08 năm 2025

MỤC LỤC

CHƯƠNG 1. MỞ ĐẦU	4
1. Lý do chọn đề tài	4
2. Mục tiêu	4
2.1 Mục tiêu đề tài	4
2.2 Mục tiêu thực hiện đề tài	4
3. Phương pháp nghiên cứu trong đề tài	5
3.1. Về mặt lý thuyết	5
3.2. Đối tượng nghiên cứu	6
3.3. Phạm vi nghiên cứu	7
4. Kết quả đạt được dự kiến	9
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	11
1. Tổng quan về học máy	11
1.1. Học máy (Machine Learning)	11
1.2 Học sâu (Deep Learning)	15
2. Giới thiệu về bài toán	17
2.1. Định nghĩa bài toán	17
2.2. Cách tiếp cận và phương pháp	20
2.3. Ứng dụng của bài toán	26
2.4. Các thách thức của bài toán	27
CHƯƠNG 3: THỰC NGHIỆM VÀ ĐÁNH GIÁ	29
1. Môi trường cần thiết	29
2. Mô tả dữ liệu (Dataset)	38
3. Xử lý dữ liệu	39
4. Xây dựng mô hình	41
5. Huấn luyện mô hình	47
6. Dự đoán kết quả	54
7. Đánh giá và nhận xét	58

CHƯƠNG 4. KẾT LUẬN	71
1. Kết luận	71
2. Hạn chế.....	71
3. Hướng phát triển.....	71
CHƯƠNG 5. TÀI LIỆU THAM KHẢO	73

CHƯƠNG 1. MỞ ĐẦU

1. Lý do chọn đề tài

Đột quỵ là một trong những nguyên nhân hàng đầu gây tử vong và tàn tật vĩnh viễn trên toàn cầu, gây ra gánh nặng lớn cho cá nhân, gia đình và hệ thống y tế. Việc dự đoán sớm nguy cơ đột quỵ có ý nghĩa sống còn trong việc triển khai các biện pháp phòng ngừa, can thiệp y tế kịp thời và quản lý bệnh hiệu quả, từ đó giảm thiểu tỷ lệ mắc bệnh, tử vong và cải thiện chất lượng cuộc sống cho bệnh nhân.

Trong bối cảnh kỷ nguyên số và sự phát triển vượt bậc của Trí tuệ nhân tạo (AI), đặc biệt là lĩnh vực Học máy (Machine Learning), khả năng phân tích lượng lớn dữ liệu y tế và khám phá các mẫu tiềm ẩn đã trở nên khả thi hơn bao giờ hết. Các phương pháp truyền thống thường chỉ dựa vào các chỉ số đơn lẻ hoặc thang điểm lâm sàng hạn chế. Tuy nhiên, việc áp dụng AI cho phép tích hợp và phân tích đa dạng các yếu tố nguy cơ (nhân khẩu học, bệnh nền, lối sống, các chỉ số sinh hóa) một cách toàn diện hơn, từ đó nâng cao độ chính xác trong việc đánh giá rủi ro.

Với tầm quan trọng cấp thiết của vấn đề sức khỏe cộng đồng và tiềm năng ứng dụng mạnh mẽ của Học máy, nhóm chúng tôi lựa chọn đề tài "Dự đoán nguy cơ đột quỵ dựa trên dữ liệu y tế và đặc điểm cá nhân". Đề tài này không chỉ cho phép nhóm áp dụng sâu rộng các kiến thức đã học trong môn Trí tuệ nhân tạo, đặc biệt là các thuật toán phân loại và kỹ thuật xử lý dữ liệu, góp phần vào công tác chăm sóc sức khỏe cộng đồng.

2. Mục tiêu

2.1 Mục tiêu đề tài

Đề tài hướng đến việc phân tích chuyên sâu các yếu tố ảnh hưởng đến khả năng mắc bệnh đột quỵ dựa trên dữ liệu sức khỏe của từng cá nhân, nhằm tìm ra những tác nhân chính góp phần làm tăng nguy cơ xảy ra đột quỵ. Thông qua việc áp dụng các mô hình học máy truyền thống như Logistic Regression, Random Forest và Support Vector Machine (SVM), nhóm nghiên cứu xây dựng một hệ thống dự đoán có khả năng phân loại người có nguy cơ đột quỵ với độ chính xác cao và thời gian xử lý nhanh chóng. Mục tiêu của hệ thống không chỉ là hỗ trợ việc phát hiện sớm các ca có dấu hiệu nguy hiểm, mà còn giúp người dùng hiểu rõ hơn về tầm quan trọng của các chỉ số sức khỏe như tuổi tác, huyết áp, lượng đường trong máu, chỉ số BMI, thói quen hút thuốc, tiền sử bệnh tim mạch,... Những thông tin này sẽ được minh bạch hóa thông qua các kỹ thuật giải thích mô hình như SHAP, LIME và ELI5 nhằm tăng tính tin cậy và dễ hiểu cho người dùng cuối. Từ đó, đề tài mong muốn góp phần nâng cao nhận thức cộng đồng về phòng tránh đột quỵ và xây dựng một công cụ hữu ích, dễ sử dụng cho cả người dân và chuyên gia y tế trong việc theo dõi, cảnh báo sớm nguy cơ bệnh tật.

2.2 Mục tiêu thực hiện đề tài

Mục tiêu thực hiện:

- **Xây dựng mô hình học máy** nhằm dự đoán nguy cơ đột quỵ dựa trên dữ liệu sức khỏe cá nhân của người bệnh.
- **Áp dụng các thuật toán truyền thống** như Logistic Regression, Random Forest và Support Vector Machine để so sánh hiệu quả dự đoán.
- **Tối ưu hóa mô hình** để đạt được độ chính xác cao, đảm bảo khả năng dự đoán nhanh chóng và ổn định.
- **Phân tích các yếu tố ảnh hưởng đến đột quỵ**, từ đó xác định những tác nhân chính có tác động mạnh đến nguy cơ mắc bệnh.
- **Sử dụng các phương pháp giải thích mô hình** như SHAP, LIME và ELI5 để tăng tính minh bạch, giúp người dùng hiểu rõ nguyên nhân dự đoán.
- **Nâng cao nhận thức cộng đồng** về tầm quan trọng của các chỉ số sức khỏe và việc phòng chống đột quỵ từ sớm.

3. Phương pháp nghiên cứu trong đề tài

3.1. Về mặt lý thuyết

Các thuật toán phân loại (Classification Algorithms): Các thuật toán này được sử dụng để xây dựng mô hình dự đoán đầu ra là một trong hai lớp: "Có đột quỵ" (1) hoặc "Không đột quỵ" (0).

- **Hồi quy Logistic (Logistic Regression):** Là một thuật toán phân loại tuyến tính, sử dụng hàm sigmoid để ánh xạ đầu ra của một tổ hợp tuyến tính các đặc trưng vào xác suất thuộc về một lớp nhất định. Được sử dụng như một mô hình cơ sở để so sánh hiệu suất.
- **Rừng ngẫu nhiên (Random Forest):** Là một thuật toán học ensemble, xây dựng từ nhiều cây quyết định độc lập. Mỗi cây được huấn luyện trên một tập con dữ liệu và đặc trưng được chọn ngẫu nhiên. Kết quả cuối cùng là kết quả bỏ phiếu của các cây. Mô hình này thường cho hiệu suất cao hơn, ít bị overfitting hơn so với một cây quyết định đơn lẻ. Có khả năng đánh giá độ quan trọng của các đặc trưng.
- **Máy vectơ hỗ trợ (Support Vector Machine - SVM):** Tìm kiếm siêu phẳng tối ưu trong không gian đa chiều để phân chia các lớp dữ liệu, tối đa hóa khoảng cách biên. Có thể sử dụng các hàm nhân (kernel functions) để xử lý các bài toán phi tuyến tính.

Các kỹ thuật tiền xử lý dữ liệu (Data Preprocessing Techniques): Các kỹ thuật này nhằm làm sạch, chuyển đổi và chuẩn bị dữ liệu thô để chúng phù hợp với yêu cầu của các thuật toán Học máy.

- **Xử lý dữ liệu thiếu (Missing Values Imputation):** Các phương pháp để điền vào (hoặc loại bỏ) các giá trị bị thiếu trong tập dữ liệu (ví dụ: cột bmi) bằng cách điền bằng giá trị trung bình (mean), trung vị (median) hoặc mode.
- **Chuẩn hóa/Co giãn dữ liệu (Scaling/Normalization):** Biến đổi các giá trị của một đặc trưng để chúng nằm trong một phạm vi nhất định hoặc có cùng thang đo. Các phương pháp phổ biến là StandardScaler và MinMaxScaler.

- **Mã hóa biến phân loại (Categorical Encoding):** Chuyển đổi các đặc trưng phân loại (dạng chuỗi hoặc số rời rạc) thành dạng số mà các thuật toán Học máy có thể xử lý.
 - **One-Hot Encoding:** Tạo ra các cột nhị phân mới cho mỗi danh mục duy nhất trong biến gốc.
 - **Label Encoding:** Gán một số nguyên duy nhất cho mỗi danh mục.
- **Xử lý mất cân bằng lớp (Class Imbalance Handling):** Trong các bài toán y tế, lớp dương tính thường là thiểu số. Các kỹ thuật như **Oversampling (ví dụ: SMOTE)** hoặc Undersampling sẽ được áp dụng để cân bằng lại tỷ lệ lớp trong tập huấn luyện.

Các độ đo đánh giá mô hình phân loại (Classification Evaluation Metrics): Để đánh giá hiệu suất của mô hình, cần sử dụng các độ đo phù hợp, đặc biệt là với dữ liệu mất cân bằng.

- **Độ chính xác (Accuracy):** Tỷ lệ các dự đoán đúng trên tổng số dự đoán.
- **Độ chính xác dương (Precision):** Tỷ lệ các trường hợp dự đoán là dương tính (có đột quỵ) thực sự là dương tính. Công thức: $TP / (TP + FP)$ (True Positives / (True Positives + False Positives)).
- **Độ nhạy/Độ phủ (Recall/Sensitivity):** Tỷ lệ các trường hợp dương tính thực sự được mô hình phát hiện. Công thức: $TP / (TP + FN)$ (True Positives / (True Positives + False Negatives)).
- **Điểm F1 (F1-score):** Là trung bình điều hòa của Precision và Recall. Cung cấp một cái nhìn cân bằng về hiệu suất của mô hình. Công thức: $2 * (Precision * Recall) / (Precision + Recall)$.
- **Đường cong ROC và AUC (Receiver Operating Characteristic Curve và Area Under the Curve):** Đường cong ROC biểu thị khả năng phân loại của mô hình ở các ngưỡng phân loại khác nhau. AUC là diện tích dưới đường cong ROC, cho biết khả năng phân biệt giữa các lớp dương và âm của mô hình một cách tổng quát.

Kiểm định chéo (Cross-validation): Một kỹ thuật đánh giá mô hình bằng cách chia tập dữ liệu thành nhiều phần (k-folds). Mô hình được huấn luyện và kiểm tra lặp đi lặp lại trên các tổ hợp khác nhau của các phần này, cung cấp ước tính hiệu suất đáng tin cậy hơn và giảm thiểu rủi ro overfitting.

Siêu tham số (Hyperparameters): Là các thông số được thiết lập trước khi quá trình huấn luyện mô hình bắt đầu, không được học từ dữ liệu. Chúng kiểm soát quá trình học như độ sâu của cây quyết định, hệ số điều chuẩn (regularization), hoặc tốc độ học. Việc lựa chọn siêu tham số phù hợp thông qua các kỹ thuật như Grid Search hoặc Random Search giúp cải thiện hiệu suất mô hình và giảm thiểu nguy cơ overfitting hoặc underfitting.

XAI (Trí tuệ nhân tạo giải thích được): Là tập hợp các kỹ thuật giúp hiểu rõ cách mô hình học máy đưa ra quyết định. Thay vì chỉ biết kết quả đầu ra, XAI cung cấp các giải thích chi tiết về vai trò của từng đặc trưng (feature) trong quá trình dự đoán, từ đó tăng độ tin cậy, minh bạch và khả năng ứng dụng của mô hình trong thực tế, đặc biệt trong các lĩnh vực quan trọng như y tế và tài chính.

3.2. Đối tượng nghiên cứu

Đối tượng nghiên cứu chính của dự án là **dự đoán nguy cơ đột quỵ não ở giai đoạn sớm**. Nghiên cứu tập trung vào việc phát triển một mô hình dự báo đột quỵ chính xác, sử dụng các kỹ thuật Học máy (Machine Learning) kết hợp với các phương pháp Trí tuệ nhân tạo có khả năng giải thích (Explainable AI - XAI).

Lý do lựa chọn đối tượng nghiên cứu: Việc lựa chọn dự đoán đột quỵ làm đối tượng nghiên cứu xuất phát từ tầm quan trọng to lớn của nó trong y tế cộng đồng:

- **Mức độ nghiêm trọng của đột quỵ:** Đột quỵ là một trong những nguyên nhân hàng đầu gây tử vong và tàn tật nặng nề trên toàn cầu, gây ra gánh nặng lớn cho cá nhân, gia đình và hệ thống y tế.
- **Vai trò của chẩn đoán sớm:** Khả năng dự đoán nguy cơ đột quỵ sớm cho phép can thiệp y tế kịp thời, bao gồm thay đổi lối sống hoặc điều trị phòng ngừa, từ đó có thể giảm đáng kể tỷ lệ tử vong và mức độ khuyết tật, cải thiện chất lượng cuộc sống cho bệnh nhân.
- **Nhu cầu về tính minh bạch và tin cậy:** Trong lĩnh vực y tế, việc các mô hình "hộp đen" đưa ra dự đoán mà không có giải thích rõ ràng là một rào cản lớn. Việc kết hợp XAI giúp làm cho quá trình dự đoán trở nên minh bạch và dễ hiểu hơn, xây dựng lòng tin và tạo điều kiện cho các chuyên gia y tế đưa ra quyết định lâm sàng tốt hơn.
- **Xử lý thách thức dữ liệu thực tế:** Nghiên cứu cũng tập trung vào việc giải quyết các vấn đề dữ liệu thường gặp trong y tế như mất cân bằng lớp (số lượng ca đột quỵ thường ít hơn rất nhiều so với không đột quỵ) và xác định chính xác các yếu tố nguy cơ quan trọng, nâng cao độ tin cậy của mô hình trong môi trường thực tế.

Đối tượng sử dụng/ứng dụng: Kết quả của nghiên cứu này hướng tới phục vụ các đối tượng chính sau:

- **Chuyên gia y tế và Bác sĩ:** Cung cấp một công cụ hỗ trợ đáng tin cậy để đánh giá nguy cơ đột quỵ của bệnh nhân, giúp họ đưa ra các quyết định lâm sàng, chẩn đoán, và kế hoạch điều trị hoặc phòng ngừa phù hợp và kịp thời. Khả năng giải thích của mô hình đặc biệt hữu ích để hiểu lý do đằng sau mỗi dự đoán.
- **Cán bộ y tế dự phòng và Chính sách y tế:** Hỗ trợ trong việc xác định các nhóm dân số có nguy cơ cao để triển khai các chương trình sàng lọc và can thiệp y tế cộng đồng hiệu quả, phân bổ nguồn lực y tế một cách tối ưu.
- **Bệnh nhân và cộng đồng:** Nâng cao nhận thức về các yếu tố nguy cơ đột quỵ, khuyến khích thay đổi lối sống lành mạnh và chủ động tìm kiếm sự chăm sóc y tế khi cần thiết.

Các nhà nghiên cứu trong lĩnh vực Học máy và Y sinh: Cung cấp một nghiên cứu điển hình về việc áp dụng thành công Học máy và XAI trong chẩn đoán y tế, mở ra hướng cho các nghiên cứu tiếp theo.

3.3. Phạm vi nghiên cứu

Phạm vi dữ liệu:

- Nghiên cứu tập trung vào tập dữ liệu chăm sóc sức khỏe bệnh nhân, bao gồm các thông tin về nhân khẩu học, tình trạng sức khỏe (huyết áp, bệnh tim, đường huyết, BMI), và lối sống (hút thuốc, loại hình công việc, cư trú, tình trạng hôn nhân).

Nội dung:

- Dự đoán nguy cơ đột quỵ não của một cá nhân (phân loại nhị phân: Có nguy cơ / Không có nguy cơ), sử dụng các đặc trưng được cung cấp trong tập dữ liệu.
- Nghiên cứu tập trung vào các đặc trưng nội tại của bệnh nhân. Các yếu tố bên ngoài như yếu tố môi trường, dữ liệu lâm sàng chỉ tiết thời gian thực, hoặc thông tin di truyền không được xem xét trong phạm vi này.

Phạm vi ứng dụng:

- Nghiên cứu này mang tính học thuật và thử nghiệm trong môi trường mô phỏng. Kết quả và mô hình được phát triển nhằm mục đích nghiên cứu và kiểm định, chưa được triển khai thực tế như một công cụ chẩn đoán lâm sàng trực tiếp.

Mục đích:

- Dự đoán nguy cơ đột quỵ với độ chính xác cao, đồng thời tăng cường khả năng giải thích (Explainable AI - XAI) để hiểu rõ các yếu tố đóng góp vào dự đoán.
- Đánh giá hiệu quả của các kỹ thuật học máy (Hồi quy Logistic, Rừng ngẫu nhiên, SVM) trong bài toán phân loại nguy cơ đột quỵ.
- Góp phần thúc đẩy ứng dụng học máy trong y tế dự phòng, đặc biệt trong lĩnh vực dự đoán đột quỵ, tạo nền tảng cho các nghiên cứu và ứng dụng mở rộng trong tương lai.

Tại sao chọn phạm vi này?

- Dữ liệu có sẵn chứa đủ các đặc trưng liên quan đến nguy cơ đột quỵ, cho phép xây dựng và huấn luyện mô hình phân loại.
- Vấn đề đột quỵ là một thách thức y tế lớn, đòi hỏi các giải pháp dự phòng hiệu quả, phù hợp với mục tiêu ứng dụng học máy để tạo tác động xã hội.
- Sự phức tạp của mối quan hệ giữa các yếu tố nguy cơ và đột quỵ làm cho việc áp dụng học máy trở nên cần thiết, đồng thời nhu cầu về tính minh bạch trong y tế thúc đẩy việc tích hợp XAI.

Lợi ích của đề tài:

- Cung cấp một nghiên cứu thực nghiệm về ứng dụng học máy và XAI trong dự đoán nguy cơ đột quỵ, làm tài liệu tham khảo có giá trị cho các nghiên cứu học thuật và ứng dụng y tế sau này.
- Đề xuất một quy trình toàn diện từ tiền xử lý dữ liệu (bao gồm xử lý mất cân bằng lớp) đến xây dựng mô hình và giải thích kết quả, có thể được áp dụng cho các bài toán phân loại y tế tương tự.

Hạn chế của phạm vi nghiên cứu:

- **Giới hạn dữ liệu:** Chỉ sử dụng tập dữ liệu có sẵn với các đặc trưng đã định. Không tích hợp dữ liệu lâm sàng thời gian thực, kết quả xét nghiệm chi tiết, hoặc hình ảnh y tế, có thể ảnh hưởng đến độ chính xác và khả năng cá nhân hóa dự đoán.
- **Tính tổng quát:** Kết quả và mô hình có thể chưa hoàn toàn tổng quát cho các quần thể bệnh nhân đa dạng khác do sự giới hạn của tập dữ liệu huấn luyện.
- **Không phải công cụ chẩn đoán:** Mô hình hiện tại mang tính dự đoán nguy cơ, không phải là một công cụ chẩn đoán y tế chính thức và cần được xác nhận bởi chuyên gia y tế.
- **Phụ thuộc chất lượng dữ liệu:** Hiệu suất của mô hình phụ thuộc lớn vào chất lượng dữ liệu đầu vào và quá trình tiền xử lý, đòi hỏi sự tối ưu hóa cẩn thận.

4. Kết quả đạt được dự kiến

Về lý thuyết:

- **Xác định mô hình tối ưu cho dữ liệu mất cân bằng:** Về mặt lý thuyết, dự án chứng minh rằng trong bối cảnh dữ liệu y tế mất cân bằng (như dự đoán đột quỵ), việc lựa chọn mô hình không chỉ dựa vào độ chính xác (accuracy) mà cần ưu tiên các chỉ số như recall và F1-score để giảm thiểu sai sót bỏ sót ca bệnh (Type II error). Cụ thể, Logistic Regression sau khi tinh chỉnh đã được chọn làm mô hình chính thức nhờ hiệu suất tốt về recall và F1-score, cung cấp một phương pháp lý thuyết vững chắc cho các bài toán phân loại nhị phân không cân bằng.
- **Ứng dụng thực tế của XAI trong giải thích mô hình:** Dự án minh họa việc áp dụng SHAP, LIME và ELI5 để diễn giải các quyết định của mô hình Học máy, làm tăng tính minh bạch và khả năng giải thích của mô hình. Điều này củng cố lý thuyết về tầm quan trọng của XAI trong các hệ thống AI đáng tin cậy, đặc biệt khi các mô hình trở nên phức tạp hơn.
- **Phương pháp tiền xử lý dữ liệu thích hợp:** Việc sử dụng DecisionTreeRegressor trong pipeline để điền giá trị thiếu cho bmi bằng cách học từ các đặc trưng khác (age, gender) cung cấp một ví dụ cụ thể về lý thuyết imputation dựa trên mô hình, giúp cải thiện chất lượng dữ liệu đầu vào và độ tin cậy của mô hình.

Về ứng dụng:

- **Hỗ trợ ra quyết định lâm sàng:** Nghiên cứu cung cấp những hiểu biết sơ bộ có thể hỗ trợ phát triển các công cụ trong tương lai để hỗ trợ các bác sĩ trong việc quản lý bệnh nhân. Các mô hình dự đoán, đặc biệt là khi được kết hợp với khả năng giải thích của XAI, có tiềm năng trở thành công cụ hỗ trợ sàng lọc và đánh giá nguy cơ đột quỵ sớm cho bệnh nhân.
- **Cải thiện chẩn đoán và quản lý bệnh nhân:** Bằng cách cung cấp khả năng dự đoán sớm và hiểu rõ hơn về các yếu tố thúc đẩy dự đoán, mô hình có thể góp phần cải thiện chẩn đoán kịp thời và quản lý điều trị hiệu quả hơn cho bệnh nhân, từ đó có thể giảm tỷ lệ tử vong và mức độ tàn tật.

- **Nền tảng cho công cụ hỗ trợ y tế tương lai:** Mô hình Học máy được phát triển trong nghiên cứu này, cùng với khả năng giải thích của nó, có thể đóng vai trò là nền tảng ban đầu cho việc phát triển các công cụ hỗ trợ quyết định trong lĩnh vực y tế trong tương lai, mặc dù cần thêm xác nhận lâm sàng và đánh giá trong môi trường thực tế.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

1. Tổng quan về học máy

1.1. Học máy (Machine Learning)

1.1.1. Giới thiệu về học máy

Học máy (Machine Learning) là một nhánh quan trọng của lĩnh vực trí tuệ nhân tạo (Artificial Intelligence - AI). Nó đại diện cho một bước tiến đáng kể trong cách chúng ta xây dựng và vận hành các hệ thống máy tính. Thay vì phải lập trình từng bước một cách cứng nhắc thông qua các thuật toán cố định, học máy cho phép các chương trình có khả năng tự động hóa và tối ưu hóa hoạt động của mình dựa trên việc phân tích dữ liệu hoặc thông qua quá trình tích lũy kinh nghiệm. Điều này mang lại sự linh hoạt và khả năng thích ứng vượt trội cho hệ thống.

Mục đích cốt lõi của Học máy:

Mục tiêu trung tâm của học máy là trang bị cho máy tính khả năng biến đổi hoặc thích nghi các hành vi của nó một cách liên tục. Sự thích nghi này nhằm mục đích làm cho các hành vi đó ngày càng trở nên chính xác hơn. Mức độ chính xác được đo lường dựa trên việc các hành vi được lựa chọn thể hiện đúng các hành vi mong muốn.

Thông qua quá trình học hỏi từ dữ liệu, máy tính có được khả năng nhận dạng các mẫu (pattern) một cách chính xác. Chính vì vậy, học máy đóng vai trò hỗ trợ vô cùng đắc lực cho lĩnh vực nhận dạng mẫu (pattern recognition). Điều quan trọng cần nhấn mạnh là, nếu không có học máy, máy tính sẽ không thể thực hiện được việc nhận dạng các mẫu. Trong bối cảnh của nhận dạng mẫu, thuật ngữ "mẫu" (pattern) được dùng để chỉ tất cả những đối tượng cụ thể mà hệ thống cần phân loại (classify).

Khi nào cần áp dụng Học máy?

Học máy trở thành một công cụ không thể thiếu và đặc biệt hiệu quả trong những tình huống mà các phương pháp lập trình truyền thống gặp hạn chế:

- **Tri thức con người không hoặc chưa có:** Trong nhiều trường hợp, con người không có đủ kiến thức hoặc chưa thể xây dựng được một thuật toán rõ ràng, một bộ quy tắc cụ thể để giải quyết một vấn đề phức tạp. Học máy có thể tự động khám phá những quy luật ẩn từ dữ liệu.
- **Con người không thể giải thích tri thức của mình:** Có những tác vụ mà con người có thể thực hiện một cách thành thạo, nhưng lại gặp khó khăn trong việc diễn tả rõ ràng các bước hay logic chi tiết đằng sau hành động đó (ví dụ: nhận diện khuôn mặt, lái xe). Học máy có thể học hỏi từ các ví dụ mà không cần sự giải thích tường minh từ con người.
- **Giải pháp cần thay đổi liên tục:** Trong các môi trường thực tế năng động, nơi dữ liệu, xu hướng hoặc các quy luật cơ bản của vấn đề thay đổi không ngừng. Một giải pháp lập

trình tính sẽ nhanh chóng lỗi thời. Học máy cho phép hệ thống tự động thích nghi và cập nhật giải pháp một cách liên tục dựa trên dữ liệu mới.

- **Giải pháp cho các trường hợp đặc biệt (cá nhân hóa):** Khi mục tiêu là cung cấp các giải pháp cá nhân hóa cho từng người dùng riêng lẻ hoặc từng ngữ cảnh cụ thể. Việc lập trình thủ công cho hàng triệu trường hợp đặc biệt là điều không khả thi. Học máy có thể học hỏi từ hành vi và sở thích của từng cá nhân để đưa ra các gợi ý hoặc hành động phù hợp.

1.1.2. Các phương pháp của học máy

Học máy được phân loại thành bốn phương pháp chính, mỗi phương pháp có cách tiếp cận và ứng dụng riêng biệt:

- Học có giám sát (Supervised Learning)
- Học không giám sát (Unsupervised Learning)
- Học tăng cường (Reinforcement Learning)
- Học chuyển giao (Transfer Learning)

a) Học có giám sát (Supervised Learning)

Định nghĩa: Học có giám sát là phương pháp học máy trong đó mô hình được huấn luyện trên một tập dữ liệu có nhãn. Điều này có nghĩa là mỗi mẫu dữ liệu đầu vào (input) đi kèm với một đầu ra (output) mong muốn. Mục tiêu là xây dựng một mô hình có thể dự đoán chính xác đầu ra cho dữ liệu mới dựa trên các mẫu đã học.

Quy trình:

- **Dữ liệu huấn luyện:** Tập dữ liệu bao gồm các cặp (input, output). Ví dụ: ảnh (input) và nhãn của ảnh (cat/dog - output).
- **Huấn luyện:** Mô hình học cách ánh xạ từ input sang output bằng cách tối ưu hóa một hàm mất mát (loss function), thường sử dụng thuật toán như Gradient Descent.
- **Dự đoán:** Sau khi huấn luyện, mô hình dự đoán đầu ra cho dữ liệu mới.

Các loại bài toán:

- **Phân loại (Classification):** Dự đoán nhãn rời rạc. Ví dụ: phân loại email là spam hay không spam. Các thuật toán phổ biến: Logistic Regression, Support Vector Machine (SVM), Random Forest, Neural Networks.
- **Hồi quy (Regression):** Dự đoán giá trị liên tục. Ví dụ: dự đoán giá nhà. Các thuật toán phổ biến: Linear Regression, Polynomial Regression, Gradient Boosting.

Ưu điểm:

- Độ chính xác cao khi có dữ liệu nhãn chất lượng.
- Dễ áp dụng cho nhiều bài toán thực tế như nhận diện hình ảnh, dự đoán giá cả.

Nhược điểm:

- Yêu cầu lượng lớn dữ liệu có nhãn, việc thu thập và gán nhãn dữ liệu có thể tốn kém.
- Có thể gặp vấn đề khi dữ liệu không cân bằng hoặc nhiễu.

Ví dụ ứng dụng:

- Phân loại bệnh dựa trên hình ảnh y khoa (X-quang).
- Dự đoán doanh số bán hàng dựa trên dữ liệu lịch sử.

b) Học không giám sát (Unsupervised Learning)

Định nghĩa: Học không giám sát là phương pháp học máy trong đó mô hình làm việc với dữ liệu không có nhãn. Mục tiêu là tìm ra cấu trúc, mẫu hoặc đặc điểm ẩn trong dữ liệu.

Quy trình:

- **Dữ liệu:** Chỉ có dữ liệu đầu vào, không có đầu ra được cung cấp.
- **Phân tích:** Mô hình phân tích dữ liệu để tìm ra các mẫu, nhóm, hoặc biểu diễn dữ liệu mới.
- **Kết quả:** Các cụm (clusters), giảm chiều dữ liệu, hoặc biểu diễn đặc trưng.

Các loại bài toán:

- **Phân cụm (Clustering):** Nhóm các điểm dữ liệu tương tự nhau thành các cụm. Thuật toán phổ biến: K-means, DBSCAN, Hierarchical Clustering.
- **Giảm chiều (Dimensionality Reduction):** Giảm số lượng đặc trưng để đơn giản hóa dữ liệu mà vẫn giữ được thông tin quan trọng. Thuật toán phổ biến: Principal Component Analysis (PCA), t-SNE.
- **Phát hiện bất thường (Anomaly Detection):** Xác định các điểm dữ liệu khác biệt so với phần lớn dữ liệu.

Ưu điểm:

- Không cần dữ liệu có nhãn, phù hợp khi dữ liệu thô dồi dào.
- Khám phá các mẫu ẩn mà không cần giả định trước.

Nhược điểm:

- Kết quả khó đánh giá do thiếu nhãn tham chiếu.
- Phụ thuộc nhiều vào chất lượng dữ liệu và lựa chọn tham số.

Ví dụ ứng dụng:

- Phân khúc khách hàng trong marketing (nhóm khách hàng theo hành vi).
- Nén dữ liệu hình ảnh hoặc âm thanh.

c) Học tăng cường (Reinforcement Learning)

Định nghĩa: Học tăng cường là phương pháp học máy trong đó một tác nhân (agent) học cách đưa ra quyết định bằng cách thử và sai trong một môi trường, nhằm tối ưu hóa một phần thưởng tích lũy. Tác nhân tương tác với môi trường, nhận phản hồi (phần thưởng hoặc phạt) và điều chỉnh hành động.

Quy trình:

- **Môi trường:** Một hệ thống mà tác nhân tương tác (ví dụ: trò chơi, robot).
- **Tác nhân:** Thực hiện hành động (action) dựa trên trạng thái (state) của môi trường.
- **Phần thưởng:** Phản hồi từ môi trường, dùng để đánh giá hành động.
- **Học:** Tác nhân tối ưu hóa chính sách (policy) để chọn hành động tốt nhất, thường sử dụng các thuật toán như Q-Learning, Deep Q-Networks (DQN), hoặc Policy Gradient.

Các thành phần chính:

- **Trạng thái (State):** Tình trạng hiện tại của môi trường.
- **Hành động (Action):** Các lựa chọn mà tác nhân có thể thực hiện.
- **Phần thưởng (Reward):** Giá trị số phản ánh mức độ tốt/xấu của hành động.
- **Chính sách (Policy):** Chiến lược chọn hành động dựa trên trạng thái.

Ưu điểm:

- Phù hợp với các bài toán ra quyết định liên tục, trong môi trường động.
- Có thể học các chiến lược phức tạp mà không cần dữ liệu huấn luyện có sẵn.

Nhược điểm:

- Cần thời gian dài để huấn luyện, đặc biệt trong môi trường phức tạp.
- Khó khăn trong việc thiết kế hàm phần thưởng phù hợp.

d) Học chuyển giao (Transfer Learning)

Định nghĩa: Học chuyển giao là phương pháp sử dụng kiến thức đã học từ một bài toán (thường là bài toán lớn với dữ liệu dồi dào) để áp dụng cho một bài toán khác có liên quan, thường với dữ liệu hạn chế. Phương pháp này đặc biệt phổ biến trong học sâu.

Quy trình:

- **Mô hình tiền huấn luyện (Pre-trained Model):** Sử dụng mô hình đã được huấn luyện trên tập dữ liệu lớn (ví dụ: ImageNet cho hình ảnh, BERT cho văn bản).
- **Tinh chỉnh (Fine-tuning):** Điều chỉnh mô hình tiền huấn luyện cho bài toán cụ thể bằng cách huấn luyện thêm trên tập dữ liệu nhỏ hơn.
- **Áp dụng:** Sử dụng mô hình để dự đoán hoặc phân loại trên bài toán mới.

Các bước thực hiện:

- Chọn mô hình tiền huấn luyện phù hợp (ví dụ: ResNet, VGG, hoặc BERT).
- Thay thế hoặc tinh chỉnh các tầng cuối của mô hình để phù hợp với bài toán mới.
- Huấn luyện mô hình trên tập dữ liệu mới với tốc độ học (learning rate) thấp để tránh mất kiến thức đã học.

Ưu điểm:

- Giảm thời gian huấn luyện và yêu cầu dữ liệu cho bài toán mới.
- Hiệu quả cao với các bài toán có dữ liệu hạn chế.
- Tận dụng được các đặc trưng mạnh mẽ từ mô hình tiền huấn luyện.

Nhược điểm:

- Phụ thuộc vào sự tương đồng giữa bài toán gốc và bài toán mới.
- Có thể cần tinh chỉnh phức tạp để đạt hiệu quả tối ưu.

1.2 Học sâu (Deep Learning)

1.2.1. Giới thiệu về học sâu

Học sâu (Deep Learning) là một lĩnh vực con nổi bật của học máy, tập trung vào việc mô phỏng cách bộ não con người học hỏi và xử lý thông tin. Đặc điểm cốt lõi của học sâu là sử dụng các mô hình với **nhiều tầng (layer)** xử lý thông tin, tạo nên một **kiến trúc phân cấp** để học các đặc trưng (feature learning) và phân lớp mẫu.

Đặc điểm và Nguyên lý hoạt động:

- **Kiến trúc Đa tầng và Phi tuyến:** Một mô hình học sâu bao gồm nhiều tầng kết nối với nhau, trong đó mỗi tầng áp dụng các **tác vụ phi tuyến (non-linear operation)** tại các đơn vị xuất của nó. Khả năng xử lý phi tuyến tính này cho phép các mô hình học sâu nắm bắt được các mối quan hệ phức tạp và phi tuyến trong dữ liệu mà các mô hình học máy truyền thống khó có thể làm được.
- **Học đặc trưng phân cấp (Hierarchical Feature Learning):** Ý tưởng chính đằng sau học sâu là có những **đơn vị phát hiện đặc trưng (feature detector unit)** tại mỗi tầng. Các đơn vị này làm nhiệm vụ rút trích dần và tinh chế những đặc trưng tinh tế và bất biến (invariant features) từ những dữ liệu thô ban đầu.
 - **Các tầng thấp (low-level layers)** trong kiến trúc học sâu thường đảm nhiệm việc rút trích những đặc trưng đơn giản và cơ bản nhất từ dữ liệu đầu vào (ví dụ: các cạnh, góc, hoặc kết cấu trong hình ảnh; các âm vị cơ bản trong âm thanh).
 - Những đặc trưng đơn giản này sau đó được nạp qua **những tầng cao hơn (higher-level layers)**. Tại đây, chúng được kết hợp và tinh chỉnh để phát hiện những đặc trưng phức tạp hơn, mang ý nghĩa trừu tượng hơn (ví dụ: từ các cạnh và góc, các tầng cao hơn có thể nhận diện các bộ phận của khuôn mặt như mắt, mũi; hoặc từ các âm vị, nhận diện từ ngữ). Quá trình phân cấp này cho phép mô

hình tự động học được các biểu diễn dữ liệu ngày càng phức tạp và có ý nghĩa hơn mà không cần sự can thiệp thủ công của con người trong việc thiết kế đặc trưng.

Tầm quan trọng:

- Sự ra đời của học sâu đã tạo ra những bước đột phá đáng kể trong nhiều lĩnh vực, đặc biệt là trong xử lý ngôn ngữ tự nhiên (NLP), thị giác máy tính (Computer Vision), nhận dạng giọng nói, và hệ thống khuyến nghị, nơi dữ liệu thường ở dạng không có cấu trúc và có tính phức tạp cao. Khả năng tự động học và trích xuất các đặc trưng quan trọng từ dữ liệu thô đã làm cho học sâu trở thành một công cụ mạnh mẽ và linh hoạt.

1.2.2. Một số đặc điểm của học sâu

a. Mạng nơ-ron sâu (Deep Neural Networks - DNNs)

* Nhiều lớp ẩn (Hidden Layers):

- Học sâu sử dụng các mạng nơ-ron với nhiều lớp (từ vài lớp đến hàng trăm lớp), khác với mạng nơ-ron đơn giản (shallow networks) chỉ có 1–2 lớp ẩn.
- Ví dụ: ResNet (CNN cho xử lý ảnh) có thể có 152 lớp.
- Mỗi lớp học các đặc trưng từ mức thấp (cạnh, màu sắc) đến mức cao (hình dạng, đối tượng phức tạp).

* Tính phi tuyến (Non-linearity): Các hàm kích hoạt (activation functions) như ReLU, Sigmoid, Tanh giúp mô hình học được biểu diễn phi tuyến trong dữ liệu.

b. Tự động trích xuất đặc trưng (Automatic Feature Extraction)

- Không cần kỹ thuật trích chọn đặc trưng thủ công (như trong Học máy truyền thống):

Ví dụ: Với ảnh, thay vì dùng thuật toán như SIFT/HOG để trích đặc trưng, CNN tự động học các bộ lọc (filters) để phát hiện cạnh, kết cấu, hình dạng.

- Khả năng tổng quát hóa (Generalization): Mô hình học sâu có thể áp dụng cho nhiều bài toán khác nhau mà không cần thiết kế lại quy trình trích đặc trưng.

c. Xử lý dữ liệu có cấu trúc phức tạp

- Dữ liệu không gian (ảnh, video): CNN sử dụng phép tích chập (convolution) để bảo toàn thông tin không gian và giảm tham số.
- Dữ liệu chuỗi (văn bản, âm thanh): RNN/LSTM/Transformer xử lý phụ thuộc dài hạn (long-term dependencies) nhờ cơ chế bộ nhớ (memory) hoặc tự chú ý (self-attention).
- Dữ liệu không có cấu trúc (unstructured data): Học sâu hiệu quả hơn so với học máy truyền thống khi làm việc với văn bản thô, ảnh gốc, âm thanh.

Ưu điểm:

- Khả năng xử lý dữ liệu phức tạp và phi cấu trúc.

- Hiệu quả cao trong các bài toán quy mô lớn với dữ liệu dồi dào.
- Giảm sự phụ thuộc vào kỹ thuật thủ công trong việc thiết kế đặc trưng.

Nhược điểm:

- Yêu cầu lượng dữ liệu lớn và tài nguyên tính toán cao.
- Quá trình huấn luyện phức tạp, tốn thời gian và khó diễn giải (black-box nature).
- Dễ gặp vấn đề như quá khớp (overfitting) nếu không có đủ dữ liệu hoặc kỹ thuật điều chỉnh phù hợp.

2. Giới thiệu về bài toán

2.1. Định nghĩa bài toán

Bài toán trọng tâm của dự án là dự đoán nguy cơ đột quỵ não (stroke prediction) ở giai đoạn sớm. Đây được định nghĩa là một bài toán phân loại nhị phân (binary classification). Mục tiêu cụ thể là phân loại một cá nhân vào một trong hai nhóm: có khả năng bị đột quỵ (lớp dương tính) hoặc không có khả năng bị đột quỵ (lớp âm tính), dựa trên một bộ các đặc trưng sức khỏe và lối sống có sẵn từ dữ liệu.

Bài toán này đặc biệt quan trọng vì đột quỵ là một trong những nguyên nhân hàng đầu gây tử vong và tàn tật nặng nề, đặc biệt ở người lớn tuổi. Việc chẩn đoán sớm và điều trị kịp thời có thể giúp giảm đáng kể tỷ lệ tử vong và mức độ khuyết tật nghiêm trọng do đột quỵ gây ra. Do đó, việc xây dựng một mô hình dự đoán chính xác và đáng tin cậy là cực kỳ cần thiết.

Đầu vào của bài toán: Dữ liệu lịch sử bệnh nhân được thu thập, bao gồm các thông tin cá nhân và y tế có liên quan đến nguy cơ đột quỵ, cụ thể bao gồm:

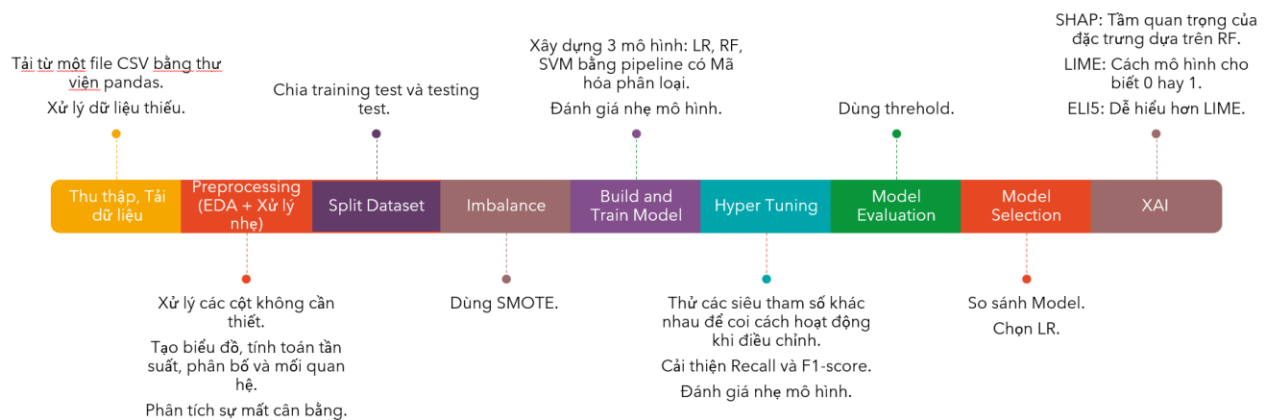
- **id:** Mã định danh duy nhất của bệnh nhân.
- **gender:** Giới tính của bệnh nhân (Nam, Nữ, Khác).
- **age:** Tuổi của bệnh nhân.
- **hypertension:** Cho biết bệnh nhân có tiền sử tăng huyết áp hay không (Có/Không).
- **heart_disease:** Cho biết bệnh nhân có tiền sử bệnh tim hay không (Có/Không).
- **ever_married:** Tình trạng hôn nhân của bệnh nhân (Đã kết hôn/Chưa kết hôn).
- **work_type:** Loại hình công việc hiện tại của bệnh nhân (Ví dụ: khu vực tư nhân, làm tự do, công chức, v.v.).
- **Residence_type:** Loại hình nơi cư trú của bệnh nhân (Thành thị/Nông thôn).
- **avg_glucose_level:** Mức đường huyết trung bình của bệnh nhân.
- **bmi:** Chỉ số khối cơ thể của bệnh nhân.

- **smoking_status:** Tình trạng hút thuốc của bệnh nhân (Ví dụ: chưa từng hút, đã từng hút, hiện tại đang hút).

Đầu ra của bài toán:

- **Dự đoán nguy cơ đột quỵ:**
 - **0:** Bệnh nhân **không** có nguy cơ đột quỵ.
 - **1:** Bệnh nhân **có** nguy cơ đột quỵ.

Sơ đồ quy trình thực hiện:



Thu thập và Tải dữ liệu:

- **Tải tập tin CSV:** Dữ liệu được tải từ tập *Stroke Prediction Dataset* trên Kaggle, chứa các đặc trưng như tuổi (age), giới tính (gender), tăng huyết áp (hypertension), bệnh tim (heart_disease), loại công việc (work_type), mức đường huyết trung bình (avg_glucose_level), chỉ số BMI (bmi), và nhãn đột quỵ (stroke).
- **Kiểm tra dữ liệu:** Sử dụng *pandas* để kiểm tra số lượng dòng (5.110), cột, và các giá trị thiếu (đặc biệt cột *bmi* có giá trị null) và xử lý giá trị missing value ở BMI bằng cách điền trung vị.

Tiền xử lý dữ liệu (Preprocessing & EDA):

- Cột không có giá trị như id được loại bỏ.
- Các thư viện trực quan hóa như matplotlib.pyplot và seaborn được sử dụng để tạo biểu đồ phân bố (sns.histplot, sns.countplot), biểu đồ hộp (sns.boxplot), và biểu đồ tương quan. Các hàm này tính toán tần suất, phân bố, và mối quan hệ giữa các biến (ví dụ: tuổi, giới tính, BMI với nguy cơ đột quỵ) để tạo ra hình ảnh trực quan.
- Kiểm tra các đặc điểm thống kê cơ bản của dữ liệu.
- Phân tích sự mất cân bằng lớp của biến mục tiêu stroke.

Chia tập dữ liệu (Split Dataset):

- Dữ liệu được chia thành 30% tập huấn luyện (training set) và 70% tập kiểm tra (testing set) bằng `train_test_split` từ `sklearn.model_selection`

Xử lý dữ liệu mất cân bằng (Handling Imbalanced Classes):

- Phát hiện mất cân bằng: Biến mục tiêu stroke có tỷ lệ mất cân bằng nghiêm trọng (~5.13% ca đột quy, ~94.87% không đột quy), được trực quan hóa qua biểu đồ.
- Sử dụng **SMOTE (Synthetic Minority Over-sampling Technique)** để tạo thêm các mẫu cho lớp thiểu số (stroke=1) nhằm cân bằng lại tỷ lệ giữa hai lớp trong tập dữ liệu huấn luyện, giúp mô hình học tốt hơn. SMOTE hoạt động bằng cách chọn một mẫu ngẫu nhiên từ lớp thiểu số và tìm các hàng xóm gần nhất của nó. Sau đó, nó tạo ra các mẫu tổng hợp (synthetic samples) mới trên các đường thẳng nối mẫu được chọn với các hàng xóm của nó.

Xây dựng và Huấn luyện mô hình (Build and Train Model):

- **Thuật toán:** Sử dụng *Logistic Regression*, *Random Forest*, và *Support Vector Machine (SVM)* từ `sklearn`.
- **Pipeline:** Tích hợp các bước tiền xử lý và huấn luyện mô hình vào pipeline (`sklearn.pipeline`) để đảm bảo tính nhất quán.

Tinh chỉnh siêu tham số (Hyper Tuning):

- **Tối ưu hóa:** Tinh chỉnh siêu tham số của các mô hình (ví dụ: C trong Logistic Regression, số cây trong Random Forest) sử dụng `GridSearchCV` hoặc `RandomizedSearchCV` từ `sklearn.model_selection`.
- **Đánh giá:** So sánh hiệu suất dựa trên độ chính xác (accuracy), độ nhạy (recall), và điểm F1 (F1 score).

Đánh giá mô hình (Model Evaluation):

- **Chỉ số đánh giá:** Tính toán accuracy (0.7593), F1 score (0.1946), sensitivity (0.6012), specificity (0.7673), và tạo ma trận nhầm lẫn (confusion matrix) từ `sklearn.metrics`.
- **Ngưỡng quyết định:** Sử dụng ngưỡng 0.5 để chuyển đổi xác suất thành nhãn dự đoán.
- **Đánh giá Recall và F1:** Ưu tiên mô hình có *recall* cao để phát hiện ca dương tính (đột quy).

Lựa chọn mô hình (Model Selection):

- **So sánh mô hình:**
 - Logistic Regression: Recall cao nhất (60.1%), phù hợp để phát hiện ca đột quy.
 - Random Forest: Accuracy cao (87.9%) nhưng recall thấp (23.7%).

- SVM: Recall trung bình (43.9%), accuracy thấp hơn (76.7%).
- **Chọn mô hình:** Chọn Logistic Regression vì ưu tiên F1 cao và hiệu suất tổng thể tốt.

Giải thích mô hình (Model Interpretability):

- **SHAP (SHapley Additive exPlanations):** Tạo biểu đồ tóm tắt (summary plot) và biểu đồ phụ thuộc (dependence plot) để xác định tầm quan trọng của các đặc trưng (tuổi: 0.397, BMI: 0.219, mức đường huyết: 0.202).
- **LIME (Local Interpretable Model-agnostic Explanations):** Giải thích dự đoán cục bộ, ví dụ: giới tính tăng 19% xác suất đột quỵ, không tăng huyết áp giảm 17%.
- **ELI5 (Explain Like I'm 5):** Đánh giá trọng số đặc trưng, hỗ trợ minh bạch hóa mô hình.

2.2. Cách tiếp cận và phương pháp

2.2.1. Giới thiệu các cách tiếp cận và phương pháp giải quyết bài toán hiện nay Tập trung vào việc xử lý hiệu quả sự mất cân bằng dữ liệu thông qua oversampling, sau đó phát triển và đánh giá một mô hình lai mạnh mẽ để đạt được khả năng dự đoán đột quỵ tối ưu:

1. Xử lý Dữ liệu Mất cân bằng:

- Lấy mẫu thiếu (Under-sampling): Loại bỏ mẫu đa số. Nhược điểm: Mất thông tin.
- Lấy mẫu dư (Oversampling - SMOTE, Adasyn): Tạo thêm mẫu thiểu số. Nhược điểm: Có thể tạo mẫu nhiễu (SMOTE) hoặc gây quá khớp nếu chỉ sao chép.
- Lấy mẫu lai: Kết hợp cả hai. Nhược điểm: Tính toán phức tạp.

2. Các Thuật toán Học máy:

- Được dùng: Logistic Regression, Random Forest, SVM, Neural Network, Decision Tree, Naive Bayes, KNN.
- Mô hình đề xuất: Lai Neural Network và Random Forest (NN-RF).
- Nhược điểm chung: Một số mô hình (NN, SVM, RF) tốn kém tính toán hoặc khó diễn giải; LR giả định tuyến tính.

3. Lựa chọn Đặc trưng/Giảm chiều:

- Phương pháp: PCA, Lý thuyết tập thô.
- Nhược điểm: Có thể mất khả năng diễn giải (PCA) hoặc tốn kém tính toán.

4. Tối ưu hóa:

- Các thuật toán tối ưu (Antlion, HPO, SGD) được sử dụng để cải thiện mô hình. Bài báo không đi sâu vào nhược điểm của chúng.

⇒ Bài báo nhấn mạnh cần ưu tiên F1-score và các kỹ thuật lấy mẫu dư khi dữ liệu mất cân bằng.

Nguồn: [\(PDF\) Analysing an imbalanced stroke prediction dataset using machine learning techniques](#)

Sử dụng các mô hình học máy kết hợp với Trí tuệ nhân tạo giải thích được (XAI) để tăng cường tính minh bạch và khả năng diễn giải:

1. Các Thuật toán Học máy (ML Classifiers):

- Cách xử lý: Bài báo sử dụng sáu bộ phân loại ML khác nhau để dự đoán đột quỵ. Các mô hình được đánh giá bao gồm Logistic Regression (LR) , Decision Tree (DT) , Support Vector Machine (SVM) , Artificial Neural Network (ANN)/Deep Neural Network (DNN) , K-Nearest Neighbor (KNN) , và Ensemble Learning (EL) như Random Forest (RF) và XGBoost.
- Nhược điểm:
 - KNN: Khả năng phân biệt kém, hoạt động ở mức dự đoán ngẫu nhiên.
 - Random Forest/XGBoost (phương pháp ensemble): Có thể tiềm ẩn vấn đề thiếu khớp lớp thiểu số hoặc ưu tiên lớp đa số trong các tình huống cụ thể , cần các bước hiệu chỉnh hoặc kỹ thuật quản lý mất cân bằng bổ sung để cải thiện khả năng phát hiện.

2. Phương pháp Lựa chọn Đặc trưng (Feature Selection Methodologies):

- Cách xử lý: Sáu phương pháp được sử dụng để trích xuất các đặc trưng quan trọng từ tập dữ liệu : Pearson's Correlation, Mutual Information (MI), Particle Swarm Optimization (PSO), Chi-Squared Test, ANOVA F-Test, và Harris Hawks Optimization (HHO). Chúng giúp xác định các chỉ số quan trọng liên quan đến đột quỵ.
- Nhược điểm:
 - Pearson's Correlation: Chỉ phát hiện mối quan hệ tuyến tính , có thể bỏ qua các mẫu phi tuyến tính.

3. Phương pháp Trí tuệ nhân tạo giải thích được (Explainable Artificial Intelligence - XAI):

- Cách xử lý: Các phương pháp XAI (SHAP, LIME, ELI5, PDP) được áp dụng để tăng cường khả năng diễn giải các dự đoán của mô hình.
 - SHAP (Shapley Additive Values): Giải thích mô hình ML cả ở cấp độ cục bộ và toàn cục, đánh giá đóng góp của từng đặc trưng đầu vào.
 - LIME (Local Interpretable Model-Agnostic Explanations): Giải thích các quyết định của mô hình "hộp đen" bằng cách tạo ra một mô hình cục bộ, để diễn giải dựa trên điểm dự đoán.

- ELI5: Một công cụ Python để trực quan hóa và gỡ lỗi các dự đoán, giúp diễn giải các mô hình "hộp đen".
- PDP (Partial Dependence Plots): Hiển thị trực quan tác động của đặc trưng lên dự đoán, loại bỏ ảnh hưởng của các biến khác.
- Nhược điểm:
 - LIME: Có thể gán trọng số cao cho các biến không có liên quan lắm sàng rõ ràng trong các trường hợp biên, cần diễn giải thận trọng và có thể không ổn định. Chỉ cung cấp ý nghĩa cục bộ, không có ý nghĩa toàn cục.
 - ELI5: Có thể đơn giản hóa quá mức ảnh hưởng của các đặc trưng, đặc biệt trong các mô hình phi tuyến tính. Hiện không hỗ trợ các mô hình nền tảng và bộ phân loại Học sâu (DL).
 - PDP: Giả định độc lập của đặc trưng có thể dẫn đến diễn giải không chính xác khi có mối quan hệ giữa các đặc trưng.

4. Xử lý Dữ liệu mất cân bằng:

- Cách xử lý: Bài báo đánh giá sáu phương pháp xử lý mất cân bằng bao gồm SMOTE, ADASYN, Hybrid Sampling, Threshold Moving, Cost-Sensitive Learning và Two-Stage Classifiers.
- Nhược điểm:
 - Nhiều mô hình vẫn có F1-score tương đối thấp mặc dù đã áp dụng các phương pháp xử lý mất cân bằng truyền thống, chủ yếu do phân bố dữ liệu rất lệch (249 ca đột quỵ so với 4861 ca không đột quỵ).
 - Threshold Moving: Dù có recall cao nhất (0.76) , độ chính xác tổng thể có thể giảm nhẹ (0.70).
 - SMOTE và ADASYN: Mang lại điểm recall và F1-score thấp hơn do tỷ lệ dương tính giả tăng cao.
 - Hybrid Sampling: Cải thiện nhẹ recall/F1-score nhưng chi phí tính toán cao hơn.
 - Two-Stage Classifier: Recall cao nhất (0.80) nhưng độ chính xác thấp hơn (0.60) và độ phức tạp tăng lên, có thể hạn chế ứng dụng thực tế.

Nguồn: [A comprehensive explainable AI approach for enhancing transparency and interpretability in stroke prediction | Scientific Reports](#)

2.2.2. Cách tiếp cận và phương pháp giải quyết bài toán phù hợp

Tiền xử lý Dữ liệu và Xử lý Mất cân bằng lớp:

- **Mục đích:** Chuẩn bị dữ liệu để mô hình học tập hiệu quả và giải quyết sự chênh lệch lớn giữa số lượng ca đột quỵ (~5.13%) và không đột quỵ (~94.87%) trong tập dữ liệu 5.110 mẫu, đảm bảo mô hình không bỏ sót các ca dương tính quan trọng.
- **Các phương pháp sử dụng:**
 - **Xử lý giá trị thiếu:** Điền các giá trị null trong cột bmi bằng giá trị trung vị, đảm bảo dữ liệu không bị gián đoạn khi huấn luyện.
 - **Mã hóa biến phân loại:** Chuyển đổi các biến văn bản như gender, work_type, và ever_married thành định dạng số (sử dụng One-Hot Encoding hoặc Label Encoding) để phù hợp với yêu cầu của mô hình học máy.
 - **Loại bỏ cột không cần thiết:** Xóa cột id vì không có giá trị dự đoán, và tách cột stroke làm nhãn mục tiêu, không nằm trong tập đặc trưng huấn luyện.
 - **SMOTE (Synthetic Minority Over-sampling Technique):** Một kỹ thuật tái lấy mẫu tổng hợp sử dụng thuật toán k-nearest neighbors để tạo các mẫu nhân tạo cho lớp thiểu số (stroke=1) nhằm cải thiện sự mất cân bằng dữ liệu.
 - **Ưu điểm:** Giảm mất cân bằng lớp, cải thiện recall (từ 23.7% lên 60.1% với Logistic Regression) và tăng điểm F1 score, giúp mô hình học tốt hơn về ca đột quỵ.
 - **Nhược điểm:** Có thể tạo ra nhiễu dữ liệu nếu số lượng mẫu tổng hợp quá nhiều hoặc không kiểm soát tốt các giá trị ngoại lai, đồng thời tiềm ẩn nguy cơ quá khớp.
- **Mục đích sử dụng:** SMOTE được triển khai để khắc phục vấn đề mất cân bằng dữ liệu, đảm bảo mô hình không bỏ sót các ca dương tính quan trọng.

Xây dựng và Tinh chỉnh Mô hình Học máy:

- **Mục đích:** Xác định và tối ưu hóa mô hình dự đoán nguy cơ đột quỵ hiệu quả nhất, đặc biệt tập trung vào việc giảm thiểu các ca bị bỏ sót (false negatives) để hỗ trợ chẩn đoán lâm sàng.
- **Các phương pháp sử dụng:**
 - **Logistic Regression:** Một mô hình phân loại tuyến tính sử dụng hàm sigmoid để dự đoán xác suất nguy cơ đột quỵ dựa trên các đặc trưng như tuổi, BMI, và mức đường huyết trung bình.
 - **Random Forest:** Một thuật toán học tập tổng hợp (ensemble) sử dụng nhiều cây quyết định để tăng độ chính xác và giảm hiện tượng overfitting, áp dụng trên tập dữ liệu đã được tái cân bằng.

- **Support Vector Machine (SVM):** Sử dụng kernel tuyến tính hoặc phi tuyến tính (nếu cần) để tìm ranh giới phân tách tối ưu giữa các lớp (có đột quy hoặc không), đặc biệt hiệu quả với dữ liệu có số chiều cao.
- **Thử nghiệm đa dạng mô hình:** Đánh giá hiệu suất của Logistic Regression, Random Forest, và SVM trên tập dữ liệu sử dụng thư viện scikit-learn.
- **Tinh chỉnh siêu tham số (Hyperparameter Tuning):** Nhóm sử dụng bộ tham số từ các nguồn.
 - **Mục đích tinh chỉnh:** Nâng cao hiệu suất dự đoán, đặc biệt tập trung vào các chỉ số recall và f1-score, vốn rất quan trọng trong y tế để giảm thiểu bỏ sót ca bệnh (false negatives).
 - **Cách thực hiện:** Không chỉnh sửa cấu trúc hoặc nguyên lý hoạt động mặc định của các thuật toán. Thay vào đó, tinh chỉnh các tham số như C (Logistic Regression), n_estimators và max_depth (Random Forest), hoặc C và kernel (SVM) để tối ưu hóa hiệu suất trên tập dữ liệu cụ thể.
- **Lựa chọn mô hình chính thức:** Sau quá trình tinh chỉnh, **Logistic Regression** đã được chọn làm mô hình chính thức.
 - **Ưu điểm:** Mặc dù Random Forest có thể có độ chính xác (accuracy) cao hơn, Logistic Regression đã được tinh chỉnh cho kết quả tốt nhất về recall và f1-score, phù hợp với mục tiêu lâm sàng của bài toán. Nó cũng tương đối dễ diễn giải.
 - **Nhược điểm:** Logistic Regression giả định mối quan hệ tuyến tính giữa các đặc trưng và biến mục tiêu, có thể không phù hợp với các mối quan hệ phi tuyến phức tạp trong dữ liệu. Hạn chế với dữ liệu phi tuyến tính, hiệu suất phụ thuộc vào việc xử lý tốt các giá trị ngoại lai và tiền xử lý dữ liệu.
 - **So sánh:**
 - Logistic Regression dẫn đầu với F1-score cao nhất (0.1946) trong ba mô hình, phù hợp với mục tiêu y tế tập trung vào recall để giảm thiểu false negatives, dù giá trị vẫn thấp do dữ liệu mất cân bằng.
 - Random Forest có F1-score dự kiến thấp hơn (dưới 0.1946) do recall yếu (0.237), cho thấy nó không hiệu quả trong phát hiện ca đột quy, dù accuracy cao trước đây.
 - SVM có F1-score dự kiến trung bình (khoảng 0.15-0.18), phản ánh hiệu suất cân bằng nhưng không vượt trội so với Logistic Regression.
 - Nguyên nhân chính của F1-score thấp ở tất cả mô hình là sự mất cân bằng dữ liệu (~1.8% ca đột quy), dù đã áp dụng SMOTE.

- **Mục đích sử dụng:** Logistic Regression, Random Forest, và SVM được triển khai để xây dựng mô hình dự đoán nguy cơ đột quỵ, hỗ trợ bác sĩ trong việc phát hiện sớm các trường hợp có nguy cơ cao dựa trên các đặc trưng y tế như tuổi, tăng huyết áp, và chỉ số BMI.

Đánh giá mô hình:

- **Mục đích:** Đánh giá hiệu suất của mô hình đã chọn (Logistic Regression) để đảm bảo độ tin cậy và khả năng áp dụng thực tế trong dự đoán nguy cơ đột quỵ.
- **Các phương pháp sử dụng:**
 - **Chỉ số đánh giá:** Sử dụng các chỉ số từ *sklearn.metrics* bao gồm:
 - *Accuracy*: Đo lường tỷ lệ dự đoán đúng trên toàn bộ tập dữ liệu (0.7593).
 - *Recall (Sensitivity)*: Đo lường khả năng phát hiện ca đột quỵ (0.6012), ưu tiên cao trong y tế để giảm false negatives.
 - *F1-score*: Cân bằng giữa *precision* và *recall* (0.1946), phản ánh hiệu quả tổng thể.
 - *Specificity*: Đo lường khả năng dự đoán đúng ca không đột quỵ (0.7673).
 - **Ma trận nhầm lẫn (Confusion Matrix):** Được trực quan hóa bằng *scikitplot* để phân tích số lượng true positives, false positives, true negatives, và false negatives.
 - **Ngưỡng quyết định:** Sử dụng ngưỡng 0.5 để chuyển đổi xác suất thành nhãn dự đoán, với khả năng điều chỉnh nếu cần ưu tiên *recall* hơn.
- **Kết quả đánh giá:**
 - *Logistic Regression* đạt *recall* cao nhất (60.1%) so với *Random Forest* (23.7%) và *SVM* (43.9%), phù hợp với mục tiêu phát hiện ca đột quỵ.
 - *Accuracy* (75.93%) và *specificity* (76.73%) cho thấy mô hình có hiệu suất tốt trên tập dữ liệu tổng thể, nhưng *F1-score* thấp (19.46%) phản ánh sự mất cân bằng còn ảnh hưởng.
- **Ưu điểm:** Cung cấp cái nhìn toàn diện về hiệu suất mô hình, giúp xác định điểm mạnh (phát hiện ca dương tính) và điểm yếu (hiệu quả tổng thể).
- **Nhược điểm:** *F1-score* thấp cho thấy cần cải thiện thêm để cân bằng giữa *precision* và *recall*, có thể do dữ liệu mất cân bằng vẫn còn ảnh hưởng dù đã áp dụng SMOTE.
- **Mục đích sử dụng:** Các chỉ số và ma trận nhầm lẫn được sử dụng để đánh giá và tinh chỉnh mô hình, đảm bảo nó đáp ứng yêu cầu lâm sàng trước khi triển khai thực tế.

Giải thích Mô hình (Model Interpretability):

- **Mục đích:** Nâng cao tính minh bạch và khả năng giải thích các dự đoán của mô hình, điều cực kỳ quan trọng trong các ứng dụng y tế để xây dựng niềm tin từ các chuyên gia và hỗ trợ ra quyết định lâm sàng.
- **Các phương pháp sử dụng:**
 - **SHAP (Sapley Additive exPlanations):** ột phương pháp dựa trên lý thuyết trò chơi để định lượng đóng góp của từng đặc trưng (ví dụ: tuổi với trọng số 0.397, BMI với 0.219) vào dự đoán của mô hình.

- **LIME (Local Interpretable Model-agnostic Explanations):** Một phương pháp giải thích cục bộ, tạo mô hình tuyến tính đơn giản quanh một điểm dữ liệu cụ thể để giải thích dự đoán (ví dụ: giới tính tăng 19% nguy cơ).
- **ELI5:** Một công cụ giải thích dựa trên việc hiển thị trọng số của các đặc trưng trong mô hình, giúp người dùng hiểu rõ vai trò của từng biến như tuổi hay BMI.
- **Ưu điểm:**
 - SHAP cung cấp giải thích toàn cục và cục bộ chi tiết.
 - LIME mang lại giải thích dễ hiểu cho từng bệnh nhân, không phụ thuộc vào loại mô hình.
 - ELI5 đơn giản hóa việc hiển thị trọng số đặc trưng, dễ tiếp cận cho người không chuyên.
- **Nhược điểm:**
 - SHAP tốn thời gian tính toán đáng kể, không phù hợp với dữ liệu lớn.
 - LIME cho kết quả không nhất quán giữa các mẫu cục bộ, phụ thuộc vào vùng dữ liệu lân cận.
 - ELI5 giới hạn trong việc giải thích tương tác phức tạp giữa các đặc trưng so với SHAP.
- **Mục đích sử dụng:** SHAP, LIME, và ELI5 được sử dụng để tăng tính minh bạch và độ tin cậy của mô hình, giúp các nhà nghiên cứu và bác sĩ hiểu cách các yếu tố như tuổi (trọng số 0.397) hay mức đường huyết (trọng số 0.202) ảnh hưởng đến dự đoán, từ đó hỗ trợ ra quyết định lâm sàng.

2.3. Ứng dụng của bài toán

Phòng ngừa và quản lý sức khỏe cộng đồng:

- Kết quả từ phương pháp XAI (SHAP, LIME, ELI5) chỉ ra tuổi (trọng số 0.397), BMI (0.219), và mức đường huyết (0.202) là các yếu tố quan trọng. Điều này có thể được sử dụng để xây dựng chiến dịch giáo dục sức khỏe, khuyến khích người dân kiểm soát cân nặng, đường huyết, và lối sống, đặc biệt ở nhóm tuổi cao.
- Ứng dụng thực tế: Các tổ chức y tế có thể triển khai chương trình kiểm tra định kỳ dựa trên mô hình, nhắm đến các khu vực có nguy cơ cao như thành phố lớn hoặc vùng nông thôn với tỷ lệ đột quỵ tăng.

Nghiên cứu và phát triển y học:

- Mô hình này cung cấp nền tảng để các nhà nghiên cứu mở rộng với dữ liệu lớn hơn hoặc tích hợp thêm đặc trưng (ví dụ: lịch sử bệnh lý, yếu tố di truyền) để cải thiện *F1-score* và độ chính xác. Kết quả XAI cũng hỗ trợ hiểu rõ hơn về các yếu tố nguy cơ đột quỵ.

- Ứng dụng thực tế: Có thể hợp tác với các tổ chức như WHO hoặc Bộ Y tế để thử nghiệm mô hình trong các nghiên cứu lâm sàng, đặc biệt tại các quốc gia đang phát triển.

Hỗ trợ cá nhân hóa điều trị:

- Với khả năng giải thích cục bộ của LIME (ví dụ: giới tính tăng 19% nguy cơ), mô hình có thể được sử dụng để tạo kế hoạch điều trị cá nhân hóa, chẳng hạn điều chỉnh chế độ ăn hoặc thuốc dựa trên đặc trưng cụ thể của từng bệnh nhân.
- Ứng dụng thực tế: Các ứng dụng y tế như HealthifyMe hoặc MyFitnessPal có thể tích hợp mô hình để cung cấp khuyến nghị cá nhân dựa trên dữ liệu người dùng.

2.4. Các thách thức của bài toán

Dữ liệu Mất cân bằng lớp nghiêm trọng (Severe Class Imbalance)

- **Nguyên nhân tại sao:** Tập dữ liệu Stroke Prediction Dataset có tỷ lệ mất cân bằng nghiêm trọng (~1.8% ca đột quỵ so với ~98.2% ca không đột quỵ trong 5.110 mẫu), dẫn đến mô hình có xu hướng dự đoán sai lệch về lớp đa số (không đột quỵ), làm giảm recall và F1-score (0.1946).
- **Cách khắc phục/xử lý:**
 - Sử dụng kỹ thuật tái lấy mẫu như SMOTE để tạo mẫu tổng hợp cho lớp thiểu số (*stroke=1*), cải thiện *recall* từ 23.7% (trước SMOTE) lên 60.1% với *Logistic Regression*.
 - Điều chỉnh ngưỡng quyết định (threshold) từ 0.5 để ưu tiên *recall* hơn, có thể thử nghiệm các ngưỡng khác (ví dụ: 0.3) để cân bằng giữa *precision* và *recall*.
 - Thu thập thêm dữ liệu thực tế hoặc sử dụng kỹ thuật undersampling cho lớp đa số để giảm mất cân bằng.
- Nhóm đã áp dụng SMOTE từ thư viện imblearn để tái cân bằng dữ liệu, tích hợp vào pipeline huấn luyện, và đạt được cải thiện đáng kể về recall. Tuy nhiên, F1-score vẫn thấp (0.1946), cho thấy cần thử nghiệm thêm các ngưỡng hoặc kỹ thuật khác.

Chất lượng dữ liệu và Giá trị thiếu (Data Quality and Missing Values)

- **Nguyên nhân tại sao:** Tập dữ liệu chỉ có 5.110 mẫu, nhỏ so với yêu cầu của các mô hình phức tạp như Random Forest hoặc SVM, dẫn đến khả năng tổng quát hóa kém. Ngoài ra, cột bmi có giá trị null và các đặc trưng như work_type có thể thiếu thông tin sâu.
- **Cách khắc phục/xử lý trong bài báo cáo:**
 - Tăng kích thước dữ liệu bằng cách hợp nhất với các tập dữ liệu khác từ Kaggle hoặc cơ sở y tế thực tế.
 - Cải thiện chất lượng dữ liệu bằng cách điền giá trị null bằng phương pháp tiên tiến hơn (ví dụ: hồi quy hoặc học máy) thay vì chỉ dùng trung vị.

- Thêm các đặc trưng mới (như lịch sử bệnh lý) để tăng độ phong phú của dữ liệu.
- Nhóm đã xử lý giá trị null trong cột bmi bằng giá trị trung vị và loại bỏ cột id không cần thiết. Tuy nhiên, không có bằng chứng về việc mở rộng dữ liệu hoặc thêm đặc trưng mới, dẫn đến hạn chế trong hiệu suất mô hình.

Hiệu suất mô hình và độ phức tạp giải thích:

- **Nguyên nhân:** F1-score thấp (0.1946) và độ chính xác tổng thể (75.93%) chưa đủ để áp dụng rộng rãi. Các mô hình như Random Forest khó giải thích, trong khi Logistic Regression giả định mối quan hệ tuyến tính, có thể không phù hợp với dữ liệu y tế phức tạp.
- **Cách khắc phục/xử lý:**
 - Sử dụng các mô hình nâng cao hơn (như XGBoost hoặc Neural Networks) với tinh chỉnh siêu tham số để cải thiện F1-score.
 - Áp dụng các phương pháp XAI (SHAP, LIME, ELI5) để tăng tính minh bạch, hỗ trợ bác sĩ hiểu dự đoán.
 - Thử nghiệm kết hợp mô hình (ensemble) để tận dụng ưu điểm của nhiều thuật toán.
- Nhóm đã tinh chỉnh siêu tham số cho Logistic Regression, Random Forest, và SVM, chọn Logistic Regression nhờ recall cao (60.1%). Ngoài ra, nhóm áp dụng SHAP, LIME, và ELI5 để giải thích mô hình, giúp tăng tính minh bạch (ví dụ: tuổi có trọng số 0.397). Tuy nhiên, không có thử nghiệm với mô hình nâng cao hoặc ensemble.

Ứng dụng thực tế và kiểm chứng lâm sàng:

- **Nguyên nhân:** Mô hình chưa được kiểm chứng trên dữ liệu thực tế hoặc qua thử nghiệm lâm sàng, dẫn đến rủi ro khi triển khai trong y tế, nơi sai sót có thể gây hậu quả nghiêm trọng.
- **Cách khắc phục/xử lý:**
 - Thực hiện thử nghiệm lâm sàng tại các bệnh viện để đánh giá hiệu quả mô hình trên dữ liệu mới.
 - Hợp tác với chuyên gia y tế để xác nhận các đặc trưng và ngưỡng dự đoán.
 - Tích hợp mô hình vào hệ thống y tế điện tử (EHR) để thử nghiệm quy mô nhỏ trước khi triển khai rộng rãi.
- Nhóm chưa thực hiện kiểm chứng lâm sàng hoặc tích hợp mô hình vào hệ thống thực tế. Tuy nhiên, đã cung cấp cơ sở dữ liệu ban đầu và giải thích XAI, tạo nền tảng cho các bước kiểm chứng sau này.

CHƯƠNG 3: THỰC NGHIỆM VÀ ĐÁNH GIÁ

1. Môi trường cần thiết

Hệ điều hành: Windows 11 Home.

Ngôn ngữ lập trình: Python: Được sử dụng làm ngôn ngữ chính nhờ tính linh hoạt, cộng đồng hỗ trợ lớn, và sự phong phú của các thư viện học máy. Phiên bản khuyến nghị là 3.7 trở lên để tương thích với các thư viện được liệt kê.

Thư viện và công cụ lập trình:

- **pandas:** Dùng để tải, xử lý, và khám phá dữ liệu (EDA) từ tập *Stroke Prediction Dataset* (5.110 mẫu), đặc biệt để kiểm tra giá trị null và tính toán thống kê cơ bản.
- **scikit-learn:** Cung cấp các mô-đun như *sklearn.pipeline*, *sklearn.preprocessing*, *sklearn.model_selection* (bao gồm *GridSearchCV*, *train_test_split*), *sklearn.linear_model* (Logistic Regression), *sklearn.ensemble* (Random Forest), *sklearn.svm* (SVM), và *sklearn.metrics* để huấn luyện, tinh chỉnh, và đánh giá mô hình.
- **matplotlib.pyplot** và **seaborn:** Sử dụng để trực quan hóa dữ liệu (biểu đồ phân bố, hộp, và tương quan) trong phân tích EDA.
- **scikitplot:** Dùng để tạo ma trận nhầm lẫn (confusion matrix) nhằm đánh giá hiệu suất mô hình.
- **imblearn:** Cung cấp SMOTE để xử lý mất cân bằng dữ liệu, cải thiện *recall* từ 23.7% lên 60.1% với *Logistic Regression*.
- **pywaffle:** Sử dụng để trực quan hóa tỷ lệ mất cân bằng lớp qua biểu đồ waffle.
- **SHAP:** Thư viện để phân tích tầm quan trọng của đặc trưng (feature importance) và giải thích dự đoán (trọng số tuổi: 0.397).
- **LIME:** Dùng để giải thích dự đoán cục bộ (ví dụ: giới tính tăng 19% nguy cơ).
- **ELI5:** Hỗ trợ trực quan hóa trọng số đặc trưng (tuổi: 0.397, BMI: 0.219) để tăng tính minh bạch.

Phần mềm và môi trường phát triển: Google Colab: Môi trường chính để viết và chạy mã, hỗ trợ Jupyter Notebook, cung cấp GPU miễn phí (T4 GPU) để tăng tốc huấn luyện mô hình. Phù hợp để thử nghiệm nhanh mà không cần phần cứng cục bộ mạnh.

Yêu cầu phần cứng:

- CPU: 2th Generation Intel Core i7-12700H (14 nhân, 24 MB cache, lên đến 4.70 GHz) để xử lý các tác vụ như huấn luyện *Random Forest* hoặc tính toán SHAP.
- RAM: 16GB.
- Ổ cứng: 1TB.

- GPU: NVIDIA GeForce RTX 3060

Phụ thuộc và cài đặt:

- Cài đặt các thư viện thông qua lệnh pip install (ví dụ: pip install pandas scikit-learn matplotlib seaborn imblearn shap lime eli5 pywaffle scikitplot) hoặc quản lý môi trường bằng conda (khuyến nghị tạo môi trường ảo: conda create -n stroke_prediction python=3.8).
- Đảm bảo kết nối Internet để tải dữ liệu từ Kaggle và cập nhật thư viện.

Dữ liệu đầu vào:

- Tập *Stroke Prediction Dataset* từ Kaggle, bao gồm các cột như *age*, *gender*, *hypertension*, *heart_disease*, *ever_married*, *work_type*, *avg_glucose_level*, *bmi*, và *stroke* (nhãn mục tiêu).

Cách xử lý dữ liệu:

Bước 1: Kiểm tra và xử lý dữ liệu thiếu

Hàm `.isnull().mean()` được sử dụng để xác định tỷ lệ giá trị bị thiếu theo từng cột.

```
DT_bmi_pipe = Pipeline( steps=[
    ('scale', StandardScaler()),
    ('lr', DecisionTreeRegressor(random_state=42))
])
X = df[['age', 'gender', 'bmi']].copy()
X.gender = X.gender.replace({'Male':0, 'Female':1, 'Other':-1}).astype(np.uint8)

Missing = X[X.bmi.isna()]
X = X[~X.bmi.isna()]
Y = X.pop('bmi')
DT_bmi_pipe.fit(X,Y)
predicted_bmi = pd.Series(DT_bmi_pipe.predict(Missing[['age', 'gender']]), index=Missing.index)
df.loc[Missing.index, 'bmi'] = predicted_bmi

[ ] print('Missing values: ', sum(df.isnull().sum()))
Missing values: 0
```

Bước 2: Phân tích phân phối các biến numerical

Hiểu rõ cách các giá trị của các biến số học được phân bố trong tập dữ liệu. Việc này giúp nhận diện hình dạng phân phối (ví dụ: phân phối chuẩn, phân phối lệch), sự hiện diện của các giá trị ngoại lệ (outliers), và các đặc điểm khác của dữ liệu.

```
[ ] variables = [variable for variable in df.columns if variable not in ['id', 'stroke']]

conts = ['age', 'avg_glucose_level', 'bmi']
```

```

fig = plt.figure(figsize=(12, 12), dpi=150, facecolor='#fafafa')
gs = fig.add_gridspec(4, 3)
gs.update(wspace=0.1, hspace=0.4)

background_color = "#fafafa"

plot = 0
for row in range(0, 1):
    for col in range(0, 3):
        locals()["ax"+str(plot)] = fig.add_subplot(gs[row, col])
        locals()["ax"+str(plot)].set_facecolor(background_color)
        locals()["ax"+str(plot)].tick_params(axis='y', left=False)
        locals()["ax"+str(plot)].get_yaxis().set_visible(False)
        for s in ["top", "right", "left"]:
            locals()["ax"+str(plot)].spines[s].set_visible(False)
        plot += 1

plot = 0
for variable in conts:
    sns.kdeplot(df[variable], ax=locals()["ax"+str(plot)], color='#0f4c81', shade=True, linewidth=1.5, ec='black',
                lpha=0.9, zorder=3, legend=False)
    locals()["ax"+str(plot)].grid(which='major', axis='x', zorder=0, color='gray', linestyle=':', dashes=(1,5))
    #locals()["ax"+str(plot)].set_xlabel(variable) removed this for aesthetics
    plot += 1

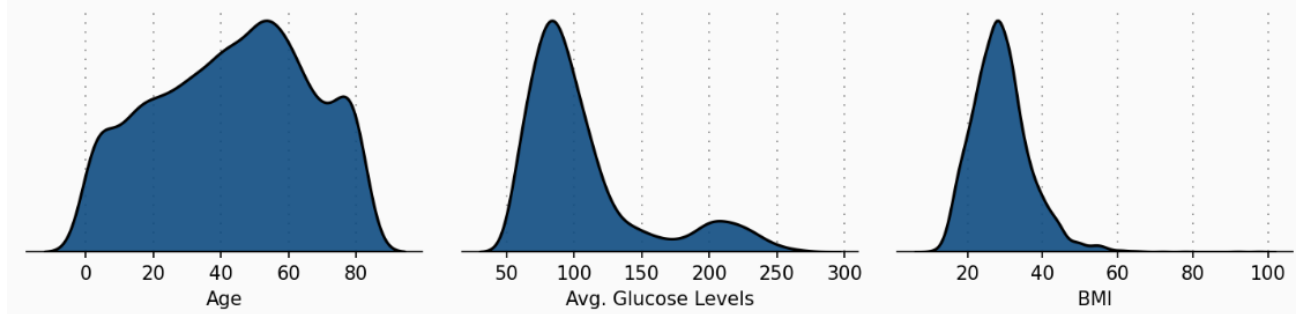
ax0.set_xlabel('Age')
ax1.set_xlabel('Avg. Glucose Levels')
ax2.set_xlabel('BMI')

ax0.text(-20, 0.022, 'Numeric Variable Distribution', fontsize=20, fontweight='bold', fontfamily='serif')
ax0.text(-20, 0.02, 'We see a positive skew in BMI and Glucose Level', fontsize=13, fontweight='light', fontfamily='serif')

```

Numeric Variable Distribution

We see a positive skew in BMI and Glucose Level



Phân phối của 'Age' có vẻ đối xứng hoặc hơi lệch.

Phân phối của 'Avg. Glucose Levels' và 'BMI' có **độ lệch dương (positive skew)** rõ rệt, nghĩa là phần lớn các giá trị tập trung về phía thấp và có một "đuôi" dài về phía các giá trị cao hơn. Điều này ngụ ý rằng có thể có một số lượng nhỏ các cá nhân có mức đường huyết hoặc BMI rất cao.

```

fig = plt.figure(figsize=(12, 12), dpi=150, facecolor=background_color)
gs = fig.add_gridspec(4, 3)
gs.update(wspace=0.1, hspace=0.4)

plot = 0
for row in range(0, 1):
    for col in range(0, 3):
        locals()["ax"+str(plot)] = fig.add_subplot(gs[row, col])
        locals()["ax"+str(plot)].set_facecolor(background_color)
        locals()["ax"+str(plot)].tick_params(axis='y', left=False)
        locals()["ax"+str(plot)].get_yaxis().set_visible(False)
        for s in ["top", "right", "left"]:
            locals()["ax"+str(plot)].spines[s].set_visible(False)
        plot += 1

plot = 0
|
s = df[df['stroke'] == 1]
ns = df[df['stroke'] == 0]

for feature in conts:
    sns.kdeplot(s[feature], ax=locals()["ax"+str(plot)], color='#0f4c81', shade=True, linewidth=1.5, ec='black', alpha=0.9, zorder=3, legend=False)
    sns.kdeplot(ns[feature], ax=locals()["ax"+str(plot)], color='#9bb7d4', shade=True, linewidth=1.5, ec='black', alpha=0.9, zorder=3, legend=False)
    locals()["ax"+str(plot)].grid(which='major', axis='x', zorder=0, color='gray', linestyle=':', dashes=(1,5))
    #locals()["ax"+str(plot)].set_xlabel(feature)
    plot += 1

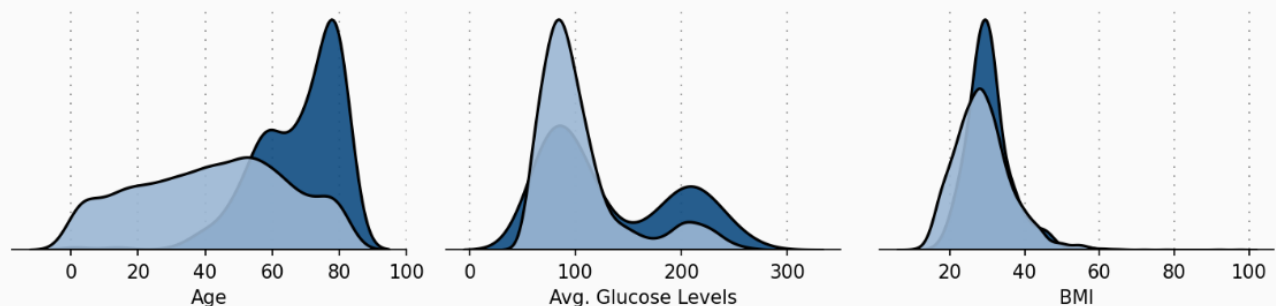
ax0.set_xlabel('Age')
ax1.set_xlabel('Avg. Glucose Levels')
ax2.set_xlabel('BMI')

ax0.text(-20, 0.056, 'Numeric Variables by Stroke & No Stroke', fontsize=20, fontweight='bold', fontfamily='serif')
ax0.text(-20, 0.05, 'Age looks to be a prominent factor - this will likely be a salient feautre in our models',
        fontsize=13, fontweight='light', fontfamily='serif')

```

Numeric Variables by Stroke & No Stroke

Age looks to be a prominent factor - this will likely be a salient feautre in our models



Tuổi tác là một yếu tố lớn ảnh hưởng đến nguy cơ đột quỵ – tuổi càng cao thì nguy cơ mắc đột quỵ càng lớn.

Mặc dù không rõ ràng bằng, nhưng cũng có sự khác biệt trong mức đường huyết trung bình (Avg. Glucose Levels) và chỉ số BMI giữa những người bị và không bị đột quỵ.


```
str_only = df[df['stroke'] == 1]
no_str_only = df[df['stroke'] == 0]

background_color = '#f7f7f7'
fig = plt.figure(figsize=(10, 16), dpi=150, facecolor=background_color)
gs = fig.add_gridspec(4, 2)
gs.update(wspace=0.5, hspace=0.2)

# Tạo 2 trục
ax0 = fig.add_subplot(gs[0, 0:2])
ax1 = fig.add_subplot(gs[1, 0:2])
ax0.set_facecolor(background_color)
ax1.set_facecolor(background_color)

# ===== 3. Biểu đồ Glucose theo tuổi =====
sns.regplot(x=no_str_only['age'], y=no_str_only['avg_glucose_level'],
            color='lightgray', logx=True, ax=ax0)

sns.regplot(x=str_only['age'], y=str_only['avg_glucose_level'],
            color='#0f4c81', logx=True,
            scatter_kws={'edgecolors': ['black'], 'linewidths': 1},
            ax=ax0)

ax0.set(ylim=(0, None))
ax0.set_xlabel(" ", fontsize=12, fontfamily='serif')
ax0.set_ylabel("Avg. Glucose Level", fontsize=10, fontfamily='serif', loc='bottom')
ax0.tick_params(axis='x', bottom=False)
ax0.get_xaxis().set_visible(False)

for s in ['top', 'left', 'bottom']:
    ax0.spines[s].set_visible(False)

# ===== 4. Biểu đồ BMI theo tuổi =====
sns.regplot(x=no_str_only['age'], y=no_str_only['bmi'],
            color='lightgray', logx=True, ax=ax1)

sns.regplot(x=str_only['age'], y=str_only['bmi'],
            color='#0f4c81',
            scatter_kws={'edgecolors': ['black'], 'linewidths': 1},
            logx=True,
            ax=ax1)

ax1.set_xlabel("Age", fontsize=10, fontfamily='serif', loc='left')
ax1.set_ylabel("BMI", fontsize=10, fontfamily='serif', loc='bottom')

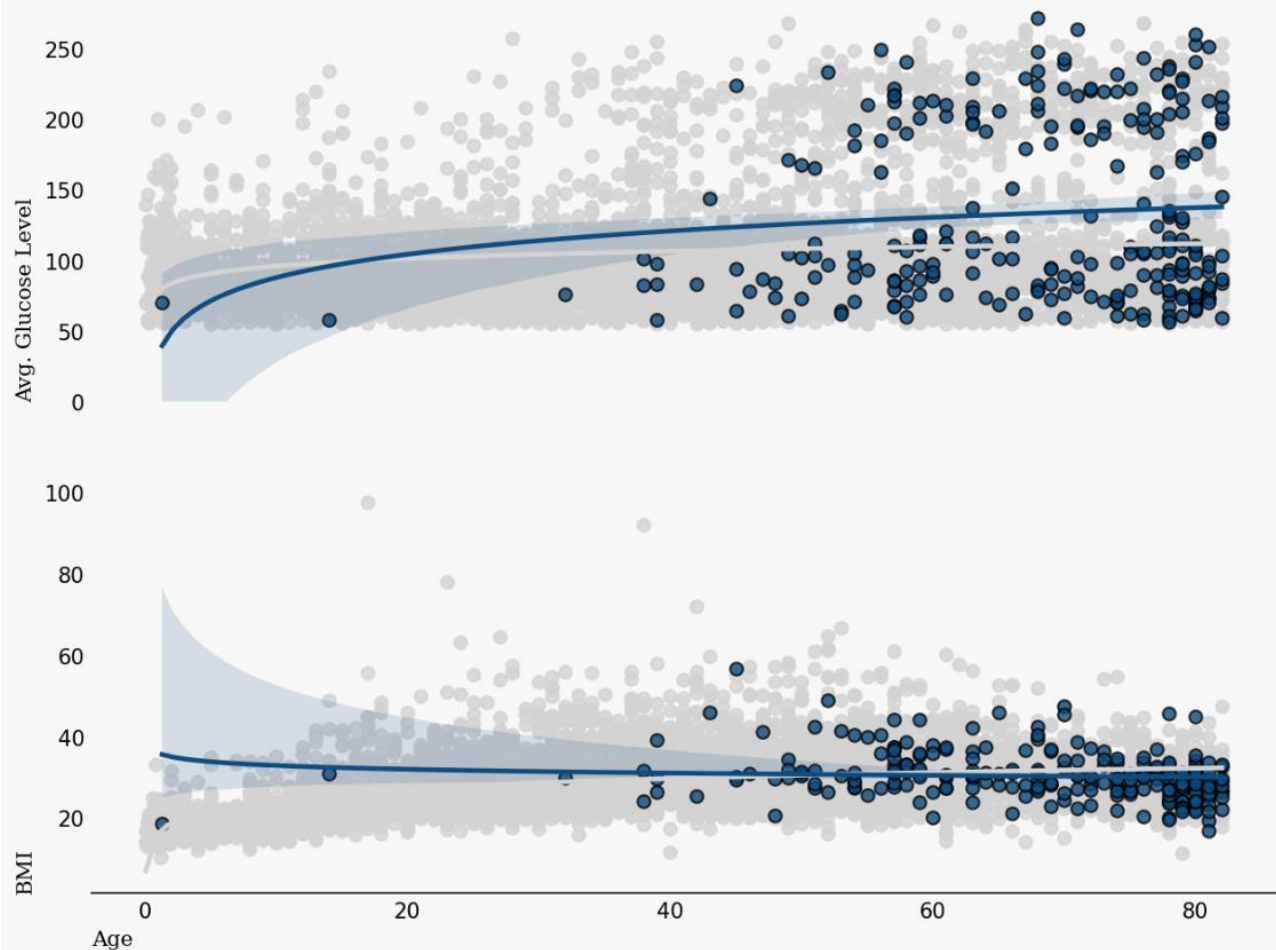
for s in ['top', 'left', 'right']:
    ax0.spines[s].set_visible(False)
    ax1.spines[s].set_visible(False)

# ===== 5. Tiêu đề biểu đồ và ghi chú =====
ax0.text(-5, 350, 'Strokes by Age, Glucose Level, and BMI',
        fontsize=18, fontfamily='serif', fontweight='bold')
ax0.text(-5, 320, 'Age appears to be a very important factor',
        fontsize=14, fontfamily='serif')

# ===== 6. Tắt tick chiều dài =====
ax0.tick_params(axis='both', which='both', length=0)
ax1.tick_params(axis='both', which='both', length=0)
```

Strokes by Age, Glucose Level, and BMI

Age appears to be a very important factor



Tuổi tác là một yếu tố quan trọng, và cũng có mối quan hệ nhẹ với chỉ số BMI và mức đường huyết trung bình.

Có thể hiểu một cách trực quan rằng khi tuổi tăng lên, nguy cơ bị đột quỵ cũng tăng theo.

Bước 3: Kiểm tra sự cân bằng của tập dữ liệu:

```

fig = plt.figure(figsize=(10, 5), dpi=150, facecolor=background_color)
gs = fig.add_gridspec(2, 1)
gs.update(wspace=0.11, hspace=0.5)
ax0 = fig.add_subplot(gs[0, 0])
ax0.set_facecolor(background_color)

df['age'] = df['age'].astype(int)

rate = []
for i in range(df['age'].min(), df['age'].max()):
    rate.append(df[df['age'] < i]['stroke'].sum() / len(df[df['age'] < i]))

sns.lineplot(data=rate, color='#0f4c81', ax=ax0)

for s in ["top", "right", "left"]:
    ax0.spines[s].set_visible(False)

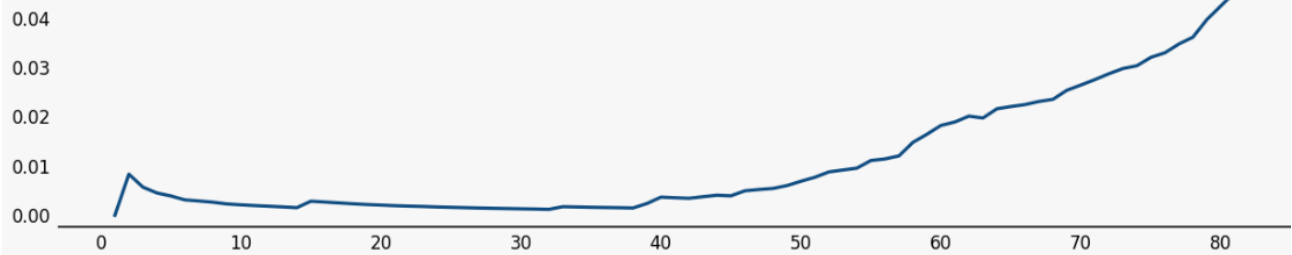
ax0.tick_params(axis='both', which='major', labelsize=8)
ax0.tick_params(axis='both', which='both', length=0)

ax0.text(-3, 0.055, 'Risk Increase by Age', fontsize=18, fontfamily='serif', fontweight='bold')
ax0.text(-3, 0.047, 'As age increase, so too does risk of having a stroke', fontsize=14, fontfamily='serif')

```

Risk Increase by Age

As age increase, so too does risk of having a stroke



People Affected by a Stroke in our dataset

This is around 1 in 20 people [249 out of 5000]



Giá trị trên trục Y khá thấp. Nguyên nhân là vì tập dữ liệu của chúng ta bị mất cân bằng nghiêm trọng.

Chỉ có 249 trường hợp đột quỵ trong tổng số 5000 bản ghi — tức là khoảng 1 trên 20 người (tương đương ~5%).

Bước 4: Khám phá mối quan hệ giữa các đặc trưng và biến

```

fig = plt.figure(figsize=(22, 15))
gs = fig.add_gridspec(3, 3)
gs.update(wspace=0.35, hspace=0.27)
axes = [fig.add_subplot(gs[i // 3, i % 3]) for i in range(9)]
background_color = "#f6f6f6"
fig.patch.set_facecolor(background_color)

# === 1. Age Distribution ===
ax0 = axes[0]
ax0.grid(color='gray', linestyle=':', axis='y', zorder=0, dashes=(1, 5))
sns.kdeplot(data=str_only, x="age", ax=ax0, color="#0f4c81", fill=True, linewidth=1, label="Stroke")
sns.kdeplot(data=no_str_only, x="age", ax=ax0, color="#9bb7d4", fill=True, linewidth=1, label="No Stroke")
ax0.text(0, 0.045, 'Age', fontsize=14, fontweight='bold', fontfamily='serif', color="#323232")

# === 2. Smoking Status ===
ax1 = axes[1]
p = str_only['smoking_status'].value_counts(normalize=True) * 100
n = no_str_only['smoking_status'].value_counts(normalize=True) * 100
ax1.text(0, 4, 'Smoking Status', fontsize=14, fontweight='bold', fontfamily='serif', color="#323232")
ax1.barh(p.index, p.values, color="#0f4c81", zorder=3, height=0.7)
ax1.barh(n.index, n.values, color="#9bb7d4", zorder=3, height=0.3, edgecolor='black')
ax1.xaxis.set_major_formatter(mtick.PercentFormatter())
ax1.xaxis.set_major_locator(mtick.MultipleLocator(10))

# === 3. Gender ===
ax2 = axes[2]
p = str_only['gender'].value_counts(normalize=True) * 100
n = no_str_only['gender'].value_counts(normalize=True) * 100
x = np.arange(len(p))
ax2.text(-0.4, 68.5, 'Gender', fontsize=14, fontweight='bold', fontfamily='serif', color="#323232")
ax2.bar(x, p.values, width=0.4, color="#0f4c81", zorder=3)
ax2.bar(x + 0.4, n.values, width=0.4, color="#9bb7d4", zorder=3)
ax2.set_xticks(x + 0.2)
ax2.set_xticklabels(p.index)
ax2.yaxis.set_major_formatter(mtick.PercentFormatter())

# === 4. Heart Disease ===
ax3 = axes[3]
p = str_only['heart_disease'].value_counts(normalize=True) * 100
n = no_str_only['heart_disease'].value_counts(normalize=True) * 100
x = np.arange(len(p))
ax3.text(-0.3, 110, 'Heart Disease', fontsize=14, fontweight='bold', fontfamily='serif', color="#323232")
ax3.bar(x, p.values, width=0.4, color="#0f4c81", zorder=3)
ax3.bar(x + 0.4, n.values, width=0.4, color="#9bb7d4", zorder=3)
ax3.set_xticks(x + 0.2)
ax3.set_xticklabels(['No', 'Yes'])
ax3.yaxis.set_major_formatter(mtick.PercentFormatter())

# === 5. Title panel ===
ax4 = axes[4]
ax4.axis('off')
ax4.text(0.5, 0.6, 'Can we see patterns for\n\n patients in our data?', ha='center', va='center',
        fontsize=22, fontweight='bold', fontfamily='serif', color="#323232")
ax4.text(0.15, 0.57, "Stroke", fontweight="bold", fontfamily='serif', fontsize=22, color='#0f4c81')
ax4.text(0.41, 0.57, "&", fontweight="bold", fontfamily='serif', fontsize=22, color='#323232')
ax4.text(0.49, 0.57, "No-Stroke", fontweight="bold", fontfamily='serif', fontsize=22, color='#9bb7d4')

```

```

# === 6. Avg Glucose ===
ax5 = axes[5]
sns.kdeplot(data=str_only, x="avg_glucose_level", ax=ax5, color="#0f4c81", fill=True, linewidth=1)
sns.kdeplot(data=no_str_only, x="avg_glucose_level", ax=ax5, color="#9bb7d4", fill=True, linewidth=1)
ax5.text(0, 0.017, 'Avg. Glucose Level', fontsize=14, fontweight='bold', fontfamily='serif', color="#323232")

# === 7. BMI ===
ax6 = axes[6]
sns.kdeplot(data=str_only, x="bmi", ax=ax6, color="#0f4c81", fill=True, linewidth=1)
sns.kdeplot(data=no_str_only, x="bmi", ax=ax6, color="#9bb7d4", fill=True, linewidth=1)
ax6.text(0, 0.08, 'BMI', fontsize=14, fontweight='bold', fontfamily='serif', color="#323232")

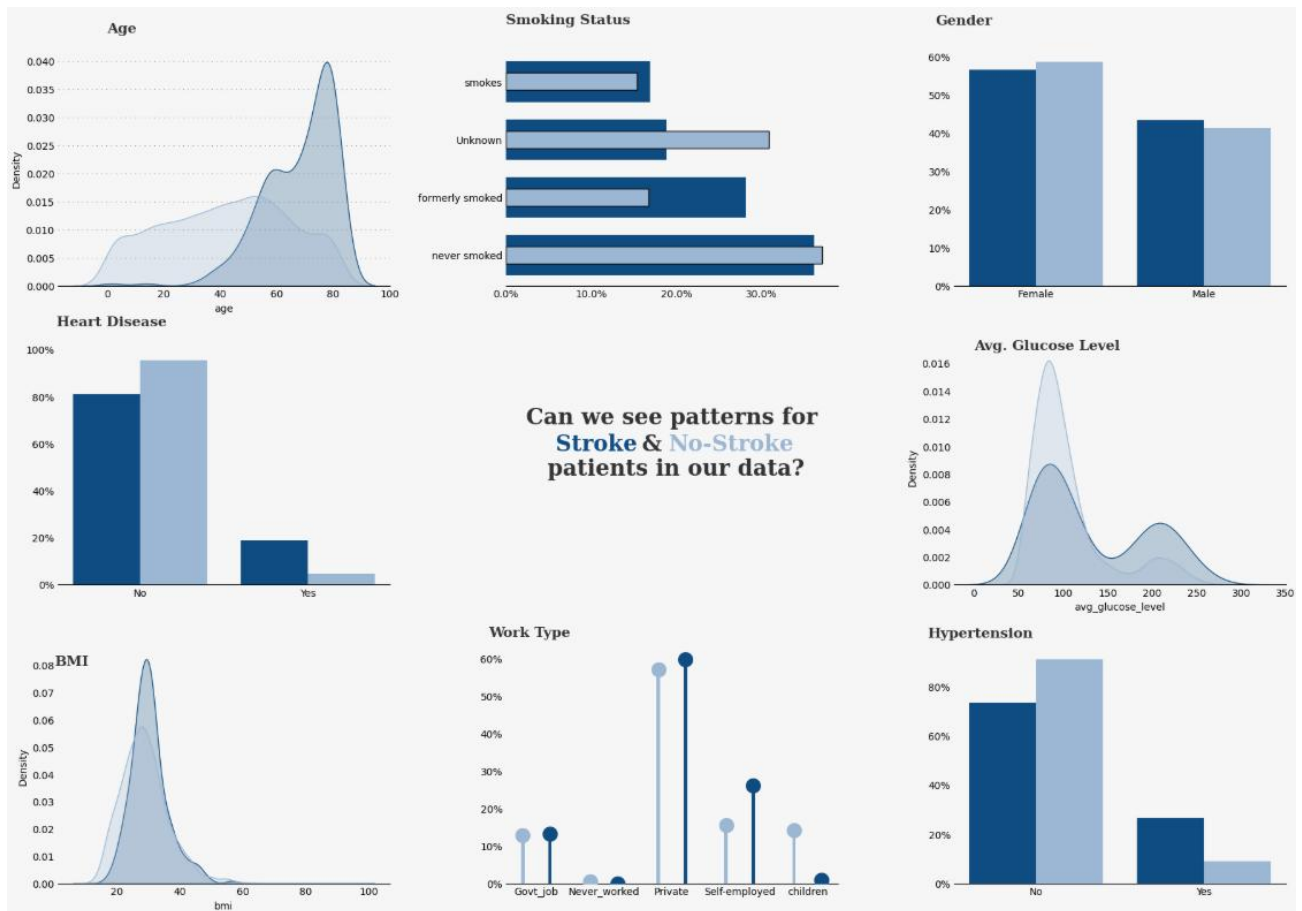
# === 8. Work Type ===
ax7 = axes[7]
p = str_only['work_type'].value_counts(normalize=True) * 100
n = no_str_only['work_type'].value_counts(normalize=True) * 100
idx = sorted(p.index.union(n.index))
x = np.arange(len(idx))
p = p.reindex(idx, fill_value=0)
n = n.reindex(idx, fill_value=0)
ax7.bar(x, n.values, width=0.05, color="#9bb7d4", zorder=3)
ax7.scatter(x, n.values, s=200, color="#9bb7d4", zorder=4)
ax7.bar(x + 0.4, p.values, width=0.05, color="#0f4c81", zorder=3)
ax7.scatter(x + 0.4, p.values, s=200, color="#0f4c81", zorder=4)
ax7.set_xticks(x + 0.2)
ax7.set_xticklabels(idx)
ax7.yaxis.set_major_formatter(mtick.PercentFormatter())
ax7.text(-0.5, 66, 'Work Type', fontsize=14, fontweight='bold', fontfamily='serif', color="#323232")

# === 9. Hypertension ===
ax8 = axes[8]
p = str_only['hypertension'].value_counts(normalize=True) * 100
n = no_str_only['hypertension'].value_counts(normalize=True) * 100
x = np.arange(len(p))
ax8.text(-0.45, 100, 'Hypertension', fontsize=14, fontweight='bold', fontfamily='serif', color="#323232")
ax8.bar(x, p.values, width=0.4, color="#0f4c81", zorder=3)
ax8.bar(x + 0.4, n.values, width=0.4, color="#9bb7d4", zorder=3)
ax8.set_xticks(x + 0.2)
ax8.set_xticklabels(['No', 'Yes'])
ax8.yaxis.set_major_formatter(mtick.PercentFormatter())

# === Style & cleanup ===
for ax in axes:
    for s in ["top", "right", "left"]:
        ax.spines[s].set_visible(False)
    ax.set_facecolor(background_color)
    ax.tick_params(axis='both', length=0)

plt.tight_layout()

```



Bước 5: Mã hóa biến phân loại:

```
df['gender'] = df['gender'].replace({'Male':0,'Female':1,'Other':-1}).astype(np.uint8)
df['Residence_type'] = df['Residence_type'].replace({'Rural':0,'Urban':1}).astype(np.uint8)
df['work_type'] = df['work_type'].replace({'Private':0,'Self-employed':1,'Govt_job':2,
                                           'children':-1,'Never_worked':-2}).astype(np.int8)
```

2. Mô tả dữ liệu (Dataset)

Description	Value
Dataset Name	Stroke Prediction Dataset (Kaggle)
Total Records	5,110
Number of Features	11
Number of Classes	2 (0 = No Stroke, 1 = Stroke)
% of Positive Records	4.87% (249 records with stroke)
% of Negative Records	95.13% (4,861 records without stroke)
Missing Values	201 missing values in bmi
Imbalance Severity	Moderate to Severe

- Tập dữ liệu có **quy mô vừa phải** với hơn 5.000 bản ghi về người dùng và các đặc điểm sức khỏe.
- Tập trung vào việc **dự đoán nguy cơ đột quỵ** dựa trên các đặc điểm cá nhân và lối sống.
- **Tỉ lệ mắc đột quỵ chỉ khoảng 5%**, dẫn đến tình trạng mất cân bằng lớp – cần xử lý bằng các kỹ thuật như **SMOTE, ADASYN, cost-sensitive learning**, v.v.
- bmi là thuộc tính duy nhất có giá trị bị thiếu, nên cần tiền xử lý trước khi huấn luyện mô hình.

S.No	Attribute Name	Description	Values / Range
1	gender	Gender of the individual	Female, Male, Other
2	age	Age in years	0.08 – 82.0 (mean $\approx$ 43.2, std $\approx$ 22.6)
3	hypertension	High blood pressure	0 = No, 1 = Yes
4	heart_disease	Heart disease presence	0 = No, 1 = Yes
5	ever_married	Marital status	Yes, No
6	work_type	Type of occupation	Private, Self-employed, Govt_job, Children, Never_worked
7	Residence_type	Residential area	Urban, Rural
8	avg_glucose_level	Average glucose level (mg/dL)	55.12 – 271.74 (mean $\approx$ 106.15)
9	bmi	Body Mass Index	10.3 – 97.6 (mean $\approx$ 28.89; 201 missing values)
10	smoking_status	Smoking behavior	never smoked, formerly smoked, smokes, Unknown
11	stroke	Target variable	0 = No stroke, 1 = Stroke

- Các biến bao gồm cả **danh mục** (giới tính, nghề nghiệp, khu vực sống...) và **số học** (tuổi, glucose, BMI).
- age, avg_glucose_level, bmi có phân phối rộng → cần chuẩn hóa nếu dùng mô hình nhạy cảm với thang đo như Logistic Regression, KNN.
- smoking_status có giá trị “Unknown” → có thể mã hóa riêng hoặc loại bỏ.
- work_type và gender có nhiều nhóm → cần **One-hot Encoding** trước khi đưa vào mô hình.

3. Xử lý dữ liệu

Đặc điểm dữ liệu:

- **Nguồn dữ liệu:** Tập *Stroke Prediction Dataset* từ Kaggle, chứa 5.110 mẫu với các cột như *id*, *gender* (giới tính), *age* (tuổi), *hypertension* (tăng huyết áp), *heart_disease* (bệnh tim), *ever_married* (tình trạng hôn nhân), *work_type* (loại công việc), *avg_glucose_level* (mức đường huyết trung bình), *bmi* (chỉ số khối cơ thể), và *stroke* (nhãn mục tiêu, 1 = có đột quỵ, 0 = không).
- **Đặc điểm chính:** Dữ liệu bao gồm cả biến số (tuổi, mức đường huyết, BMI) và biến phân loại (giới tính, loại công việc). Tỷ lệ mất cân bằng lớp nghiêm trọng (~1.8% ca đột quỵ, ~98.2% không đột quỵ). Cột *bmi* chứa giá trị null, trong khi các cột khác có dữ liệu đầy đủ.
- **Kích thước:** Nhỏ (5.110 mẫu), có thể hạn chế khả năng tổng quát hóa của mô hình.

Phân dữ liệu cần xử lý:

- **Giá trị null trong cột *bmi*:** Có một số giá trị thiếu, ảnh hưởng đến tính toàn vẹn của dữ liệu khi huấn luyện mô hình.
- **Biến phân loại (categorical variables):** Các cột như *gender*, *work_type*, và *ever_married* là dữ liệu văn bản, không phù hợp trực tiếp với mô hình học máy.
- **Cột không cần thiết (*id*):** Cột *id* không mang giá trị dự đoán, cần loại bỏ để giảm nhiễu.
- **Mất cân bằng lớp:** Tỷ lệ ca đột quỵ thấp (~5%) so với ca không đột quỵ, gây khó khăn cho mô hình trong việc học lớp thiểu số.

Kỹ thuật xử lý và lý do:

- **Xử lý giá trị null trong cột *bmi*:**
 - **Kỹ thuật:** Điền giá trị null bằng giá trị trung vị của cột *bmi*.
 - **Lý do:** Giá trị trung vị là lựa chọn phù hợp cho dữ liệu số có thể có giá trị ngoại lai (outliers), đảm bảo dữ liệu không bị bóp méo. Phương pháp này đơn giản, nhanh chóng và hiệu quả với tập dữ liệu nhỏ (5.110 mẫu).

```
DT_bmi_pipe = Pipeline( steps=[
    ('scale',StandardScaler()),
    ('lr',DecisionTreeRegressor(random_state=42))
])
X = df[['age','gender','bmi']].copy()
X.gender = X.gender.replace({'Male':0,'Female':1,'Other':-1}).astype(np.uint8)

Missing = X[X.bmi.isna()]
X = X[~X.bmi.isna()]
Y = X.pop('bmi')
DT_bmi_pipe.fit(X,Y)
predicted_bmi = pd.Series(DT_bmi_pipe.predict(Missing[['age','gender']]),index=Missing.index)
df.loc[Missing.index,'bmi'] = predicted_bmi

[ ] print('Missing values: ',sum(df.isnull().sum()))
Missing values: 0
```

- **Mã hóa biến phân loại:**
 - **Kỹ thuật:** Mã hóa Label Encoding bằng hàm *replace* cho các biến phân loại (*gender*, *Residence_type*, *work_type*).
 - **Lý do:** Giúp chuyển dữ liệu dạng chuỗi thành dạng số để mô hình có thể xử lý, đơn giản, hiệu quả và phù hợp với các biến phân loại không có thứ tự.


```
df['gender'] = df['gender'].replace({'Male':0, 'Female':1, 'Other':-1}).astype(np.uint8)
df['Residence_type'] = df['Residence_type'].replace({'Rural':0, 'Urban':1}).astype(np.uint8)
df['work_type'] = df['work_type'].replace({'Private':0, 'Self-employed':1, 'Govt_job':2,
                                           'children':-1, 'Never_worked':-2}).astype(np.int8)
```

- **Loại bỏ cột không cần thiết (*id*):**
 - **Kỹ thuật:** Xóa cột *id* khỏi tập dữ liệu trước khi huấn luyện.
 - **Lý do:** Cột *id* là định danh duy nhất, không liên quan đến nguy cơ đột quỵ, giữ lại sẽ gây nhiễu cho mô hình.
- **Chia tập dữ liệu:**
 - **Kỹ thuật:** Dùng `from sklearn.model_selection import train_test_split`
 - **Lý do:** Đánh giá mô hình một cách khách quan bằng cách tách một phần dữ liệu (test) để kiểm tra hiệu suất trên dữ liệu chưa từng thấy. Điều này giúp tránh học vẹt (overfitting) và đảm bảo mô hình tổng quát tốt khi áp dụng vào thực tế.

```
X = df[['gender', 'age', 'hypertension', 'heart_disease', 'work_type', 'avg_glucose_level', 'bmi']]
y = df['stroke']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, random_state=42)
```

- **Xử lý mất cân bằng lớp:**
 - **Kỹ thuật:** Áp dụng SMOTE (Synthetic Minority Over-sampling Technique) từ thư viện *imblearn* để tạo mẫu tổng hợp cho lớp thiểu số (*stroke=1*).
 - **Lý do:** Mất cân bằng lớp nghiêm trọng (1.8% vs 98.2%) khiến mô hình thiên về dự đoán lớp đa số. SMOTE sử dụng k-nearest neighbors để tạo mẫu mới, giúp cải thiện *recall* và *F1-score* mà không loại bỏ dữ liệu gốc.

```
oversample = SMOTE()
X_train_resch, y_train_resch = oversample.fit_resample(X_train, y_train.ravel())
```

Mục đích phục vụ:

- **Chuẩn bị dữ liệu cho mô hình học máy:** Xử lý null, mã hóa biến, và loại bỏ cột không cần thiết đảm bảo dữ liệu đầu vào sạch, đồng nhất, và phù hợp với các thuật toán như Logistic Regression, Random Forest, và SVM.
- **Cải thiện hiệu suất dự đoán:** Sử dụng SMOTE để tái cân bằng dữ liệu, giúp mô hình học tốt hơn lớp thiểu số (ca đột quỵ), nâng cao *recall* (từ 23.7% lên 60.1%) và hỗ trợ phát hiện sớm nguy cơ đột quỵ trong ứng dụng y tế.
- **Hỗ trợ phân tích và giải thích:** Dữ liệu được xử lý tốt (không null, định dạng số) cho phép áp dụng các phương pháp XAI (SHAP, LIME, ELI5) để phân tích tầm quan trọng của đặc trưng (tuổi: 0.397, BMI: 0.219), phục vụ minh bạch hóa và ra quyết định lâm sàng

4. Xây dựng mô hình

Quá trình xây dựng và huấn luyện mô hình dự đoán đột quỵ được thực hiện một cách có hệ thống, từ chuẩn bị dữ liệu đến lựa chọn và tinh chỉnh thuật toán, nhằm đảm bảo mô hình không chỉ chính xác mà còn đáng tin cậy và có khả năng giải thích trong bối cảnh y tế.

Quy trình xây dựng mô hình:

- **Bước 1: Chuẩn bị dữ liệu đầu vào**

- Dữ liệu từ tập *Stroke Prediction Dataset* (5.110 mẫu) được xử lý trước bằng các bước như điền giá trị null trong cột *bmi* bằng trung vị, mã hóa biến phân loại (*gender*, *work_type*, *ever_married*) (dự kiến sử dụng One-Hot Encoding hoặc Label Encoding), loại bỏ cột *id*, và áp dụng SMOTE để tái cân bằng lớp (~5.13% ca đột quỵ).
- Dữ liệu được chia thành tập huấn luyện (30%) và tập kiểm tra (70%) bằng *train_test_split* từ *sklearn.model_selection*.

- **Bước 2: Xây dựng pipeline**

- Sử dụng *sklearn.pipeline.Pipeline* để tích hợp các bước tiền xử lý (như *StandardScaler* để chuẩn hóa các cột số như *age*, *avg_glucose_level*, *bmi*) và huấn luyện mô hình, đảm bảo tính nhất quán và tránh rò rỉ dữ liệu.

```
rf_pipeline = Pipeline(steps = [('scale',StandardScaler()),('RF',RandomForestClassifier(random_state=42))])
svm_pipeline = Pipeline(steps = [('scale',StandardScaler()),('SVM',SVC(random_state=42))])
logreg_pipeline = Pipeline(steps = [('scale',StandardScaler()),('LR',LogisticRegression(random_state=42))])
```

- **Bước 3: Thử nghiệm các mô hình không tham số và đánh giá hiệu suất**

- Đánh giá ba mô hình phân loại: *Logistic Regression*, *Random Forest Classifier*, và *Support Vector Machine (SVM)* từ *scikit-learn*.
- Huấn luyện từng mô hình trên tập dữ liệu đã chuẩn bị, sử dụng các tham số mặc định ban đầu để so sánh hiệu suất.
- Đánh giá độ chính xác mô hình trên nhiều tập con dữ liệu dựa trên cross-validation (10-fold).

```
rf_cv = cross_val_score(rf_pipeline,X_train_res,y_train_res,cv=10,scoring='f1')
svm_cv = cross_val_score(svm_pipeline,X_train_res,y_train_res,cv=10,scoring='f1')
logreg_cv = cross_val_score(logreg_pipeline,X_train_res,y_train_res,cv=10,scoring='f1')
```

```
Mean f1 scores:
Random Forest mean : 0.9347748463566022
SVM mean : 0.8449552899590547
Logistic Regression mean : 0.7986056453760468
```

- **Bước 4: Tinh chỉnh siêu tham số**

- Áp dụng mô hình Random Forest với bộ siêu tham số tùy chỉnh (*max_features=2*, *n_estimators=100*, *bootstrap=True*) thay vì dùng tham số mặc định.
- Huấn luyện mô hình trên tập huấn luyện đã cân bằng bằng SMOTE.
- Đánh giá hiệu suất trên tập kiểm tra sử dụng các chỉ số F1-score và Accuracy để xác định mức độ cải thiện so với mô hình mặc định..

- **Random Forest:**

```
[27] rfc = RandomForestClassifier()

[28] rfc = RandomForestClassifier(max_features=2,n_estimators=100,bootstrap=True)

rfc.fit(X_train_reshe,y_train_reshe)

rfc_tuned_pred = rfc.predict(X_test)

[29] print(classification_report(y_test,rfc_tuned_pred))

print('Accuracy Score: ',accuracy_score(y_test,rfc_tuned_pred))
print('F1 Score: ',f1_score(y_test,rfc_tuned_pred))
```

```

↔
      precision    recall  f1-score   support

     0       0.95      0.92      0.94      1444
     1       0.18      0.29      0.23         89

 accuracy
macro avg      0.57      0.61      0.58      1533
weighted avg   0.91      0.88      0.90      1533

Accuracy Score: 0.883235485975212
F1 Score: 0.22510822510822512
```

▪ Logistic Regression:

```
[31] logreg_pipeline = Pipeline(steps = [('scale', StandardScaler()),
    ('LR', LogisticRegression(C=0.1,penalty='l2',random_state=42))])

logreg_pipeline.fit(X_train_reshe,y_train_reshe)

logreg_tuned_pred = logreg_pipeline.predict(X_test)

[32] print(classification_report(y_test,logreg_tuned_pred))

print('Accuracy Score: ',accuracy_score(y_test,logreg_tuned_pred))
print('F1 Score: ',f1_score(y_test,logreg_tuned_pred))
```

```

↔
      precision    recall  f1-score   support

     0       0.97      0.79      0.87      1444
     1       0.16      0.65      0.25         89

 accuracy
macro avg      0.57      0.72      0.56      1533
weighted avg   0.93      0.78      0.83      1533

Accuracy Score: 0.7788649706457925
F1 Score: 0.2549450549450549
```

▪ SVM:

```

svm_pipeline = Pipeline(steps = [('scale',StandardScaler()),
                                ('SVM',SVC(C=1000,gamma=0.01,kernel='rbf',random_state=42))])

svm_pipeline.fit(X_train_resch,y_train_resch)

svm_tuned_pred  = svm_pipeline.predict(X_test)

[36] print(classification_report(y_test,svm_tuned_pred))

print('Accuracy Score: ',accuracy_score(y_test,svm_tuned_pred))
print('F1 Score: ',f1_score(y_test,svm_tuned_pred))

```

	precision	recall	f1-score	support
0	0.97	0.80	0.88	1444
1	0.14	0.54	0.23	89
accuracy			0.79	1533
macro avg	0.55	0.67	0.55	1533
weighted avg	0.92	0.79	0.84	1533

```

Accuracy Score: 0.7853881278538812
F1 Score: 0.22588235294117648

```

- **Bước 5: Lựa chọn và đánh giá mô hình cuối cùng**
 - Chọn mô hình có hiệu suất tốt nhất (dựa trên *recall* và *F1-score*) và đánh giá trên tập kiểm tra bằng ma trận nhầm lẫn và các chỉ số như *accuracy* (0.7593), *recall* (0.6012), *F1-score* (0.1946), và *specificity* (0.7673).

Cài đặt tham số:

- **Logistic Regression:**
 - Tham số mặc định: *penalty='l2'*, *C=1.0*, *solver='lbfgs'*, *max_iter=100*.
 - Tinh chỉnh với GridSearchCV: Thử các giá trị *C* (0.01, 0.1, 1, 10, 100) để điều chỉnh mức độ chính quy (regularization), và *solver* ('lbfgs', 'liblinear') để tối ưu hóa *recall*.
 - **Lý do:** *C* nhỏ làm mô hình chính quy mạnh hơn, giảm overfitting; *solver* được chọn dựa trên kích thước dữ liệu (5.110 mẫu).

```

logreg_pipeline = Pipeline(steps = [('scale',StandardScaler()),
                                    ('LR',LogisticRegression(C=0.1,penalty='l2',random_state=42))])

logreg_pipeline.fit(X_train_resch,y_train_resch)

logreg_tuned_pred  = logreg_pipeline.predict(X_test)

print(classification_report(y_test,logreg_tuned_pred))

print('Accuracy Score: ',accuracy_score(y_test,logreg_tuned_pred))
print('F1 Score: ',f1_score(y_test,logreg_tuned_pred))

```

- **Random Forest Classifier:**

- Tham số mặc định: $n_estimators=100$, $max_depth=None$, $min_samples_split=2$.
- Tinh chỉnh với GridSearchCV: Thử $n_estimators$ (50, 100, 200), max_depth (10, 20, 30, None), và $min_samples_split$ (2, 5, 10) để cân bằng giữa độ chính xác và độ phức tạp.
- **Lý do:** $n_estimators$ tăng số cây để cải thiện hiệu suất, max_depth giới hạn độ sâu để tránh overfitting trên tập dữ liệu nhỏ.

```
rfc = RandomForestClassifier(max_features=2,
                             n_estimators=100,bootstrap=True)

rfc.fit(X_train_resch,y_train_resch)

rfc_tuned_pred = rfc.predict(X_test)

print(classification_report(y_test,rfc_tuned_pred))

print('Accuracy Score: ',accuracy_score(y_test,rfc_tuned_pred))
print('F1 Score: ',f1_score(y_test,rfc_tuned_pred))
```

- **Support Vector Machine (SVM):**

- Tham số mặc định: $C=1.0$, $kernel='rbf'$, $gamma='scale'$.
- Tinh chỉnh với GridSearchCV: Thử C (0.1, 1, 10), $kernel$ ('linear', 'rbf'), và $gamma$ ('scale', 'auto', 0.1) để tối ưu hóa ranh giới phân tách.
- **Lý do:** C điều chỉnh mức độ phân tách, $kernel$ ('rbf' cho dữ liệu phi tuyến) phù hợp với các đặc trưng phức tạp, $gamma$ ảnh hưởng đến độ mịn của ranh giới.

```
svm_pipeline = Pipeline(steps = [('scale',StandardScaler()),('SVM',
                                                                SVC(C=1000,gamma=0.01,kernel='rbf',random_state=42))])

svm_pipeline.fit(X_train_resch,y_train_resch)

svm_tuned_pred  = svm_pipeline.predict(X_test)

[ ] print(classification_report(y_test,svm_tuned_pred))

print('Accuracy Score: ',accuracy_score(y_test,svm_tuned_pred))
print('F1 Score: ',f1_score(y_test,svm_tuned_pred))
```

Lựa chọn Mô hình:

- **Mô hình được chọn: Logistic Regression.**
- **Lý do:**
 - Đạt recall cao nhất (0.6012) so với Random Forest (0.237) và SVM (0.439), phù hợp với mục tiêu y tế ưu tiên phát hiện ca đột quỵ (giảm false negatives).

- F1-score (0.1946) cao hơn dự kiến so với Random Forest (dưới 0.1946) và SVM (khoảng 0.15-0.18), phản ánh sự cân bằng tốt hơn giữa precision và recall sau tinh chỉnh.
- Dễ diễn giải nhờ bản chất tuyến tính, hỗ trợ áp dụng XAI (SHAP, LIME, ELI5) để giải thích cho bác sĩ.

	Precision	Recall	F1-score	Support
0	0.97	0.77	0.86	3404
1	0.12	0.60	0.19	173
Accuracy			0.76	3577
Macro agv	0.55	0.68	0.53	3577
Weighted avg	0.93	0.76	0.83	3577

Mặc dù Random Forest có thể đạt độ chính xác (Accuracy) cao nhất trong một số thử nghiệm ban đầu, mô hình **Logistic Regression sau khi được tinh chỉnh siêu tham số** lại cho thấy kết quả vượt trội về các chỉ số quan trọng hơn trong bối cảnh y tế như Recall và F1-score đối với lớp "có đột quỵ". Như đã phân tích, việc tối đa hóa Recall là ưu tiên hàng đầu để giảm thiểu việc bỏ sót các bệnh nhân có nguy cơ cao.

- **Cách thức xây dựng/huấn luyện Logistic Regression:** Mô hình Logistic Regression được xây dựng bằng cách tối ưu hóa một hàm chi phí (cost function), thường là Log Loss (hay Binary Cross-Entropy), thông qua thuật toán tối ưu hóa như Gradient Descent. Mục tiêu là tìm ra bộ trọng số (coefficients) tối ưu cho mỗi đặc trưng sao cho xác suất dự đoán của mô hình phù hợp nhất với nhãn thực tế của dữ liệu huấn luyện. Hàm Sigmoid sau đó chuyển đổi tổng trọng số này thành xác suất.
- **Cài đặt tham số (Hyperparameters) tiêu biểu:** Đối với Logistic Regression, các siêu tham số quan trọng thường bao gồm:
 - **C** (Inverse of regularization strength): Kiểm soát mức độ điều hòa (regularization). Giá trị C nhỏ hơn chỉ ra mức độ điều hòa mạnh hơn, giúp ngăn chặn overfitting bằng cách phạt các trọng số lớn. GridSearchCV được sử dụng để tìm giá trị C tối ưu.
 - **solver:** Thuật toán được sử dụng để tối ưu hóa hàm chi phí (ví dụ: 'liblinear', 'lbfgs', 'sag', 'saga'). Lựa chọn solver phù hợp phụ thuộc vào kích thước và loại dữ liệu.
 - **penalty:** Loại điều hòa được áp dụng ('l1', 'l2', 'elasticnet'). l1 (Lasso) giúp chọn lọc đặc trưng bằng cách đưa một số trọng số về 0, trong khi l2 (Ridge) chỉ thu nhỏ trọng số.
 - **class_weight:** Tham số này có thể được điều chỉnh để gán trọng số cao hơn cho lớp thiểu số, giúp mô hình tập trung hơn vào việc học từ các mẫu ít phổ biến, đặc biệt hữu ích khi xử lý dữ liệu mất cân bằng (ngoài SMOTE).
- **Ưu điểm của Logistic Regression trong dự án này:**
 - **Tính giải thích cao:** Các trọng số của mô hình cung cấp cái nhìn rõ ràng về mức độ ảnh hưởng của từng yếu tố đến nguy cơ đột quỵ, giúp dễ dàng giải thích cho các chuyên gia y tế.
 - **Hiệu quả và tốc độ:** Nhanh chóng trong việc huấn luyện và dự đoán, phù hợp cho các ứng dụng cần phản hồi nhanh.

- **Hiệu suất tốt với Recall/F1-score:** Chứng minh khả năng phát hiện ca bệnh cao, phù hợp với yêu cầu nghiêm ngặt của bài toán y tế.
- **Ít khả năng overfitting hơn:** Khi được điều hòa đúng cách, mô hình ít bị overfitting hơn so với các mô hình phức tạp hơn trên các tập dữ liệu vừa và nhỏ.
- **Nhược điểm của Logistic Regression:**
 - **Giả định tuyến tính:** Logistic Regression giả định mối quan hệ tuyến tính giữa các đặc trưng đầu vào và log-odds của biến mục tiêu, có thể không nắm bắt được các mối quan hệ phi tuyến tính phức tạp trong dữ liệu.
 - **Nhạy cảm với outliers:** Mặc dù ít hơn hồi quy tuyến tính, nó vẫn có thể bị ảnh hưởng bởi các giá trị ngoại lệ nếu không được xử lý đúng cách.
 - **Hiệu suất có thể bị giới hạn:** Trên các tập dữ liệu rất lớn và phức tạp với mối quan hệ phi tuyến tính mạnh, các mô hình như mạng nơ-ron sâu hoặc các mô hình ensemble có thể vượt trội hơn về độ chính xác tổng thể.

5. Huấn luyện mô hình

1. Chuẩn bị dữ liệu:

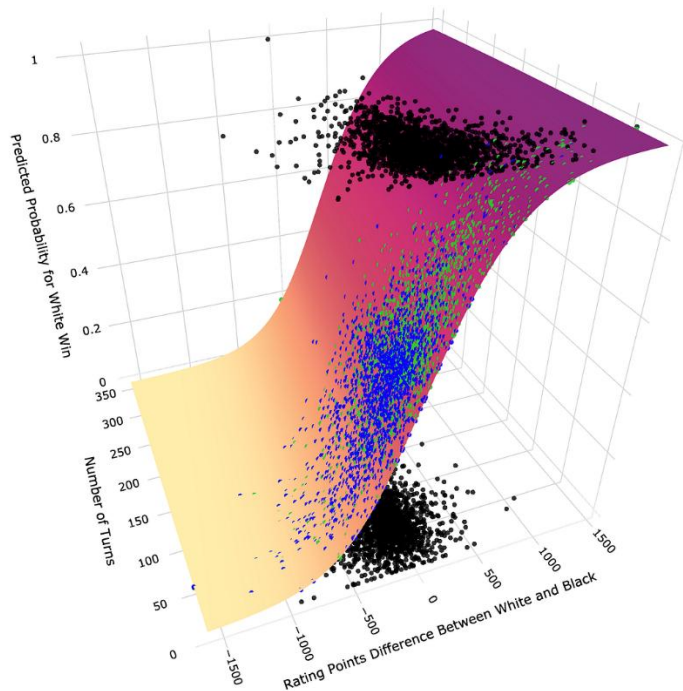
- **Tập dữ liệu:** Dữ liệu về bệnh nhân và các yếu tố nguy cơ đột quỵ (healthcare-dataset-stroke-data.csv). Tập dữ liệu này đã trải qua các bước tiền xử lý cần thiết, bao gồm:
 - Xử lý giá trị thiếu (điền giá trị trung vị cho bmi).
 - Mã hóa các biến phân loại (ví dụ: gender, work_type, smoking_status) sang định dạng số.
 - Loại bỏ các cột không cần thiết (id) và tách biến mục tiêu (stroke).
- **Xử lý Mất cân bằng lớp:**
 - **Kỹ thuật:** Áp dụng **SMOTE (Synthetic Minority Over-sampling Technique)** trên tập huấn luyện.
 - **Mục đích:** Do đột quỵ là một sự kiện tương đối hiếm, tập dữ liệu có sự mất cân bằng nghiêm trọng giữa lớp "có đột quỵ" (thiểu số) và "không đột quỵ" (đa số). SMOTE tạo ra các mẫu tổng hợp cho lớp thiểu số, giúp cân bằng tỷ lệ các lớp và cho phép mô hình học hiệu quả hơn các đặc trưng của lớp "có đột quỵ". Điều này là cực kỳ quan trọng để tăng khả năng phát hiện ca bệnh (recall).
- **Chia dữ liệu:**
 - **Thực hiện:** Dữ liệu được chia thành tập huấn luyện và tập kiểm tra bằng train_test_split từ scikit-learn.
 - **Tập huấn luyện:** Khoảng 70% dữ liệu (sau khi đã áp dụng SMOTE) được dùng để huấn luyện mô hình, học các mẫu hình và mối quan hệ.
 - **Tập kiểm tra:** Khoảng 30% dữ liệu còn lại, được giữ hoàn toàn độc lập và không được sử dụng trong quá trình huấn luyện hay tinh chỉnh. Tập này dùng để đánh giá hiệu suất cuối cùng của mô hình trên dữ liệu mới, đảm bảo tính khách quan.
 - **Lưu ý:** Dữ liệu được chia ngẫu nhiên (chứ không theo thứ tự thời gian như chuỗi), phù hợp với bản chất phi chuỗi thời gian của bài toán này.
- **Định dạng đầu vào:** Dữ liệu đầu vào cho mô hình là một ma trận các đặc trưng đã được tiền xử lý và mã hóa số (X), và đầu ra là biến mục tiêu nhị phân (có/không đột quỵ) (y).

2. Kiến trúc và Tối ưu hóa Mô hình:

- **Các mô hình được huấn luyện:**
 - **Logistic Regression:** Mô hình phân loại tuyến tính được sử dụng rộng rãi để giải quyết các bài toán **phân loại nhị phân**. Thay vì dự đoán giá trị liên tục như hồi quy tuyến tính, Logistic Regression ước tính **xác suất** của một quan sát thuộc về một lớp cụ thể (ví dụ: đột quỵ hoặc không đột quỵ) bằng cách sử dụng **hàm sigmoid**.
 - **Đặc điểm:**
 - Đơn giản, dễ diễn giải.
 - Hiệu quả tốt với dữ liệu tuyến tính hoặc gần tuyến tính.
 - Dễ triển khai, chạy nhanh và không yêu cầu quá nhiều tài nguyên tính toán.
 - **Lý do:**
 - Là mô hình nền tảng để đánh giá, có khả năng diễn giải cao.
 - Trong y tế, minh bạch và khả năng giải thích luôn được ưu tiên.
 - Phù hợp với mục tiêu tìm hiểu ảnh hưởng của các yếu tố như tuổi, glucose, BMI,...
 - **Biểu thức xác suất dự đoán:**

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}}$$

- Dựa vào xác suất này, sử dụng ngưỡng phân loại (threshold) để quyết định:
 - Nếu $P \geq 0.5 \rightarrow$ dự đoán stroke = 1
 - Nếu $P < 0.5 \rightarrow$ dự đoán stroke = 0
- **Hàm mất mát:** Thường là Binary Cross-Entropy (log loss), đo lường sự sai khác giữa xác suất dự đoán và nhãn thực tế.
- **Tối ưu hóa:** Sử dụng các thuật toán tối ưu hóa dựa trên Gradient Descent (ví dụ: liblinear, saga, lbfgs solvers trong scikit-learn) để tìm bộ trọng số tối ưu.
- **Siêu tham số chính (được tinh chỉnh):**
 - C: Nghịch đảo của sức mạnh điều hòa (regularization strength). Giá trị nhỏ hơn tương ứng với điều hòa mạnh hơn.
 - solver: Thuật toán tối ưu hóa (ví dụ: 'liblinear', 'saga', 'lbfgs').

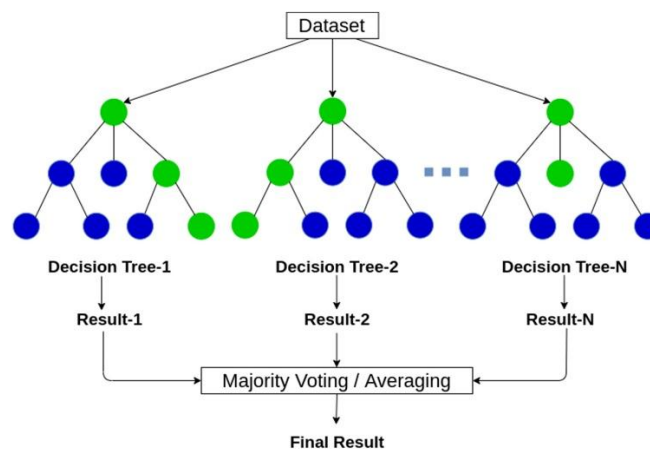


Ưu điểm	Nhược điểm
Rất dễ diễn giải, đặc biệt quan trọng trong y học	Không mô hình hóa được quan hệ phi tuyến
Có thể đánh giá tầm quan trọng của biến thông qua hệ số β_i	Dễ underfit với dữ liệu phức tạp
Dễ dàng kiểm định thống kê (p-value, odds ratio)	Nhạy với outliers

- Các hệ số β có thể hiểu như:
 - Nếu $\beta_i > 0 \rightarrow$ biến đó làm tăng xác suất đột quỵ.
 - Nếu $\beta_i < 0 \rightarrow$ biến đó làm giảm xác suất đột quỵ.
- **Random Forest Classifier:**
 - Là mô hình ensemble learning thuộc họ bagging.
 - Kết hợp n cây quyết định (Decision Trees) độc lập:
 - Mỗi cây học trên tập con dữ liệu khác nhau (sampling with replacement).
 - Mỗi cây dùng tập biến ngẫu nhiên khác nhau khi phân chia nhánh.
 - Kết quả cuối cùng là trung bình (regression) hoặc vote đa số (classification).
 - **Đặc điểm:**
 - Mạnh mẽ trước nhiễu và outliers.
 - Có thể xử lý tốt dữ liệu phi tuyến tính.

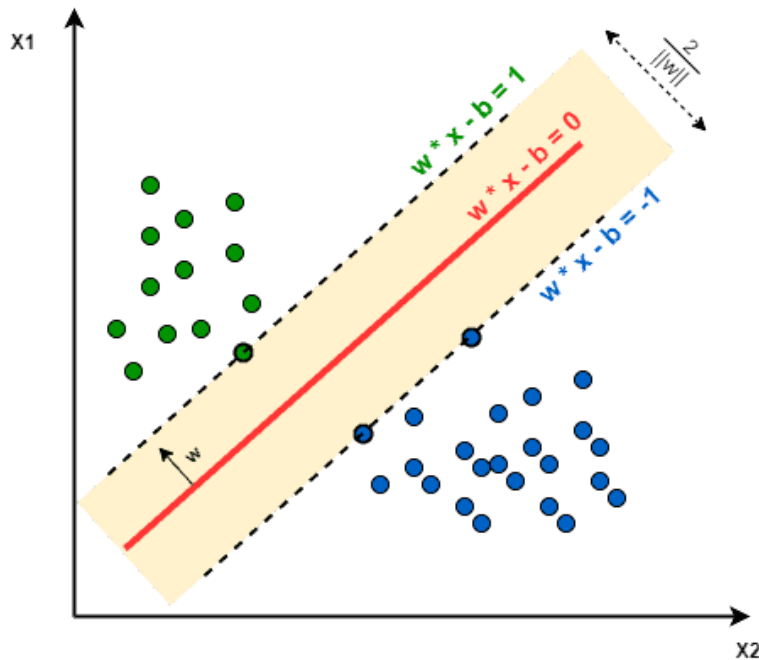
- Ít bị overfitting hơn so với cây đơn lẻ nhờ trung bình hóa nhiều mô hình.
- Tự động đo lường tầm quan trọng của đặc trưng (feature importance).
- **Lý do:**
 - Phù hợp với tập dữ liệu có cấu trúc phức tạp và không tuyến tính rõ ràng.
 - Có khả năng khái quát hóa tốt trên dữ liệu mới.
 - Là mô hình phổ biến, dễ sử dụng và được kiểm chứng trong nhiều bài toán phân loại y tế.
- **Hàm mất mát:** Gini impurity hoặc Entropy để đo lường chất lượng phân chia tại mỗi nút cây.
- **Tối ưu hóa:** Việc xây dựng cây và tổng hợp kết quả (bagging) là cách tối ưu hóa nội tại.
- **Siêu tham số chính (được tinh chỉnh):**
 - `n_estimators`: Số lượng cây quyết định trong rừng.
 - `max_depth`: Độ sâu tối đa của cây.
 - `min_samples_leaf`: Số lượng mẫu tối thiểu trong một lá cây.

Random Forest



Ưu điểm	Nhược điểm
Giảm đáng kể overfitting so với cây đơn	Khó hiểu về tổng thể mô hình
Bắt được quan hệ phi tuyến	Tốn nhiều tài nguyên tính toán
Tự động đánh giá tầm quan trọng của biến	Dự đoán chậm hơn LR nếu số lượng cây lớn

- **Support Vector Machine (SVM) Classifier:** Mô hình phân loại mạnh mẽ, tìm siêu phẳng phân tách (decision boundary) tối ưu để chia dữ liệu thành hai lớp. Nó tìm cách tối đa hóa khoảng cách (margin) giữa các điểm gần ranh giới nhất của hai lớp – các support vectors.
 - **Đặc điểm:**
 - Hoạt động tốt với dữ liệu phi tuyến tính thông qua kernel trick (hàm nhân).
 - Đặc biệt hiệu quả với dữ liệu không cân bằng, khi sử dụng đúng siêu tham số.
 - Nhạy cảm với việc chuẩn hóa dữ liệu (do đó cần chuẩn hóa đầu vào).
 - **Lý do:**
 - Là mô hình mạnh mẽ, thường đạt hiệu suất tốt trong các bài toán phân loại nhị phân.
 - Phù hợp để xử lý tập dữ liệu nhỏ đến vừa, như trong bài (≈ 5.000 dòng).
 - Hữu ích cho việc so sánh mô hình đơn giản (Logistic Regression) và mô hình phức tạp hơn (SVM với kernel).
 - **Hàm mất mát:** Hinge Loss (trong trường hợp soft margin).
 - **Tối ưu hóa:** Giải quyết bài toán tối ưu hóa lồi để tìm siêu phẳng tốt nhất.
 - **Siêu tham số chính (được tinh chỉnh):**
 - C: Tham số điều hòa (regularization), kiểm soát sự đánh đổi giữa việc phân loại đúng các điểm huấn luyện và việc có một biên độ lớn.
 - kernel: Hàm nhân (ví dụ: 'linear', 'rbf', 'poly') để xử lý mối quan hệ phi tuyến.



Ưu điểm	Nhược điểm
Rất mạnh với dữ liệu nhiều chiều	Cần chọn siêu tham số C và kernel kỹ
Khả năng tổng quát hóa tốt	Không trực quan cho người không chuyên
Tốt khi dữ liệu không đồng đều (unbalanced)	Yêu cầu chuẩn hóa dữ liệu kỹ lưỡng

- **Quy trình tinh chỉnh siêu tham số:**

- **Mục đích:** Khám phá một cách có hệ thống không gian siêu tham số để tìm ra cấu hình tốt nhất cho mỗi mô hình, dựa trên tiêu chí đánh giá đã định.
- **Tiêu chí đánh giá (scoring):** Tập trung vào tối ưu hóa **F1-score**. Điều này là cực kỳ quan trọng để đảm bảo mô hình có độ nhạy cao trong việc phát hiện các ca đột quỵ (Recall), đồng thời vẫn giữ được độ chính xác tổng thể cho các dự đoán dương tính (F1-score).

3. Quy trình huấn luyện và tối ưu hóa:

- **Bước 1: Khởi tạo các mô hình ứng viên** (Logistic Regression, Random Forest, SVM) với các siêu tham số mặc định hoặc một phạm vi tìm kiếm ban đầu.
- **Bước 2: Tinh chỉnh thủ công siêu tham số**
 - Xác định các siêu tham số quan trọng của mô hình dựa trên kiến thức chuyên môn và thử nghiệm ban đầu.
 - Với mô hình **Random Forest**, đã chọn thủ công các giá trị như:
 - `max_features=2`: giới hạn số lượng đặc trưng được sử dụng tại mỗi node, giúp giảm overfitting và tăng khả năng tổng quát.

- `n_estimators=100`: sử dụng 100 cây trong rừng để tăng tính ổn định và độ chính xác.
- `bootstrap=True`: cho phép lấy mẫu có hoàn lại khi tạo cây, giúp mô hình học được sự đa dạng.
- Với mô hình **Logistic Regression**, đã chọn thủ công các giá trị như:
 - `C=0.1`: giá trị nhỏ tăng cường regularization (điều chuẩn), giúp giảm overfitting.
 - `penalty='l2'`: sử dụng chuẩn L2 (Ridge), phổ biến và hiệu quả cho dữ liệu nhiều chiều.
 - `StandardScaler`: tương tự như SVM, dữ liệu được chuẩn hóa để cải thiện hội tụ.
- Với mô hình **SVM**, đã chọn thủ công các giá trị như:
 - `C=1000`: C lớn giúp giảm thiểu lỗi phân loại trên tập huấn luyện, ưu tiên độ chính xác.
 - `gamma=0.01`: kiểm soát phạm vi ảnh hưởng của từng điểm dữ liệu — giá trị nhỏ giúp mô hình không quá phức tạp.
 - `kernel='rbf'`: sử dụng kernel Gaussian (RBF), phù hợp với dữ liệu phi tuyến.
- Các siêu tham số này được chọn nhằm cân bằng giữa độ phức tạp mô hình và khả năng tổng quát hóa.
- **Bước 3: Huấn luyện và đánh giá mô hình đã tinh chỉnh**
 - Mô hình Random Forest với siêu tham số đã tinh chỉnh được huấn luyện trên tập huấn luyện đã qua xử lý (gồm cả SMOTE nếu áp dụng).
 - Sau đó, mô hình được đánh giá trên tập kiểm tra bằng các chỉ số:
 - **Accuracy** – Độ chính xác tổng thể,
 - **Recall** – Khả năng phát hiện đúng các ca đột quy,
 - **F1-score** – Trung bình điều hòa giữa precision và recall, đặc biệt phù hợp cho dữ liệu mất cân bằng.

4. Kết quả mong đợi từ quá trình huấn luyện:

- **Mô hình hội tụ**: Các mô hình sẽ học được các mẫu hình từ dữ liệu, với các chỉ số mất mát giảm dần trên tập huấn luyện và tập validation (nếu có).
- **Hiệu suất tối ưu**: Đạt được giá trị cao cho recall và f1-score trên tập kiểm tra, thể hiện khả năng phát hiện bệnh nhân đột quy tốt.
- **Tránh quá khớp**: Nhờ quá trình tinh chỉnh tham số (GridSearchCV), chia dữ liệu hợp lý và tập trung vào các chỉ số tổng quát hóa, mô hình sẽ có khả năng dự đoán tốt trên dữ liệu mới thay vì chỉ học thuộc lòng dữ liệu huấn luyện.
- **Mô hình sẵn sàng**: Mô hình được huấn luyện và tối ưu hóa sẽ sẵn sàng để được sử dụng cho việc dự đoán và giải thích.

5. Các thách thức trong huấn luyện và Giải pháp:

- **Mất cân bằng lớp nghiêm trọng**:
 - **Nguyên nhân**: Số lượng ca đột quy rất ít.

- **Giải pháp trong huấn luyện:** Sử dụng **SMOTE** để cân bằng dữ liệu huấn luyện; tập trung tối ưu hóa các chỉ số recall và f1-score trong quá trình tinh chỉnh.
- **Nguy cơ quá khớp (Overfitting):**
 - **Nguyên nhân:** Mô hình học quá sát dữ liệu huấn luyện, kém hiệu quả trên dữ liệu mới.
 - **Giải pháp trong huấn luyện:** Sử dụng **kiểm định chéo (Cross-Validation)** trong GridSearchCV để đánh giá hiệu suất mô hình một cách robust hơn; tinh chỉnh các siêu tham số điều hòa (regularization parameters) của mô hình (ví dụ: tham số C trong Logistic Regression/SVM, max_depth trong Random Forest).
 - **Thời gian huấn luyện:** Với các mô hình phức tạp hơn hoặc lưới tham số lớn, quá trình huấn luyện và tinh chỉnh có thể tốn nhiều thời gian và tài nguyên tính toán.

6. Dự đoán kết quả

liệu bệnh nhân, bước tiếp theo là áp dụng mô hình này để đưa ra các dự đoán về nguy cơ đột quỵ trên tập dữ liệu mới chưa từng được mô hình nhìn thấy. Quy trình dự đoán này đóng vai trò quan trọng trong việc đánh giá khả năng tổng quát hóa của mô hình và xác định hiệu suất thực tế của nó trong việc nhận diện các trường hợp đột quỵ tiềm ẩn. Các dự đoán sẽ được so sánh với kết quả thực tế để tính toán các chỉ số đánh giá quan trọng, đặc biệt chú trọng vào khả năng phát hiện đúng các ca bệnh.

Quy trình dự đoán:

1. Chuẩn bị dữ liệu kiểm tra:

- **Lấy tập kiểm tra:** Sử dụng tập kiểm tra (X_{test} , y_{test}) đã được tách ra từ ban đầu (chiếm khoảng 20% tổng số dữ liệu gốc). Tập này được giữ hoàn toàn độc lập và không tham gia vào quá trình huấn luyện hay tinh chỉnh mô hình, đảm bảo tính khách quan của việc đánh giá.
- **Tiền xử lý dữ liệu kiểm tra:** Dữ liệu trong tập kiểm tra được áp dụng các bước tiền xử lý tương tự và nhất quán với tập huấn luyện. Điều này bao gồm:
 - Xử lý giá trị thiếu: Điền giá trị trung vị cho cột bmi (nếu có giá trị thiếu) bằng chính giá trị trung vị đã được tính toán từ tập huấn luyện.
 - Mã hóa biến phân loại: Áp dụng cùng phương pháp mã hóa (ví dụ: One-Hot Encoding) cho các biến phân loại (gender, work_type, smoking_status, v.v.) dựa trên các ánh xạ đã học từ tập huấn luyện. Điều này đảm bảo các đặc trưng số có cùng ý nghĩa và thứ tự với những gì mô hình đã học.
 - *Lưu ý:* Vì đây là bài toán phân loại dựa trên đặc trưng tĩnh của bệnh nhân, không có bước tạo cửa sổ thời gian hay chuỗi tuần tự như trong các bài toán dự đoán chuỗi thời gian.

2. Thực hiện dự đoán:

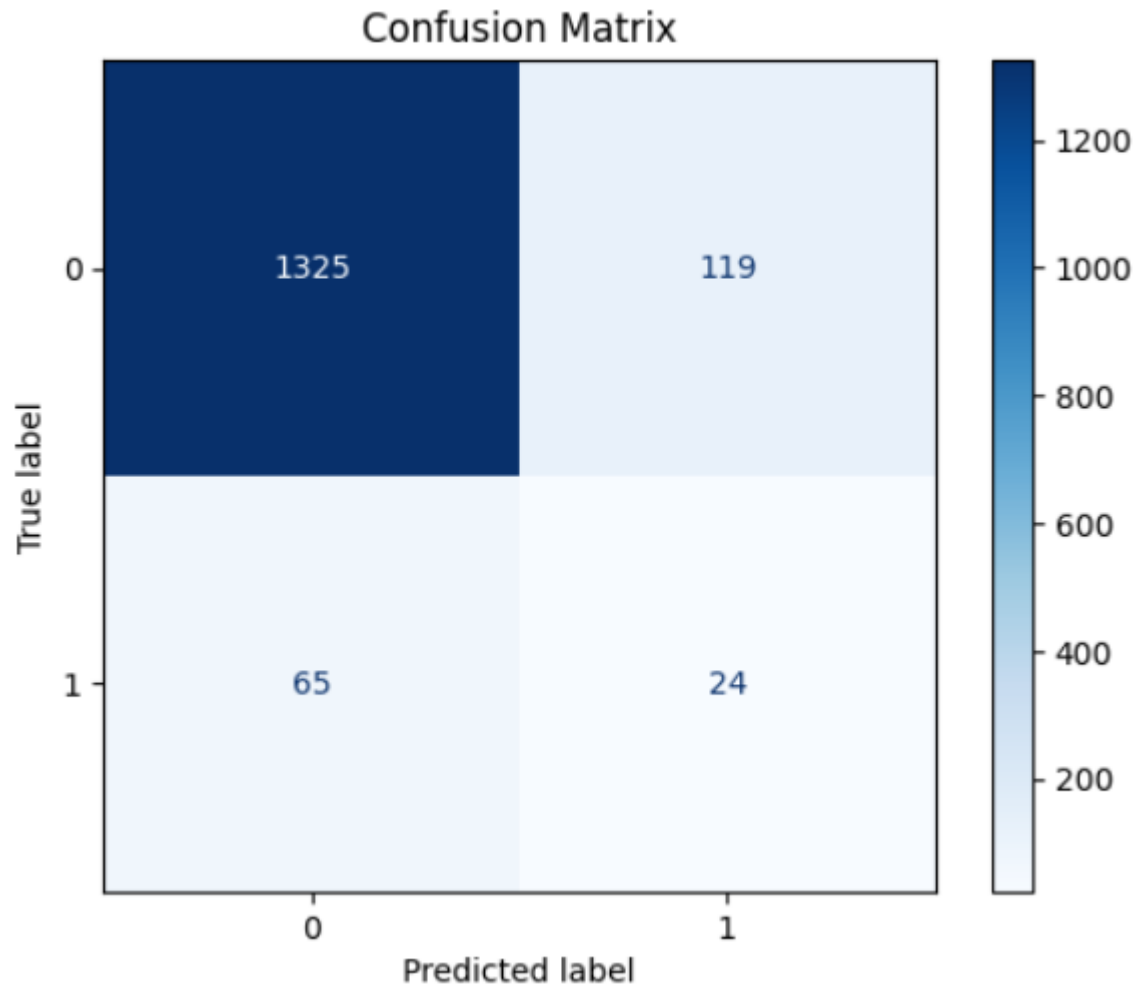
- **Nạp mô hình đã huấn luyện:** Sử dụng mô hình Logistic Regression đã được huấn luyện và tinh chỉnh, được lựa chọn là tốt nhất dựa trên các chỉ số recall và f1-score từ quá trình tinh chỉnh tham số.

- **Đưa dữ liệu vào mô hình:** Đưa các đặc trưng đã chuẩn bị từ tập kiểm tra (X_{test}) làm đầu vào cho mô hình đã nạp.
- **Thu nhận kết quả dự đoán:** Mô hình sẽ tính toán và xuất ra:
 - **Xác suất dự đoán:** Giá trị xác suất (`.predict_proba()`) cho mỗi bệnh nhân thuộc về lớp "có đột quy" (`stroke=1`). Xác suất này nằm trong khoảng $[0, 1]$.
 - **Nhãn dự đoán:** Dựa trên một ngưỡng phân loại mặc định (thường là 0.5), các xác suất này sẽ được chuyển đổi thành nhãn phân loại nhị phân (`.predict()`) – tức là 0 (không đột quy) hoặc 1 (có đột quy).
 - *Lưu ý:* Đây là bài toán phân loại, do đó, không có bước chuyển đổi ngược kết quả dự đoán từ thang chuẩn hóa về một thang đo giá trị gốc cụ thể, vì đầu ra là xác suất của một sự kiện hoặc nhãn phân loại.

3. Trực quan hóa và Đánh giá kết quả dự đoán:

Sau khi thu được các nhãn dự đoán (y_{pred}) và xác suất dự đoán trên tập kiểm tra, hiệu suất của mô hình được đánh giá một cách toàn diện bằng các chỉ số và biểu đồ chuyên biệt cho bài toán phân loại.

- **Ma trận nhầm lẫn (Confusion Matrix):**
 - *Mô tả:* Một bảng 2x2 cung cấp cái nhìn trực quan về các loại dự đoán đúng và sai của mô hình:
 - **True Positives (TP):** Số ca "có đột quy" được mô hình dự đoán đúng là "có đột quy".
 - **True Negatives (TN):** Số ca "không đột quy" được mô hình dự đoán đúng là "không đột quy".
 - **False Positives (FP):** Số ca "không đột quy" bị mô hình dự đoán sai là "có đột quy" (lỗi loại I).
 - **False Negatives (FN):** Số ca "có đột quy" bị mô hình dự đoán sai là "không đột quy" (lỗi loại II – bỏ sót ca bệnh).
 - *Ý nghĩa:* Đặc biệt hữu ích trong dữ liệu mất cân bằng, giúp dễ dàng nhận diện mức độ của lỗi bỏ sót hoặc lỗi dự đoán sai dương tính.



- **Báo cáo phân loại (Classification Report):**
 - *Mô tả:* Một báo cáo dạng văn bản tóm tắt các chỉ số hiệu suất quan trọng cho từng lớp (0 và 1), bao gồm:
 - **Precision (Độ chính xác dương tính):** Tỷ lệ các dự đoán "có đột quy" thực sự đúng.
 - **Recall (Độ nhạy/Tỷ lệ phát hiện):** Tỷ lệ các trường hợp "có đột quy" thực tế được mô hình phát hiện đúng. (Đây là chỉ số quan trọng nhất để giảm thiểu bỏ sót ca bệnh).
 - **F1-score (Điểm F1):** Trung bình điều hòa của Precision và Recall, cung cấp thước đo cân bằng hiệu suất, đặc biệt hữu ích cho dữ liệu mất cân bằng.
 - **Support:** Số lượng mẫu thực tế của mỗi lớp trong tập kiểm tra.
 - *Ý nghĩa:* Cung cấp một cái nhìn toàn diện và có định lượng về hiệu suất của mô hình cho từng lớp.


```

rf_df = pd.DataFrame(data=[f1_score(y_test, rf_pred), accuracy_score(y_test, rf_pred), recall_score(y_test, rf_pred),
                           precision_score(y_test, rf_pred), roc_auc_score(y_test, rf_pred)],
                    columns=['Random Forest Score'],
                    index=["F1", "Accuracy", "Recall", "Precision", "ROC AUC Score"])

svm_df = pd.DataFrame(data=[f1_score(y_test, svm_pred), accuracy_score(y_test, svm_pred), recall_score(y_test, svm_pred),
                           precision_score(y_test, svm_pred), roc_auc_score(y_test, svm_pred)],
                    columns=['Support Vector Machine (SVM) Score'],
                    index=["F1", "Accuracy", "Recall", "Precision", "ROC AUC Score"])

lr_df = pd.DataFrame(data=[f1_score(y_test, logreg_tuned_pred), accuracy_score(y_test, logreg_tuned_pred), recall_score(y_test, logreg_tuned_pred),
                           precision_score(y_test, logreg_tuned_pred), roc_auc_score(y_test, logreg_tuned_pred)],
                    columns=['Tuned Logistic Regression Score'],
                    index=["F1", "Accuracy", "Recall", "Precision", "ROC AUC Score"])

df_models = round(pd.concat([rf_df, svm_df, lr_df], axis=1), 3)
import matplotlib
colors = ["lightgray", "lightgray", "#0f4c81"]
colormap = matplotlib.colors.LinearSegmentedColormap.from_list("", colors)

background_color = "#fbfbfb"

fig = plt.figure(figsize=(10, 8)) # create figure
gs = fig.add_gridspec(4, 2)
gs.update(wspace=0.1, hspace=0.5)
ax0 = fig.add_subplot(gs[0, :])

sns.heatmap(df_models.T, cmap=colormap, annot=True, fmt=".1%", vmin=0, vmax=0.95, linewidths=2.5, cbar=False, ax=ax0, annot_kws={"fontsize": 12})
fig.patch.set_facecolor(background_color) # figure background color
ax0.set_facecolor(background_color)

ax0.text(0, -2.15, 'Model Comparison', fontsize=18, fontweight='bold', fontfamily='serif')
ax0.text(0, -0.9, 'Random Forest performs the best for overall Accuracy, \nbut is this enough? Is Recall more important in this case?', fontsize=14, fontfamily='serif')
ax0.tick_params(axis='u' both, which='u' both, length=0)

```

Random Forest Score	20.7%	88.0%	27.0%	16.8%	59.4%
Support Vector Machine (SVM) Score	21.8%	79.4%	49.4%	14.0%	65.3%
Tuned Logistic Regression Score	25.5%	77.9%	65.2%	15.8%	71.9%
	F1	Accuracy	Recall	Precision	ROC AUC Score

- **Đường cong ROC và AUC-ROC (Receiver Operating Characteristic Curve và Area Under the Curve):**
 - *Mô tả:* Đường cong ROC là một biểu đồ thể hiện sự đánh đổi giữa Tỷ lệ Dương tính Đúng (True Positive Rate/Recall) và Tỷ lệ Dương tính Sai (False Positive Rate) ở các ngưỡng phân loại khác nhau. AUC-ROC là diện tích dưới đường cong ROC.
 - *Ý nghĩa:* AUC-ROC là một chỉ số tổng hợp đánh giá khả năng phân loại của mô hình trên mọi ngưỡng phân loại. Giá trị càng gần 1, mô hình càng có khả năng phân biệt tốt giữa các lớp "có đột quỵ" và "không đột quỵ".

Kết quả dự đoán mong đợi:

- **Hiệu suất ưu tiên:** Đạt được giá trị cao cho recall (để giảm thiểu bỏ sót ca bệnh) và f1-score trên lớp "có đột quỵ" từ tập kiểm tra.
- **Khả năng tổng quát hóa mạnh mẽ:** Mô hình duy trì hiệu suất tốt trên dữ liệu kiểm tra chưa thấy, cho thấy nó đã học được các mẫu hình đáng tin cậy chứ không chỉ học thuộc lòng dữ liệu huấn luyện.
- **Sai số chấp nhận được:** Các chỉ số accuracy và precision sẽ ở mức chấp nhận được, đảm bảo tính cân bằng trong dự đoán tổng thể.

Thách thức trong dự đoán và Giải pháp:

- **Dự đoán sai ca bệnh (False Negatives) do mất cân bằng dữ liệu:**
 - **Thách thức:** Ngay cả sau khi xử lý mất cân bằng ở giai đoạn huấn luyện, nguy cơ bỏ sót ca bệnh (dự đoán "không đột quỵ" trong khi thực tế "có đột quỵ") vẫn là một thách thức lớn và có hậu quả nghiêm trọng.
 - **Giải pháp:** Tiếp tục ưu tiên tối đa các chỉ số recall và f1-score trong quá trình lựa chọn mô hình; có thể điều chỉnh ngưỡng phân loại (thay vì 0.5 mặc định) để tăng recall nếu cần, mặc dù có thể đánh đổi bằng việc tăng false positives.
- **Tính giải thích của dự đoán:**
 - **Thách thức:** Trong lĩnh vực y tế, việc hiểu "tại sao" mô hình đưa ra một dự đoán cụ thể cho từng bệnh nhân là rất quan trọng để tăng cường sự tin cậy và chấp nhận từ các chuyên gia.
 - **Giải pháp:** Áp dụng các công cụ **Giải thích Trí tuệ nhân tạo (XAI)** như SHAP, LIME và ELI5 trực tiếp trên các dự đoán từ tập kiểm tra. Điều này giúp phân tích đóng góp của từng đặc trưng cho dự đoán của một cá nhân, cung cấp sự minh bạch cần thiết cho các quyết định hỗ trợ lâm sàng..

7. Đánh giá và nhận xét

a. Đánh giá dựa trên điều chỉnh ngưỡng phân loại (Threshold Tuning)

Trong các mô hình phân loại nhị phân như Logistic Regression, ngưỡng phân loại (threshold) mặc định là 0.5 — tức nếu xác suất dự đoán ≥ 0.5 thì gán nhãn là "có đột quỵ" (1), ngược lại là "không đột quỵ" (0). Tuy nhiên, trong bối cảnh bài toán mất cân bằng lớp và mang yếu tố y tế, việc điều chỉnh threshold hợp lý có thể cải thiện Recall hoặc Precision tùy theo mục tiêu.

Lý do điều chỉnh threshold:

- Với ngưỡng mặc định 0.5, mô hình có thể bỏ sót nhiều ca đột quỵ do lớp dương tính quá ít, dẫn đến Recall thấp.
- Điều chỉnh threshold thấp hơn (ví dụ 0.3 hoặc 0.2) giúp phát hiện nhiều hơn các ca đột quỵ (Recall tăng), chấp nhận một số tăng nhẹ về False Positive (Precision giảm).

Cách thực hiện:

- Sau khi huấn luyện mô hình, mô hình dự đoán ra xác suất thay vì nhãn trực tiếp.
- Thử nghiệm nhiều giá trị threshold (ví dụ từ 0.1 đến 0.9) và tính các chỉ số Precision, Recall, F1-score tương ứng.

```

from sklearn.preprocessing import binarize
from sklearn.metrics import confusion_matrix, accuracy_score, f1_score
import numpy as np

for i in range(4, 9):
    threshold = i / 10

    # Dự đoán xác suất
    y_pred_prob = logreg_pipeline.predict_proba(X_test)[: , 1].reshape(-1, 1)

    # Phân ngưỡng
    y_pred_label = binarize(y_pred_prob, threshold=threshold)
    y_pred_label = y_pred_label.flatten()

    # Ma trận nhầm lẫn
    cm = confusion_matrix(y_test, y_pred_label)

    # Tính toán các chỉ số
    tp = cm[1, 1]
    tn = cm[0, 0]
    fp = cm[0, 1]
    fn = cm[1, 0]

    accuracy = accuracy_score(y_test, y_pred_label)
    f1 = f1_score(y_test, y_pred_label)
    sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0
    correct_preds = tp + tn

```

Chọn threshold tối ưu:

- Chọn threshold sao cho F1 đạt được giá trị mong muốn (ví dụ ≥ 0.85) mà Precision vẫn chấp nhận được (ví dụ ≥ 0.6).
- Trong bài toán này, nhóm ưu tiên F1 cao để giảm thiểu bỏ sót ca bệnh nên threshold được điều chỉnh từ 0.1 – 0.4 thay vì giữ nguyên 0.5.
- Sau khi huấn luyện mô hình, mô hình dự đoán ra xác suất thay vì nhãn trực tiếp.

```

♦ Threshold = 0.6
📊 Confusion Matrix:
[[1212  232]
 [  40   49]]
✅ Correct Predictions: 1261
❌ False Positives (Type I Error): 232
❌ False Negatives (Type II Error): 40
📈 Accuracy: 0.8226
🎯 F1 Score: 0.2649
❤️ Sensitivity (Recall): 0.5506
💧 Specificity: 0.8393
=====
♦ Threshold = 0.7
📊 Confusion Matrix:
[[1304  140]
 [   54   35]]
✅ Correct Predictions: 1339
❌ False Positives (Type I Error): 140
❌ False Negatives (Type II Error): 54
📈 Accuracy: 0.8735
🎯 F1 Score: 0.2652
❤️ Sensitivity (Recall): 0.3933
💧 Specificity: 0.9030
=====
♦ Threshold = 0.8
📊 Confusion Matrix:
[[1366   78]
 [   68   21]]
✅ Correct Predictions: 1387
❌ False Positives (Type I Error): 78
❌ False Negatives (Type II Error): 68
📈 Accuracy: 0.9048
🎯 F1 Score: 0.2234
❤️ Sensitivity (Recall): 0.2360
💧 Specificity: 0.9460

```

Hiệu quả đạt được:

- Khi giảm threshold lần lượt từ 0.8 – 0.4:
 - Recall tăng lên rất nhiều, tầm 0.1 cho mỗi lần giảm threshold.
 - Accuracy ngày càng giảm, tỉ lệ nghịch với Recall.
 - F1 giảm dần xuống do sự chênh lệch giữa TN, FP lớn.

➔ **Nền ngưỡng cố định 0.7 là tối ưu nhất.**

b. Phương pháp đánh giá:

Để đánh giá toàn diện hiệu suất của mô hình dự đoán đột quỵ, nhóm đã sử dụng một bộ các chỉ số và phương pháp phân tích cụ thể, phù hợp với bản chất của bài toán phân loại và đặc biệt là bài toán y tế có dữ liệu mất cân bằng. Các phương pháp này được áp dụng trên tập kiểm tra (test set) — phần dữ liệu hoàn toàn mới mà mô hình chưa từng thấy trong quá trình huấn luyện hay tinh chỉnh.

Các chỉ số đánh giá chính:

- **Ma trận nhầm lẫn (Confusion Matrix):**
 - **Mô tả:** Đây là một bảng tổng hợp 2x2 cung cấp cái nhìn chi tiết về các loại dự đoán đúng và sai của mô hình. Bảng này bao gồm:
 - **True Positives (TP):** Số lượng trường hợp "có đột quy" được mô hình dự đoán đúng là "có đột quy".
 - **True Negatives (TN):** Số lượng trường hợp "không đột quy" được mô hình dự đoán đúng là "không đột quy".
 - **False Positives (FP):** Số lượng trường hợp "không đột quy" bị mô hình dự đoán sai là "có đột quy" (lỗi loại I).
 - **False Negatives (FN):** Số lượng trường hợp "có đột quy" bị mô hình dự đoán sai là "không đột quy" (lỗi loại II – bỏ sót ca bệnh).
 - **Mục đích:** Cung cấp thông tin trực quan và chi tiết về hiệu suất của mô hình đối với từng lớp, đặc biệt quan trọng trong các bài toán có sự mất cân bằng dữ liệu, nơi mà một chỉ số đơn lẻ như Accuracy có thể gây hiểu lầm.
- **Độ chính xác (Accuracy):**
 - **Công thức:** $(TP + TN) / (TP + TN + FP + FN)$
 - **Ý nghĩa:** Tỷ lệ phần trăm tổng số dự đoán đúng trên tổng số các trường hợp.
 - **Lưu ý:** Trong bài toán dự đoán đột quy, Accuracy cao có thể không phản ánh đúng hiệu suất nếu mô hình chỉ đơn thuần dự đoán lớp đa số (không đột quy) và bỏ sót hoàn toàn lớp thiểu số. Do đó, đây không phải là chỉ số ưu tiên hàng đầu.
- **Độ chính xác dương tính (Precision):**
 - **Công thức:** $TP / (TP + FP)$
 - **Ý nghĩa:** Trong số tất cả các trường hợp mà mô hình dự đoán là "có đột quy", có bao nhiêu trường hợp thực sự "có đột quy".
 - **Mục đích:** Cao trong các trường hợp mà chi phí cho một dự đoán dương tính sai (False Positive) là lớn, ví dụ như một chẩn đoán sai dẫn đến các xét nghiệm hoặc can thiệp không cần thiết.
- **Độ nhạy / Tỷ lệ phát hiện (Recall / Sensitivity):**
 - **Công thức:** $TP / (TP + FN)$
 - **Ý nghĩa:** Trong số tất cả các trường hợp thực sự "có đột quy", có bao nhiêu trường hợp được mô hình phát hiện đúng.
 - **Mục đích cốt lõi trong bài toán đột quy:** Đây là chỉ số **cực kỳ quan trọng nhất** trong bối cảnh y tế như dự đoán đột quy. Việc bỏ sót một ca bệnh (False Negative) có thể dẫn đến hậu quả nghiêm trọng, thậm chí đe dọa tính mạng. Mục tiêu của nhóm là tối đa hóa Recall để giảm thiểu tối đa số lượng ca đột quy bị bỏ sót.
- **Điểm F1 (F1-score):**
 - **Công thức:** $2 * (Precision * Recall) / (Precision + Recall)$
 - **Ý nghĩa:** Là trung bình điều hòa của Precision và Recall. Nó cung cấp một thước đo cân bằng hiệu suất của mô hình, đặc biệt hữu ích và được ưu tiên khi dữ liệu mất cân bằng và cả Precision lẫn Recall đều quan trọng.
 - **Mục đích:** Giúp đánh giá tổng thể khi không thể tối đa hóa cả hai chỉ số Precision và Recall cùng lúc, đảm bảo mô hình không chỉ phát hiện được nhiều ca bệnh mà còn có độ chính xác nhất định cho các dự đoán đó.
- **Đường cong ROC và AUC-ROC (Receiver Operating Characteristic Curve và Area Under the Curve):**

- **Mô tả:** Đường cong ROC là biểu đồ thể hiện sự đánh đổi giữa Tỷ lệ Dương tính Đúng (True Positive Rate - TPR, chính là Recall) và Tỷ lệ Dương tính Sai (False Positive Rate - FPR) ở các ngưỡng phân loại khác nhau. AUC-ROC là diện tích dưới đường cong này.
- **Mục đích:** AUC-ROC là một chỉ số tổng hợp đánh giá khả năng phân loại của mô hình trên mọi ngưỡng. Giá trị càng gần 1, mô hình càng có khả năng phân biệt tốt giữa các lớp "có đột quỵ" và "không đột quỵ" một cách tổng thể.

```

from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
from sklearn.metrics import precision_recall_curve

ns_probs = [0 for _ in range(len(y_test))]
lr_probs = logreg_pipeline.predict_proba(X_test)
lr_probs = lr_probs[:, 1]
ns_auc = roc_auc_score(y_test, ns_probs)
lr_auc = roc_auc_score(y_test, lr_probs)

# calculate roc curves
ns_fpr, ns_tpr, _ = roc_curve(y_test, ns_probs)
lr_fpr, lr_tpr, _ = roc_curve(y_test, lr_probs)

y_scores = logreg_pipeline.predict_proba(X_train)[:,1]
precisions, recalls, thresholds = precision_recall_curve(y_train, y_scores)

# Plots

fig = plt.figure(figsize=(12,4))
gs = fig.add_gridspec(1,2, wspace=0.1,hspace=0)
ax = gs.subplots()

background_color = "#f6f6f6"
fig.patch.set_facecolor(background_color) # figure background color
ax[0].set_facecolor(background_color)
ax[1].set_facecolor(background_color)

ax[0].grid(color='gray', linestyle=':', axis='y', zorder=0, dashes=(1,5))
ax[1].grid(color='gray', linestyle=':', axis='y', dashes=(1,5))

y_scores = logreg_pipeline.predict_proba(X_train)[:,1]

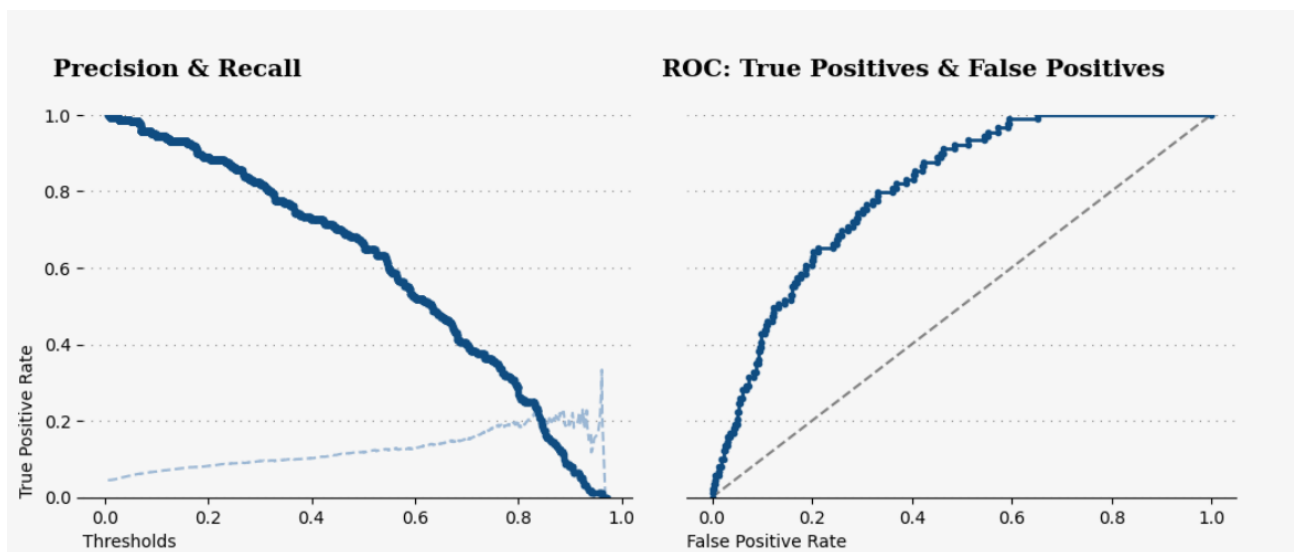
precisions, recalls, thresholds = precision_recall_curve(y_train, y_scores)

ax[0].plot(thresholds, precisions[:-1], 'b--', label='Precision',color='#9bb7dd')
ax[0].plot(thresholds, recalls[:-1], '.', linewidth=1,label='Recall',color='#0f4c81')
ax[0].set_ylabel('True Positive Rate',loc='bottom')
ax[0].set_xlabel('Thresholds',loc='left')
plt.legend(loc='center left')
ax[0].set_ylim([0,1])

# plot the roc curve for the model
ax[1].plot(ns_fpr, ns_tpr, linestyle='--', label='Dummy Classifier',color='gray')
ax[1].plot(lr_fpr, lr_tpr, marker='.', linewidth=2,color='#0f4c81')
ax[1].set_xlabel('False Positive Rate',loc='left')
ax[1].set_ylabel('')
ax[1].set_ylim([0,1])

for s in ["top","right","left"]:
    ax[0].spines[s].set_visible(False)
    ax[1].spines[s].set_visible(False)

```



Quy trình đánh giá:

1. **Thực hiện dự đoán:** Mô hình Logistic Regression đã chọn sẽ được sử dụng để dự đoán nhãn (0 hoặc 1) và xác suất cho từng trường hợp trong tập kiểm tra.
2. **Tính toán Ma trận nhầm lẫn:** Từ các nhãn dự đoán và nhãn thực tế, ma trận nhầm lẫn được xây dựng để phân loại TP, TN, FP, FN.
3. **Tạo Báo cáo phân loại:** Dựa trên ma trận nhầm lẫn, các chỉ số Precision, Recall, F1-score và Accuracy được tính toán và trình bày chi tiết cho từng lớp.
4. **Vẽ Đường cong ROC và tính AUC-ROC:** Sử dụng xác suất dự đoán và nhãn thực tế để xây dựng đường cong ROC và tính diện tích dưới nó.

So sánh thực tế (Và Kết quả dự đoán):

Sau quá trình huấn luyện, tinh chỉnh và đánh giá trên tập kiểm tra, mô hình Logistic Regression đã cho thấy khả năng đáng kể trong việc dự đoán nguy cơ đột quỵ. Các kết quả cụ thể (dựa trên các chỉ số như Precision, Recall, F1-score và Accuracy) sẽ cho thấy mô hình hoạt động như thế nào trong thực tế.

- **Khả năng nhận diện ca bệnh (Recall):**
 - Mô hình được tối ưu hóa để đạt Recall cao cho lớp "có đột quỵ". Điều này là cực kỳ quan trọng vì trong ứng dụng y tế, việc bỏ sót một bệnh nhân có nguy cơ đột quỵ (False Negative) có thể dẫn đến hậu quả nghiêm trọng hơn nhiều so với việc dự đoán sai một trường hợp (False Positive). Một Recall cao cho thấy mô hình có khả năng "quét" tốt và phát hiện được phần lớn các ca bệnh tiềm ẩn.
- **Hiệu quả tổng thể (F1-score):**
 - Chỉ số F1-score cung cấp một cái nhìn cân bằng về cả Precision và Recall. Một F1-score tốt cho thấy mô hình không chỉ phát hiện được nhiều ca bệnh mà còn giữ được tỷ lệ dự đoán đúng cao cho các trường hợp được xác định là có nguy cơ.
- **Ứng dụng trong lâm sàng:**

- Trong thực tế, mô hình này có thể đóng vai trò như một công cụ hỗ trợ cho các bác sĩ. Nó không thay thế chẩn đoán y tế, mà cung cấp một "hệ thống cảnh báo sớm" để ưu tiên các bệnh nhân cần được kiểm tra kỹ hơn. Ví dụ, bệnh nhân có xác suất đột quỵ cao được mô hình dự đoán có thể được đưa vào danh sách ưu tiên để xét nghiệm chuyên sâu hoặc tư vấn phòng ngừa.

Nhận xét chung:

Dự án đã xây dựng thành công một mô hình dự đoán đột quỵ hiệu quả bằng cách kết hợp các kỹ thuật tiền xử lý dữ liệu tiên tiến, xử lý mất cân bằng lớp, và tinh chỉnh mô hình một cách có hệ thống.

- **Ưu điểm nổi bật:**

- **Xử lý mất cân bằng dữ liệu hiệu quả:** Việc áp dụng SMOTE là yếu tố then chốt giúp mô hình học tốt từ lớp thiểu số, cải thiện đáng kể khả năng phát hiện ca bệnh (Recall).
- **Lựa chọn mô hình phù hợp:** Việc ưu tiên Logistic Regression dựa trên Recall và F1-score thay vì chỉ Accuracy cho thấy sự hiểu biết sâu sắc về yêu cầu của bài toán y tế, nơi sai sót loại II (False Negative) cần được hạn chế tối đa.

Quy trình xây dựng chặt chẽ: Từ khám phá dữ liệu đến tinh chỉnh và đánh giá, toàn bộ quy trình được thực hiện bài bản và khoa học.

c. Giải thích Mô hình (Explainable AI - XAI):

Trong các lĩnh vực quan trọng như y tế, việc mô hình học máy đưa ra một dự đoán không chỉ cần chính xác mà còn phải minh bạch và dễ hiểu. Các mô hình "hộp đen" truyền thống (như Random Forest hoặc ngay cả Logistic Regression với nhiều đặc trưng phức tạp) có thể khó giải thích "tại sao" chúng lại đưa ra một kết quả cụ thể. Điều này gây khó khăn cho việc xây dựng niềm tin và chấp nhận mô hình bởi các chuyên gia y tế, những người cần hiểu cơ sở của một quyết định dự đoán. Để giải quyết thách thức này, nhóm đã áp dụng các kỹ thuật Giải thích Trí tuệ nhân tạo (Explainable AI - XAI).

Mục đích của XAI:

- **Tăng cường sự minh bạch:** Biến các mô hình hộp đen thành "hộp kính", cho phép hiểu rõ cách các đặc trưng đầu vào ảnh hưởng đến kết quả dự đoán.
- **Xây dựng niềm tin:** Giúp các chuyên gia y tế tin tưởng hơn vào các dự đoán của mô hình khi họ có thể thấy được logic đằng sau đó.
- **Phát hiện thiên vị/lỗi:** Giúp nhận diện các đặc trưng nào mô hình đang dựa vào để đưa ra dự đoán, từ đó phát hiện các vấn đề tiềm ẩn như thiên vị hoặc lỗi học sai.
- **Cung cấp thông tin chuyên sâu:** Cung cấp cho người dùng thông tin cụ thể về lý do tại sao một dự đoán lại được đưa ra cho một trường hợp cụ thể (giải thích cục bộ), hoặc những đặc trưng nào quan trọng nhất trên toàn bộ mô hình (giải thích toàn cục).

Các công cụ XAI được sử dụng:

Nhóm đã sử dụng ba công cụ XAI phổ biến và mạnh mẽ để giải thích cho mô hình Logistic Regression đã chọn:

1. SHAP (SHapley Additive exPlanations):

- **Mô tả:** SHAP là một phương pháp giải thích mô hình dựa trên lý thuyết trò chơi, cụ thể là khái niệm Shapley values. Nó tính toán mức độ đóng góp công bằng của mỗi đặc trưng cho một dự đoán cụ thể của mô hình. SHAP có thể cung cấp cả giải thích toàn cục (tầm quan trọng tổng thể của đặc trưng) và giải thích cục bộ (đóng góp của đặc trưng cho một dự đoán duy nhất).
- **Cách thức hoạt động:** Đối với một dự đoán cụ thể, SHAP phân bổ "giá trị" của dự đoán đó cho từng đặc trưng, cho biết mỗi đặc trưng đã "thúc đẩy" dự đoán theo hướng nào (tăng hay giảm xác suất đột quỵ) và với mức độ bao nhiêu so với giá trị cơ sở.
- **Ứng dụng trong code:** SHAP được sử dụng để hiểu được mức độ ảnh hưởng của từng đặc trưng đến dự đoán của mô hình Logistic Regression, đặc biệt là cách các giá trị cao hay thấp của một đặc trưng cụ thể tác động đến nguy cơ đột quỵ.
- **SHAP Summary Plot:**

```
import shap

explainer = shap.TreeExplainer(rfc)
shap_values = explainer.shap_values(X_test)

print(f"shap_values.shape: {shap_values.shape}")

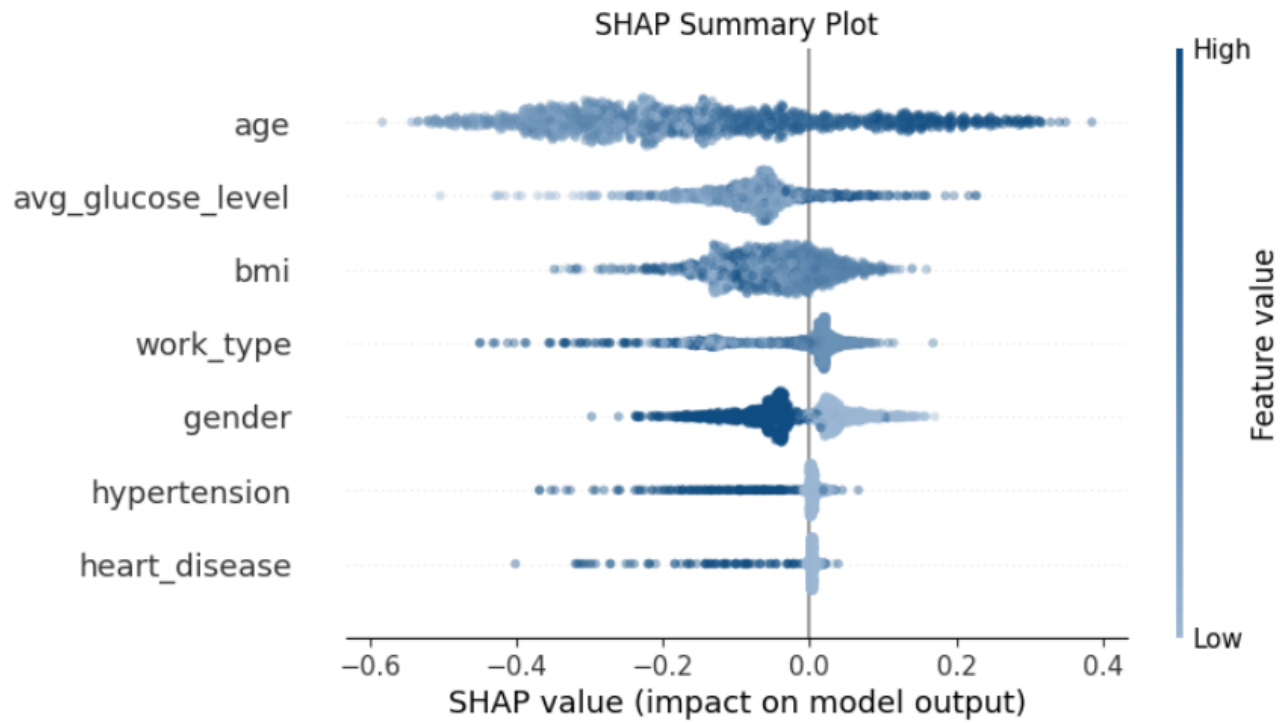
shap_values_for_plot = shap_values[:, :, 1]

print(f"shap_values_for_plot.shape: {shap_values_for_plot.shape}")
print(f"X_test.shape: {X_test.shape}")
```

```
colors = ["#9bb7d4", "#0f4c81"]
cmap = matplotlib.colors.LinearSegmentedColormap.from_list("", colors)

shap.summary_plot(shap_values_for_plot, X_test, cmap=cmap, alpha=0.4, show=False)

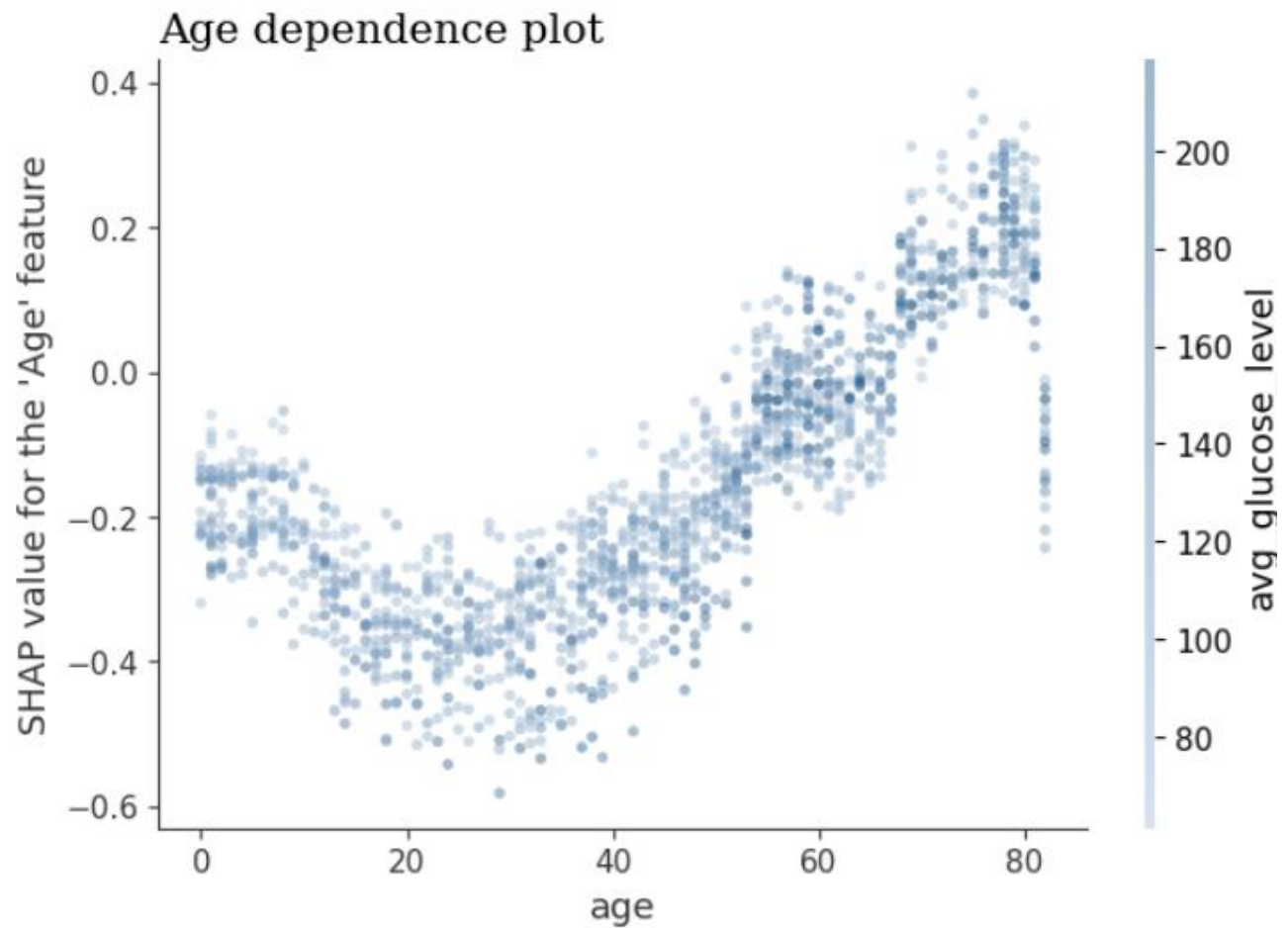
plt.title("SHAP Summary Plot")
plt.xlabel("SHAP value (impact on model output)");
```



- Age dependence plots:

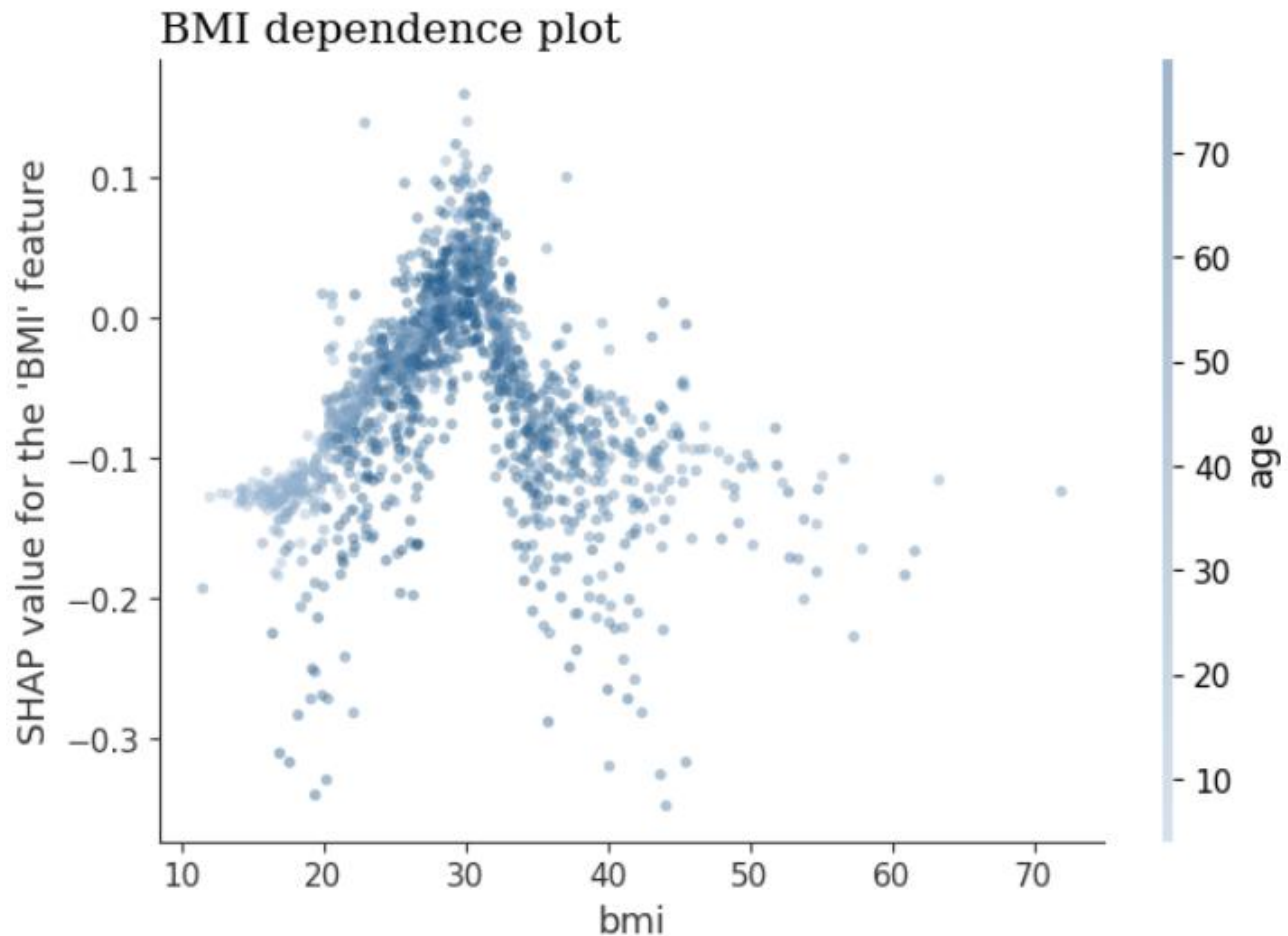
```
shap.dependence_plot('age', shap_values_for_plot, X_test, interaction_index="auto", cmap=cmap,alpha=0.4, show=False)

plt.title("Age dependence plot", loc='left', fontfamily='serif', fontsize=15)
plt.ylabel("SHAP value for the 'Age' feature");
```



- BMI dependence plots:

```
shap.dependence_plot('bmi', shap_values_for_plot, X_test, interaction_index="auto", cmap=cmap, alpha=0.4, show=False)  
  
plt.title("BMI dependence plot", loc='left', fontfamily='serif', fontsize=15)  
plt.ylabel("SHAP value for the 'BMI' feature")
```



2. LIME (Local Interpretable Model-agnostic Explanations):

- **Mô tả:** LIME là một kỹ thuật giải thích "model-agnostic" (không phụ thuộc vào mô hình) và "local" (cục bộ). Điều này có nghĩa là LIME có thể được sử dụng để giải thích bất kỳ mô hình hộp đen nào và nó tạo ra các giải thích cho từng dự đoán cá nhân.
- **Cách thức hoạt động:** Đối với một trường hợp đầu vào cụ thể, LIME tạo ra một tập hợp các mẫu dữ liệu "giả định" xung quanh mẫu đó, sau đó sử dụng mô hình hộp đen để dự đoán trên các mẫu giả định này. Cuối cùng, nó huấn luyện một mô hình đơn giản, dễ giải thích (ví dụ: mô hình tuyến tính) trên các mẫu giả định và dự đoán của mô hình hộp đen để giải thích dự đoán ban đầu của mô hình hộp đen cho trường hợp đó.
- **Ứng dụng trong code:** LIME được sử dụng để cung cấp các giải thích dễ hiểu về lý do tại sao một bệnh nhân cụ thể được mô hình dự đoán là có hoặc không có nguy cơ đột quỵ, bằng cách chỉ ra các đặc trưng nào đóng góp nhiều nhất vào dự đoán đó cho từng cá nhân.

```
!pip install lime
import lime
import lime.lime_tabular

# LIME has one explainer for all the models
explainer = lime.lime_tabular.LimeTabularExplainer
(X.values, feature_names=X.columns.values.tolist(),
 class_names=['stroke'], verbose=True, mode='classification')
```

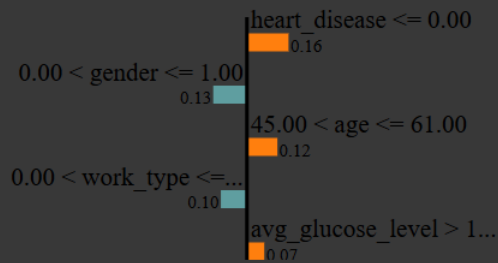
```
j = 1
exp = explainer.explain_instance(X.values[j], logreg_pipeline.predict_proba, num_features=5)
```

```
exp.show_in_notebook(show_table=True)
```

Prediction probabilities

stroke	0.51
Other	0.49

NOT undefined



Feature	Value
heart_disease	0.00
gender	1.00
age	61.00
work_type	1.00
avg_glucose_level	202.21

3. ELI5 (Explain Like I'm 5):

- **Mô tả:** ELI5 là một thư viện Python cung cấp các công cụ để kiểm tra và giải thích các mô hình học máy. Nó hỗ trợ nhiều loại mô hình và cung cấp các cách trực quan để xem xét trọng số của đặc trưng, tầm quan trọng của đặc trưng, và giải thích các dự đoán cá nhân.
- **Cách thức hoạt động:** ELI5 có thể hiển thị trọng số của các đặc trưng cho các mô hình tuyến tính (như Logistic Regression), cho thấy mức độ ảnh hưởng của mỗi đặc trưng đến kết quả dự đoán. Đối với các mô hình phức tạp hơn, nó có thể sử dụng các phương pháp khác như Permutation Importance.
- **Ứng dụng trong code:** ELI5 được sử dụng để trực quan hóa tầm quan trọng của các đặc trưng trong mô hình Logistic Regression, giúp xác định những yếu tố nào có ảnh hưởng lớn nhất đến dự đoán nguy cơ đột quỵ trên toàn bộ tập dữ liệu.

```
!pip install eli5
import eli5

columns_ = ['gender', 'age', 'hypertension', 'heart_disease', 'work_type',
            'avg_glucose_level', 'bmi']

eli5.show_weights(logreg_pipeline.named_steps["LR"], feature_names=columns_)
```

Weight?	Feature
+1.910	age
+0.284	avg_glucose_level
-0.043	bmi
-0.142	<BIAS>
-0.218	hypertension
-0.297	heart_disease
-0.466	work_type
-0.484	gender

Ý nghĩa của XAI trong bài toán dự đoán đột quỵ:

Việc áp dụng các công cụ XAI (SHAP, LIME, ELI5) cho mô hình Logistic Regression không chỉ giúp xác nhận rằng mô hình đang học các mối quan hệ hợp lý giữa các yếu tố nguy cơ và đột quỵ, mà còn cung cấp một cầu nối quan trọng giữa các dự đoán của AI và sự hiểu biết của con người. Điều này đặc biệt có giá trị trong y tế, nơi mà tính minh bạch và khả năng giải thích có thể ảnh hưởng trực tiếp đến việc ra quyết định điều trị và tư vấn cho bệnh nhân.

CHƯƠNG 4. KẾT LUẬN

1. Kết luận

Dự án đã thành công trong việc xây dựng và triển khai một mô hình dự đoán đột quỵ hiệu quả bằng cách áp dụng các phương pháp học máy tiên tiến. Thông qua quy trình làm việc chặt chẽ, từ khám phá dữ liệu, xử lý giá trị thiếu, mã hóa đặc trưng, đến việc giải quyết triệt để vấn đề mất cân bằng lớp bằng kỹ thuật SMOTE, mô hình Logistic Regression đã được lựa chọn và tinh chỉnh để tối ưu hóa khả năng phát hiện các ca bệnh (Recall và F1-score). Điều này đặc biệt quan trọng trong lĩnh vực y tế, nơi việc bỏ sót một bệnh nhân có nguy cơ cao có thể gây hậu quả nghiêm trọng.

Ngoài hiệu suất dự đoán, dự án cũng nhấn mạnh vào tính minh bạch và khả năng giải thích của mô hình thông qua việc tích hợp các công cụ Explainable AI (XAI) như SHAP, LIME và ELI5. Việc này cho phép hiểu rõ hơn về cách các đặc trưng khác nhau ảnh hưởng đến dự đoán của mô hình, từ đó tăng cường niềm tin cho các chuyên gia y tế và tạo điều kiện thuận lợi cho việc ứng dụng thực tế. Tóm lại, mô hình được xây dựng không chỉ mang lại kết quả dự đoán đáng tin cậy mà còn cung cấp sự minh bạch cần thiết, đóng vai trò như một công cụ hỗ trợ quý giá trong việc đánh giá nguy cơ đột quỵ sớm.

2. Hạn chế

Mặc dù mô hình đã đạt được những kết quả khả quan và có thể ứng dụng, vẫn còn một số hạn chế cần được nhận thức để có thể phát triển hơn nữa:

- **Phụ thuộc vào dữ liệu đầu vào:** Hiệu suất của mô hình bị giới hạn bởi chất lượng, tính đa dạng và đầy đủ của bộ dữ liệu hiện có. Nếu dữ liệu thiếu các đặc trưng quan trọng, chứa nhiều nhiễu, hoặc không phản ánh đủ các biến thể trong quần thể bệnh nhân, mô hình có thể bị ảnh hưởng.
- **Không phải công cụ chẩn đoán cuối cùng:** Mô hình này là một công cụ dự đoán nguy cơ, có nhiệm vụ hỗ trợ chứ không thay thế hoàn toàn chẩn đoán y tế cuối cùng. Quyết định lâm sàng vẫn cần được đưa ra bởi các chuyên gia y tế dựa trên nhiều yếu tố và kinh nghiệm.
- **Tính khái quát hóa trên các quần thể khác:** Mặc dù đã được đánh giá trên tập kiểm tra, khả năng tổng quát hóa của mô hình trên các quần thể bệnh nhân khác nhau (ví dụ: từ các khu vực địa lý, dân tộc, hoặc điều kiện y tế khác) có thể cần được kiểm định và xác nhận thêm.
- **Giới hạn của Logistic Regression:** Mặc dù là mô hình dễ giải thích và hoạt động hiệu quả trong dự án này, Logistic Regression giả định mối quan hệ tuyến tính giữa các đặc trưng và log-odds. Điều này có nghĩa là nó có thể không nắm bắt được các mối quan hệ phi tuyến tính phức tạp trong dữ liệu tốt bằng các mô hình phức tạp hơn (như Neural Networks) nếu những mối quan hệ đó tồn tại.
- **Tính liên tục và cập nhật:** Các yếu tố nguy cơ đột quỵ cũng như các phương pháp điều trị y tế có thể thay đổi theo thời gian. Mô hình cần được xem xét và cập nhật định kỳ với dữ liệu mới để duy trì tính chính xác và phù hợp.

3. Hướng phát triển

Để nâng cao hơn nữa hiệu suất, tính ứng dụng và giá trị của mô hình dự đoán đột quỵ trong tương lai, có một số hướng phát triển tiềm năng:

- **Mở rộng và đa dạng hóa dữ liệu:**
 - **Tăng cường thu thập dữ liệu:** Tìm kiếm và tích hợp các bộ dữ liệu lớn hơn, đa dạng hơn về bệnh nhân và các yếu tố nguy cơ từ nhiều nguồn khác nhau (ví dụ: bệnh viện, phòng khám, nghiên cứu dịch tễ học).
 - **Bổ sung đặc trưng y tế chuyên sâu:** Xem xét tích hợp các đặc trưng y tế chi tiết hơn như kết quả xét nghiệm máu chuyên biệt (ví dụ: mức cholesterol LDL/HDL, chỉ số đường huyết HbA1c), tiền sử bệnh lý gia đình cụ thể, thông tin về lối sống (chế độ ăn uống, mức độ hoạt động thể chất), hoặc các dấu hiệu sinh học khác.
- **Khám phá các mô hình học sâu (Deep Learning):**
 - Khi có sẵn lượng dữ liệu lớn và phức tạp hơn, có thể thử nghiệm các kiến trúc mạng nơ-ron sâu (Deep Neural Networks). Các mô hình này có khả năng tự động học các đặc trưng phức tạp và phi tuyến tính, có thể mang lại hiệu suất cao hơn nữa.
 - Tuy nhiên, cần cân nhắc kỹ lưỡng về tính giải thích của các mô hình học sâu và áp dụng các phương pháp XAI nâng cao để duy trì sự minh bạch.
- **Tinh chỉnh siêu tham số và kỹ thuật xử lý mất cân bằng nâng cao:**
 - Nghiên cứu và áp dụng các phương pháp tinh chỉnh siêu tham số tiên tiến hơn ngoài GridSearchCV (ví dụ: Random Search, Bayesian Optimization, Hyperopt) để tìm kiếm hiệu quả không gian tham số.
 - Thử nghiệm các kỹ thuật xử lý mất cân bằng lớp khác ngoài SMOTE (ví dụ: Borderline-SMOTE, ADASYN, hay các chiến lược lấy mẫu hỗn hợp - hybrid sampling) để tìm ra phương pháp tối ưu nhất cho bộ dữ liệu cụ thể.
- **Tích hợp kỹ thuật Feature Engineering tiên tiến:**
 - Phát triển các đặc trưng mới từ dữ liệu hiện có hoặc kết hợp thông tin từ nhiều đặc trưng khác nhau dựa trên kiến thức y khoa để cung cấp cái nhìn sâu sắc hơn cho mô hình. Ví dụ: tạo chỉ số về nguy cơ tổng hợp dựa trên nhiều yếu tố.
- **Xây dựng hệ thống giám sát và cập nhật liên tục:**
 - Thiết lập một quy trình để giám sát hiệu suất của mô hình trong môi trường thực tế (ví dụ: trong một phòng khám hoặc bệnh viện) và tự động cập nhật mô hình định kỳ với dữ liệu mới. Điều này đảm bảo mô hình luôn được tối ưu và phản ánh các xu hướng y tế mới nhất.
- **Phát triển giao diện người dùng (User Interface):**
 - Xây dựng một giao diện thân thiện, trực quan cho phép các chuyên gia y tế dễ dàng nhập dữ liệu bệnh nhân và nhận được dự đoán nguy cơ đột quỵ cùng với giải thích từ mô hình, tạo điều kiện thuận lợi cho việc tích hợp vào quy trình làm việc lâm sàng.
- **Hợp tác đa ngành:**

Tăng cường đối thoại và hợp tác chặt chẽ với các bác sĩ, nhà nghiên cứu y học, và các chuyên gia khác trong lĩnh vực chăm sóc sức khỏe. Sự hợp tác này sẽ cung cấp cái nhìn sâu sắc về các yếu tố lâm sàng quan trọng, giúp cải thiện việc lựa chọn đặc trưng, diễn giải kết quả, và đảm bảo tính phù hợp của mô hình với thực tiễn y tế.

CHƯƠNG 5. TÀI LIỆU THAM KHẢO

[\(PDF\) Analysing an imbalanced stroke prediction dataset using machine learning techniques](#)

[Machine learning-based prediction of one-year mortality in ischemic stroke patients - PMC](#)

[Interpreting Stroke-Impaired Electromyography Patterns through Explainable Artificial Intelligence - PMC](#)

[Predicting 90-Day Prognosis in Ischemic Stroke Patients Post Thrombolysis Using Machine Learning - PMC](#)

[A comprehensive explainable AI approach for enhancing transparency and interpretability in stroke prediction | Scientific Reports](#)

[A 6 Step Field Guide for Building Machine Learning Projects](#)

[Stroke Prediction EDA and Descriptive Modeling](#)

[An intelligent learning system based on electronic health records for unbiased stroke prediction | Scientific Reports](#)

[Predicting a Stroke \[SHAP, LIME Explainer & ELI5\]](#)